Machine Learning

Unit 1

- Introduction to Machine Learning
- Concept of the learning task, inductive learning, and the concepts of hypothesis space
- Introduction to different types of machine learning approaches, examples of machine learning applications
- Different types of learning: supervised learning, unsupervised learning, reinforcement learning.
- Feature Extraction- Setting up your machine learning platform: training, validation and testing, overfitting and under-fitting, different types of error calculation.

What is Learning?

"To gain **knowledge** or **understanding** of, or **skill** in **by study**, **instruction** or **experience**"

- Learning a set of new facts.
- Learning HOW to do something .
- Improving ability of something already learned.

Machine Learning

- Term popularized by Arthur Samuel in 1959.
- Sub-part of Artificial Intelligence that uses data to teach machines.
- Field of study that gives computers the ability to learn without being explicitly programmed.
- Building machines that automatically learn from experience.
- Machine learning use the theory of statistics in building mathematical models, because the core task is making inference from a sample.

Machine Learning

Humans learn from experience and adapt their actions for future tasks.

Can machines adapt their behavior using experience?



Human can learn from past experience
and make decision of its own

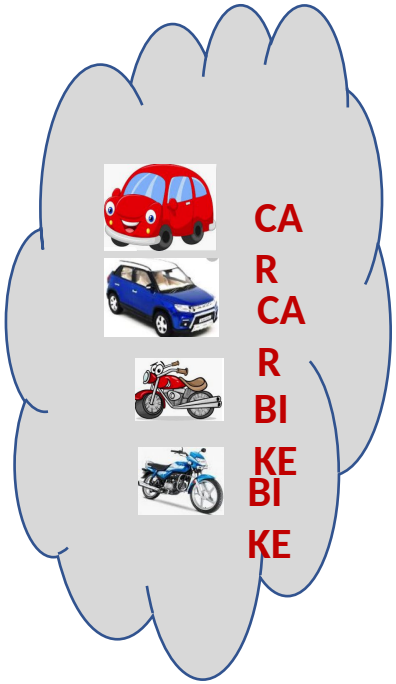


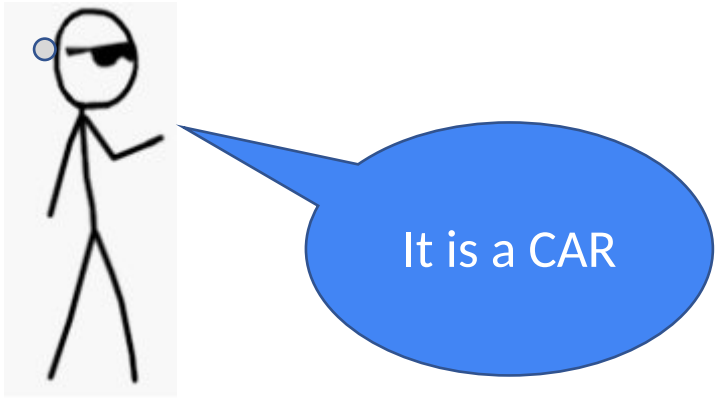
What is this object?





What is this object?





Let us ask the same question to him



What is this object?



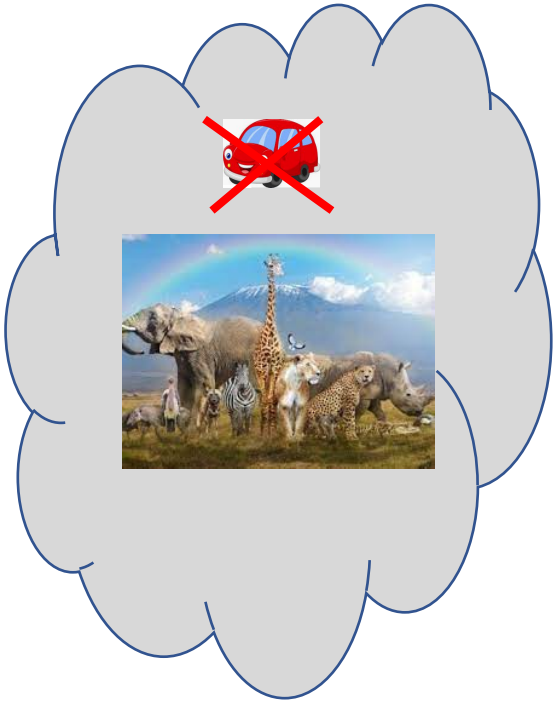
Let us ask the same question to him



What is this object?







[But, he is a human being. He can observe and learn]

Let us make him learn



show
him









Let us make him learn



show him



CAR



CAR



BIKE



BIKE

Let us ask the same question now



What is this object?





CAR



CAR



BIKE



BIKE

Past experience

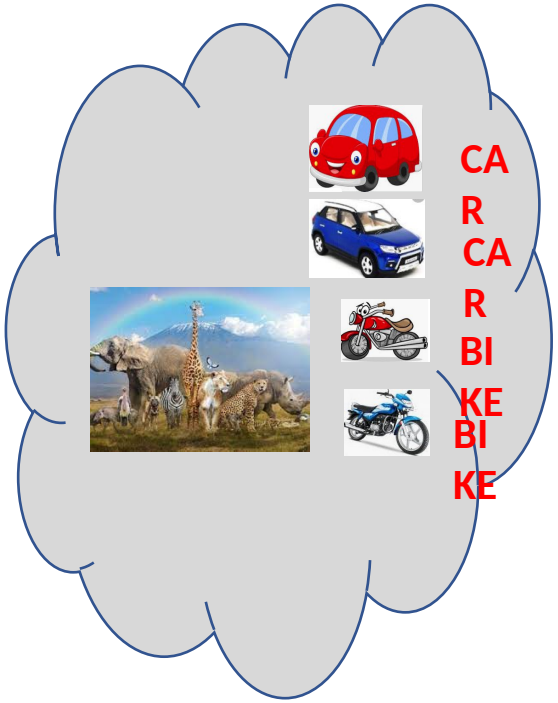
Let us ask the same question now

CAR

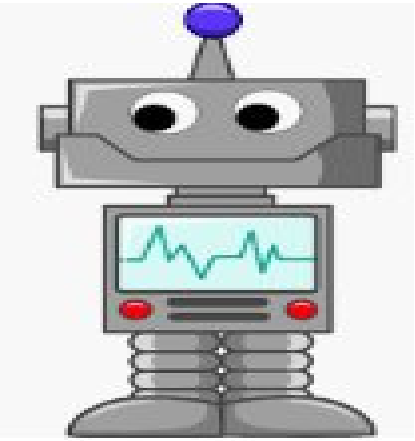
What is this object?







What about a Machine ?



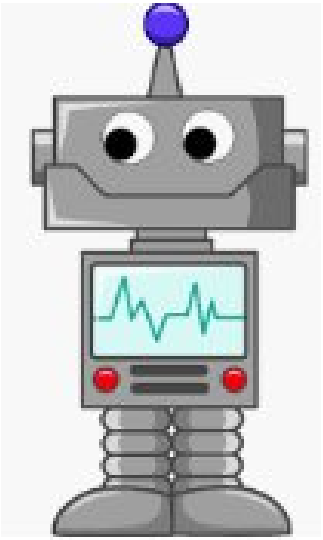
Machines follow instructions

It can not take decision of its own

What about a Machine ?

We can ask a machine

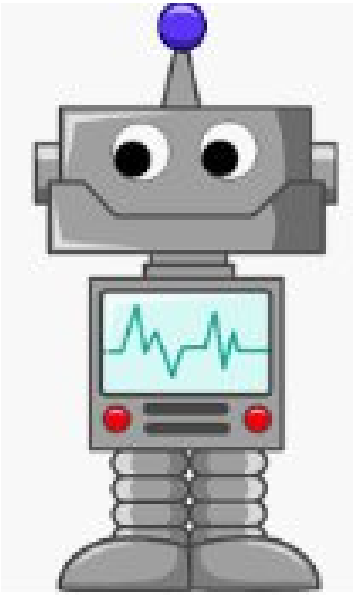
- To perform an arithmetic operations such as
  - Addition
  - Multiplication
  - Division



Machines follow instructions

What about a Machine ?

- Comparison
- Print
- Plotting a chart



Machines follow instructions

But, we can ask a machine to make a decision of its own

What is Machine Learning?

We want a machine to act like a human

What is Machine Learning?



[to identify this object.]



What is Machine Learning?



[predict the price in future]



Price in 2025?

What is Machine Learning?



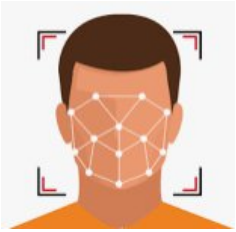
I ~~made~~ met him yesterday

[Natural Language understand, and correct grammar]

What is Machine Learning?



[Recognize Faces]



What is Machine Learning?



Just like, what we did to human,

we need to provide experience to the machine.

What is Machine Learning?



+



Dataset

This what we called as Data or Training dataset

So, we first need to provide training dataset to the machine

What is Machine Learning?

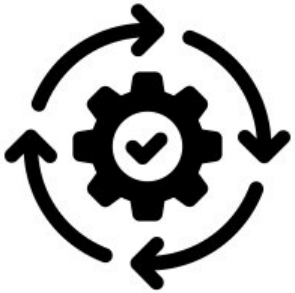


+



Dataset

+



[Then, devise algorithms and execute programs on the data

With respect to the underlying target tasks]

What is Machine Learning?

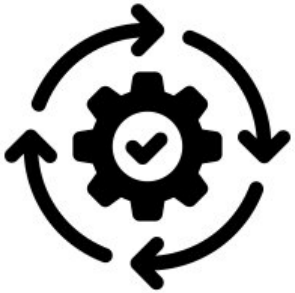


+

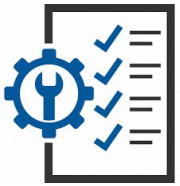


Datase
t

+

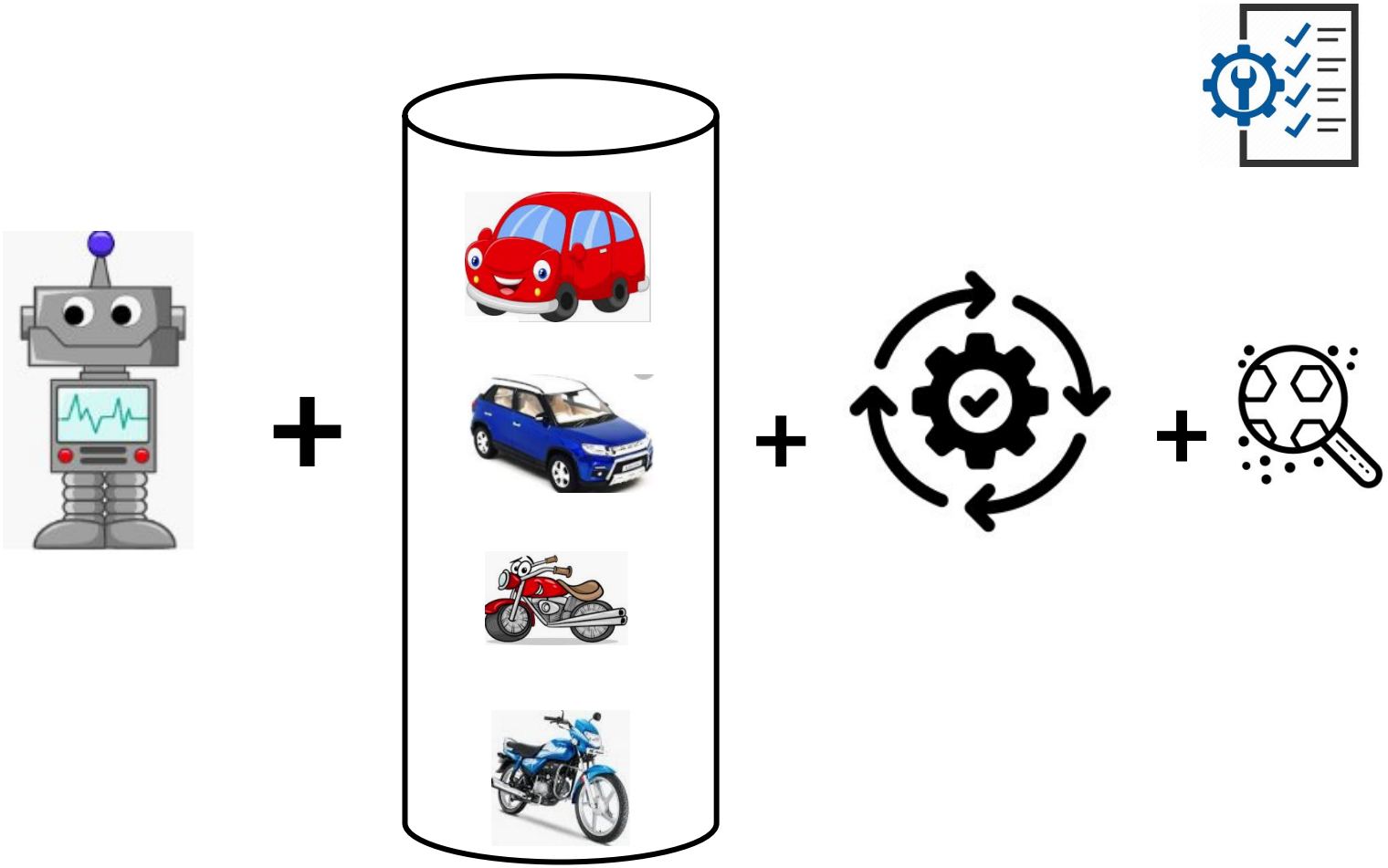


+



[Then, using the programs,
Identify required rules]

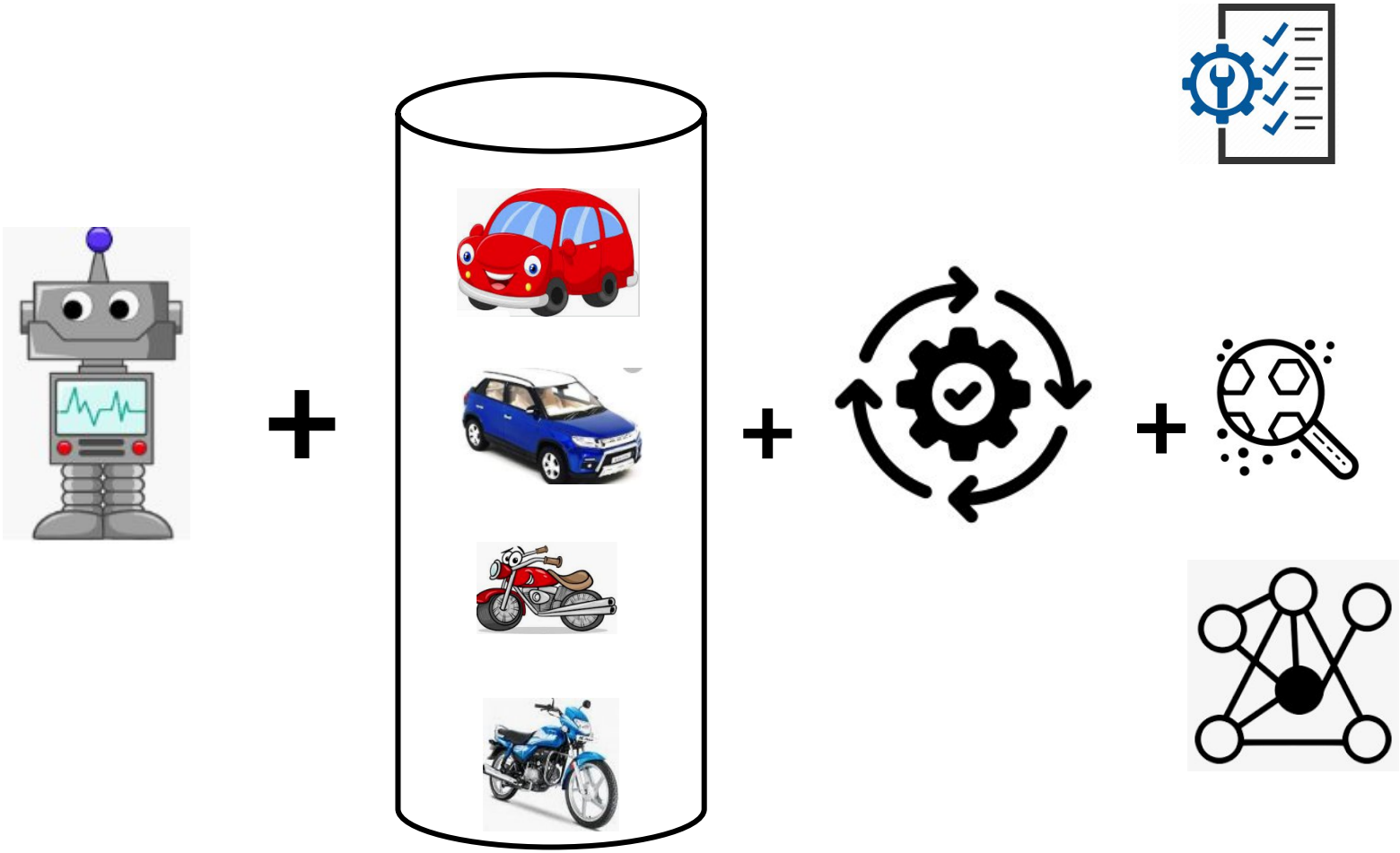
What is Machine Learning?



Datase
t

[extract required patterns]

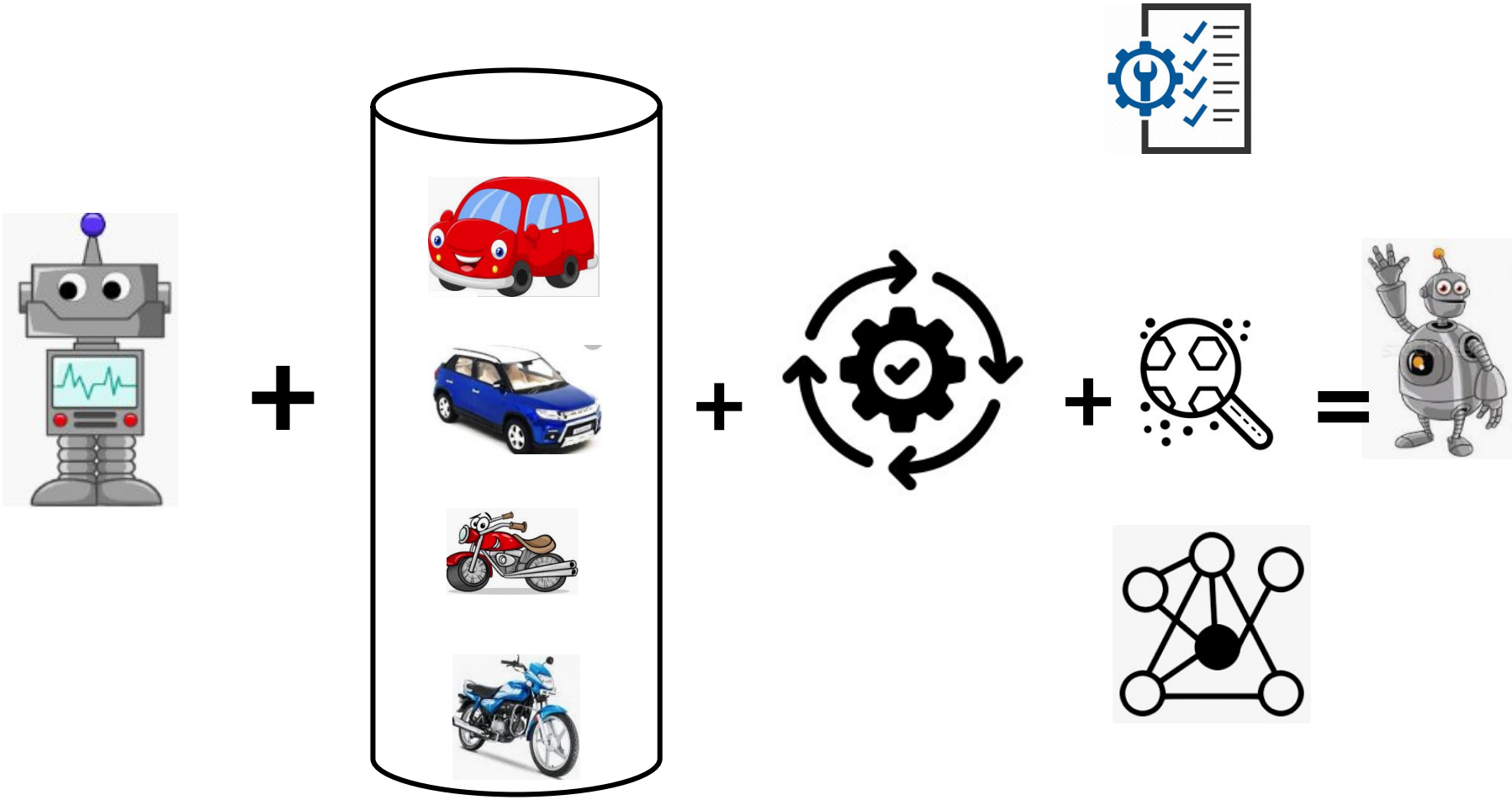
What is Machine Learning?



Dataset

[Identify relations]

What is Machine Learning?



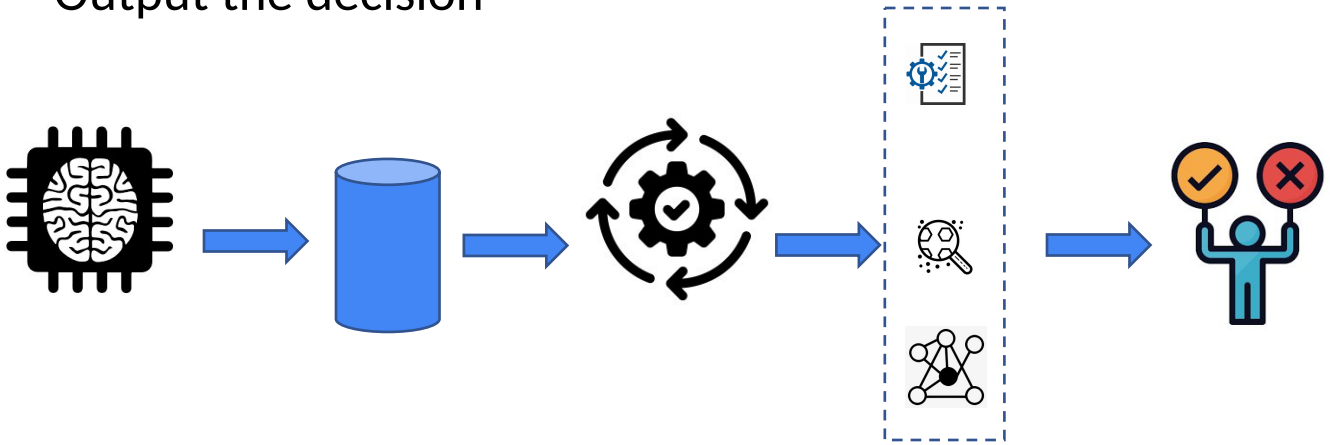
Datase
t

[So that machine can derive inferences from the data]

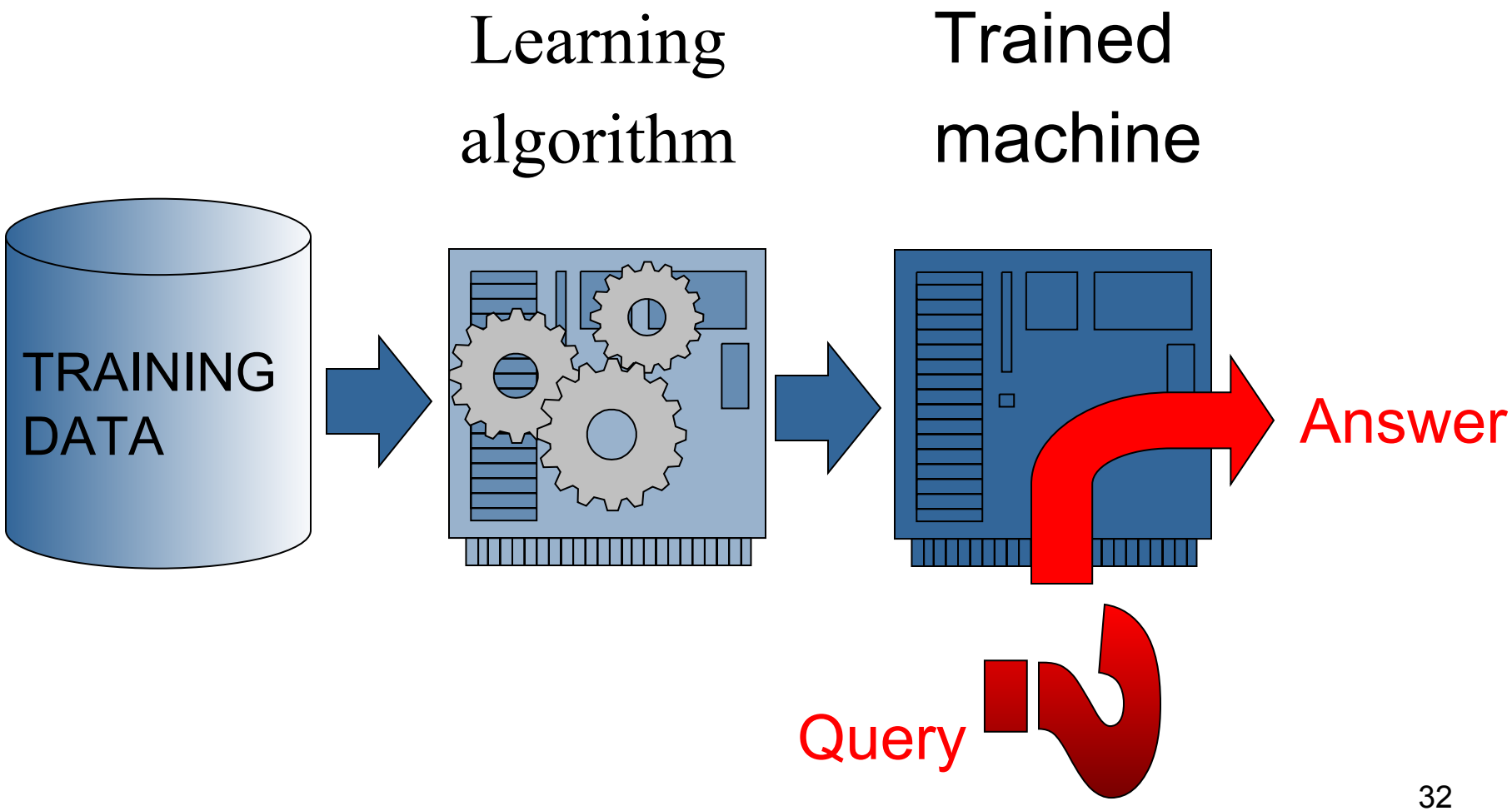
In summary, what is machine learning?

Given a machine learning problem

- Identify and create the appropriate dataset
- Perform computation to learn
  - Required rules, pattern and relations
- Output the decision

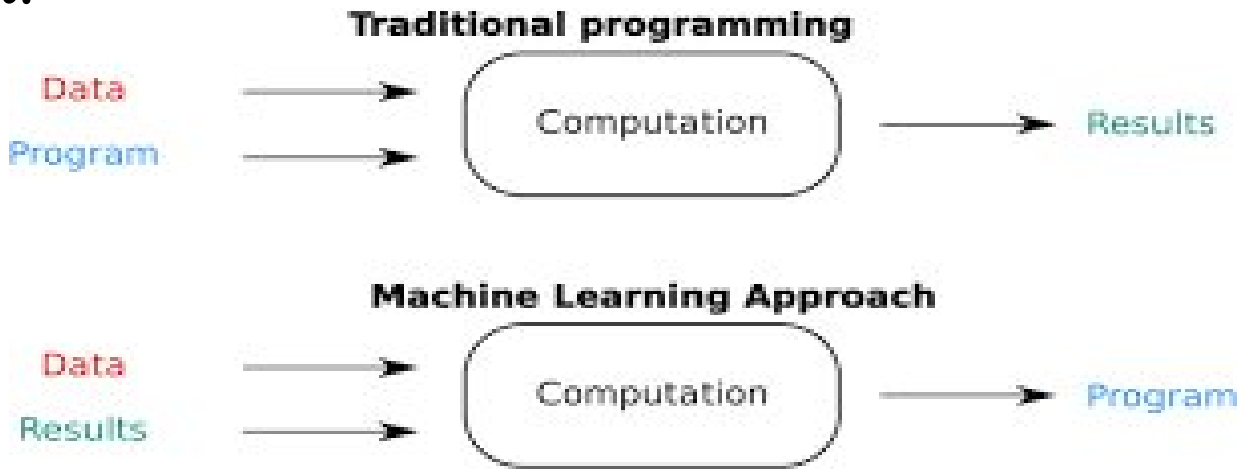


What is Machine Learning?



Machine Learning

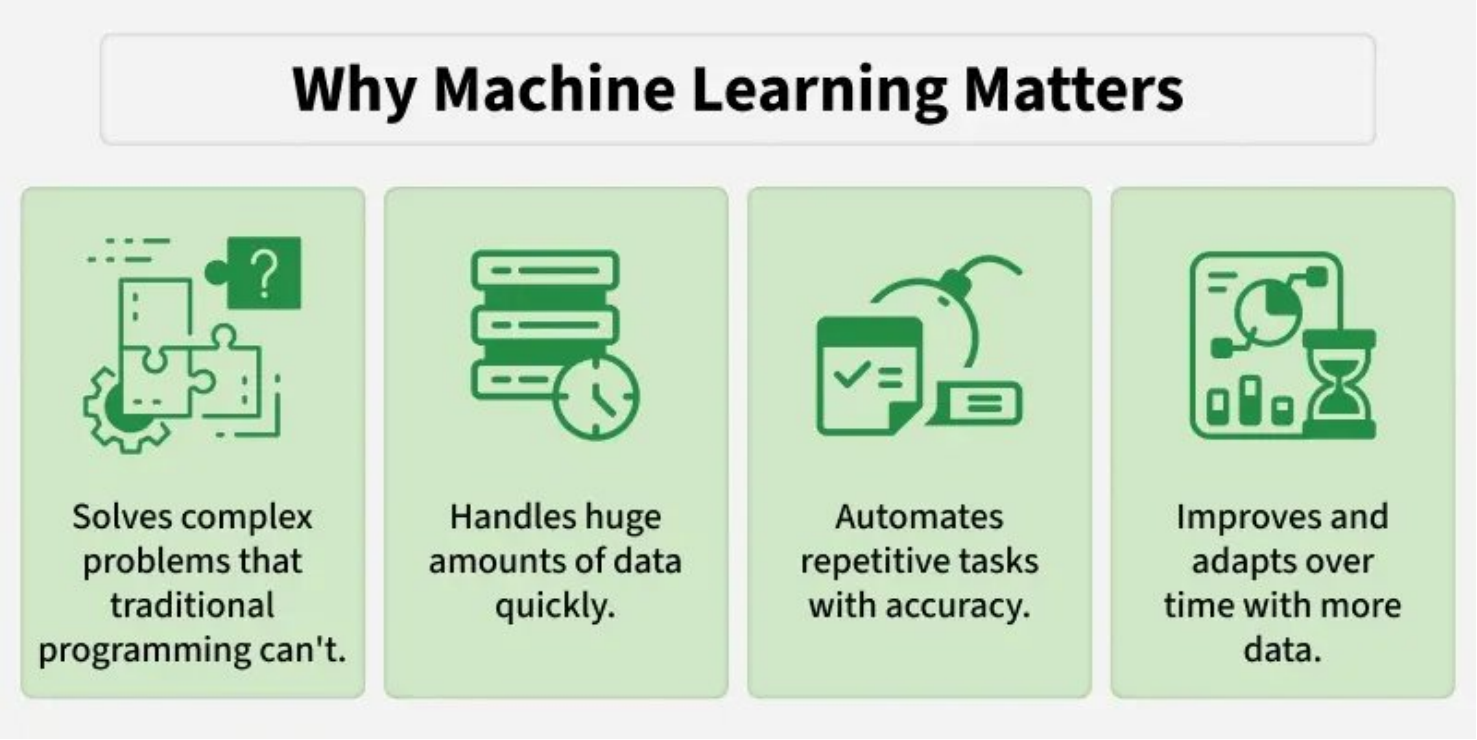
- Machine Learning system learns from historical data (training data) , builds prediction models and whenever it receives new data, predicts the output for it.



Need for Machine Learning

Machine Learning is important because traditional programming cannot handle complex tasks or large amounts of data efficiently.

ML overcomes this by learning from data and making predictions without fixed rules.



Need for Machine Learning

1. Solving Complex Business Problems

Traditional programming struggles with tasks like language understanding and medical diagnosis. ML learns from data and predicts outcomes easily.

Examples:

- Image and speech recognition in healthcare.
- Language translation and sentiment analysis.

Need for Machine Learning

2. Handling Large Volumes of Data

The internet generates huge amounts of data every day. Machine Learning processes and analyzes this data quickly by providing valuable insights and real time predictions.

Examples:

- Fraud detection in financial transactions.
- Personalized feed recommendations on Facebook and Instagram from billions of interactions.

Need for Machine Learning

3. Automate Repetitive Tasks

ML automates time consuming, repetitive tasks with high accuracy hence reducing manual work and errors.

Examples:

- Gmail filtering spam emails automatically.
- Chatbots handling order tracking and password resets.
- Automating large scale invoice analysis for key insights.

Need for Machine Learning

4. Personalized User Experience

ML enhances user experience by tailoring recommendations to individual preferences. It analyze user behavior to deliver highly relevant content.

Examples:

- Netflix suggesting movies and TV shows based on our viewing history.
- E-commerce sites recommending products we're likely to buy.

Need for Machine Learning

5. Self Improvement in Performance

ML models evolve and improve with more data helps in making them smarter over time. They adapt to user behavior and increase their performance.

Examples:

- Voice assistants like Siri and Alexa learning our preferences and accents.
- Search engines refining results based on user interaction.
- Self driving cars improving decisions using millions of miles of driving data.

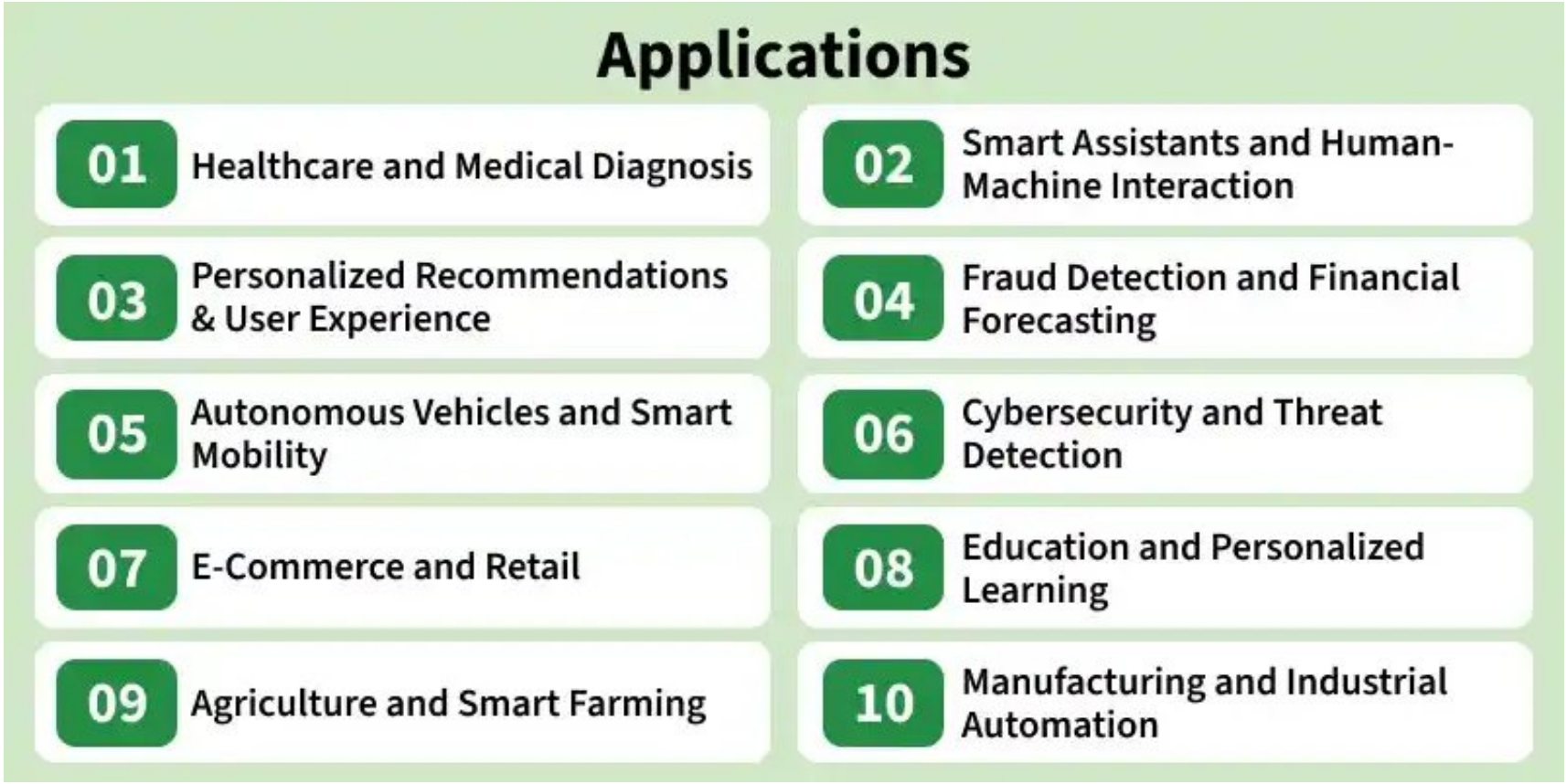
Advantages of ML

1. Machine Learning improves processes by automating tasks and extracting insights from data, making systems smarter and more efficient.
2. Automates repetitive tasks and improves productivity
3. Finds patterns in large data for better decisions
4. Provides customized recommendations and experiences
5. Enables robots and systems to perform complex tasks accurately

Disadvantages of ML

1. Application specific algorithms.
2. Real world problems have too many variables and sensors might be too noisy.
3. Computational complexity.
4. ML models learn from training data and if the data is biased, model's decisions can be unfair so it's important to select and monitor data carefully.
5. Since it depends on large amounts of data, there is a risk of sensitive information being exposed so protecting privacy is important.
6. Complex ML models can be difficult to understand which makes it difficult to explain why they make certain decisions. This can affect trust and accountability.
7. Automation may replace some jobs so retraining and helping workers learn new skills is important to adapt to these changes.

Applications of Machine Learning



Applications of Machine Learning

1. Healthcare and Medical Diagnosis

ML algorithms can analyze large volumes of patient data, medical scans and genetic information to aid in diagnosis and treatment.

- **Disease Detection**: ML models are used to identify diseases like cancer, pneumonia and Parkinson's from medical images. They often achieve accuracy comparable to or better than human doctors.
- **Predictive Analytics**: By analyzing patient history and symptoms, models can predict the risk of certain diseases or potential complications.
- **Drug Discovery**: ML accelerates the drug development process by predicting how different compounds will interact, reducing the time and cost of research.

Applications of Machine Learning

2. Smart Assistants and Human-Machine Interaction

Virtual assistants systems rely on natural language processing (NLP) and speech recognition to understand commands and respond intelligently.

- **Voice Assistants**: Tools like Siri, Alexa and Google Assistant convert spoken input into actionable commands.
- **Voice Search & Transcription**: ML enables users to perform hands-free web searches and get transcription during meetings or phone calls.
- **Chatbots**: Businesses use AI-powered chatbots for 24/7 customer support, helping resolve queries faster and more efficiently.

Applications of Machine Learning

3. Personalized Recommendations and User Experience

Modern digital platforms uses personalization which is done by using **recommender systems. M**achine learning models analyze user behavior to deliver relevant content, improving engagement and satisfaction.

- **Streaming Platforms**: Netflix and Spotify suggest shows and songs based on your watching or listening history.
- **E-commerce**: Sites like Amazon recommend products tailored to your preferences, browsing patterns and past purchases.
- **Social Media**: Algorithms curate content feeds, prioritize posts and suggest friends or pages.

These systems use techniques like collaborative filtering and content-based filtering to create personalized digital experiences.

Applications of Machine Learning

4. Fraud Detection and Financial Forecasting

In finance, vast sums of money move digitally and machine learning plays an important role in fraud detection and market analysis.

- **Transaction Monitoring**: Banks use ML models to detect unusual spending behavior and flag suspicious transactions.
- **Loan Risk Assessment**: Credit scoring models analyze customer profiles and predict the likelihood of default.
- **Stock Market Prediction**: ML is used to analyze historical stock data and forecast price movements. Stock markets are complex, algorithmic trading uses these predictions for better decision-making.

Applications of Machine Learning

5. Autonomous Vehicles and Smart Mobility

Self-driving vehicles use ML to understand their environment, navigate safely and make immediate decisions.

- **Computer Vision**: Recognizing lanes, pedestrians, traffic signals and obstacles.
- **Sensor Fusion**: Combining data from cameras, LiDAR and radar for a 360-degree view.
- **Behavior Prediction**: Anticipating how other drivers or pedestrians may act.

Autonomous vehicles are capable of operating with minimal human input. Beyond cars, ML is also being used in traffic optimization, smart navigation systems and predictive maintenance in transportation.

Applications of Machine Learning

6. Cybersecurity and Threat Detection

Machine learning helps identify cyber threats, detect suspicious activities, and improve digital security systems by analyzing network behavior and user activity patterns.

- **Spam and Phishing Detection:** ML filters detect spam emails, phishing links, and malicious attachments.
- **Intrusion Detection:** Security systems monitor unusual network activity to identify cyberattacks in real-time.
- **Malware Detection:** ML models analyze software behavior to detect malicious programs and ransomware.

Applications of Machine Learning

7. E-Commerce and Retail

E-commerce platforms use machine learning to optimize customer experience, inventory management, and sales strategies.

- **Demand Forecasting:** Predicts product demand to improve inventory planning and reduce shortages.
- **Dynamic Pricing:** Adjusts product prices based on demand, competition, and customer behavior.
- **Customer Segmentation:** Groups customers based on interests and purchasing behavior for targeted marketing.

Applications of Machine Learning

8. Education and Personalized Learning

Machine learning improves learning experiences by adapting educational content to individual student needs.

- **Personalized Learning:** Recommends customized study materials and learning paths.
- **Automated Grading:** Evaluates quizzes, assignments, and objective answers automatically.
- **Student Performance Prediction:** Identifies students who may need additional academic support.

Applications of Machine Learning

9. Agriculture and Smart Farming

ML-powered systems help farmers improve productivity, monitor crops, and optimize agricultural resources.

- **Crop Disease Detection:** Identifies plant diseases using image analysis and computer vision.
- **Yield Prediction:** Predicts crop production using weather and soil data.
- **Smart Irrigation:** Optimizes water usage based on environmental conditions and crop requirements.

Applications of Machine Learning

10. Manufacturing and Industrial Automation

Machine learning enhances industrial efficiency through predictive analysis and automation.

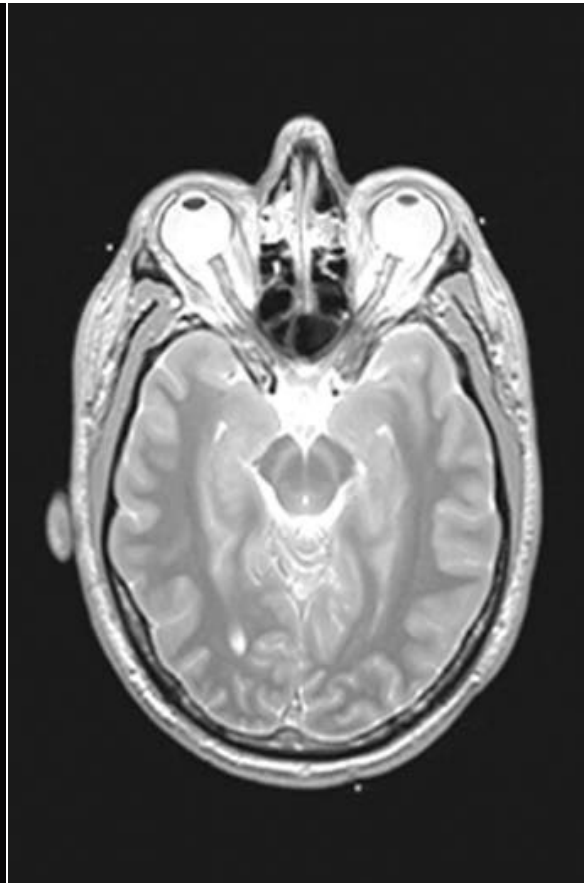
- **Predictive Maintenance:** Detects equipment issues before failures occur.
- **Quality Inspection:** Uses computer vision to identify defects in manufactured products.
- **Robotics Automation:** Enables intelligent robots to perform repetitive industrial tasks efficiently

Drug discovery



Medical diagnosis





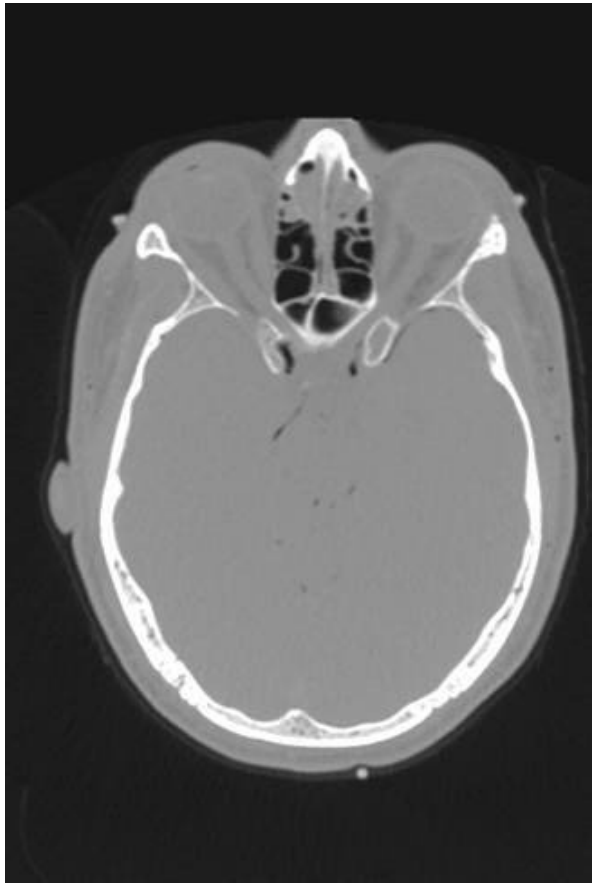
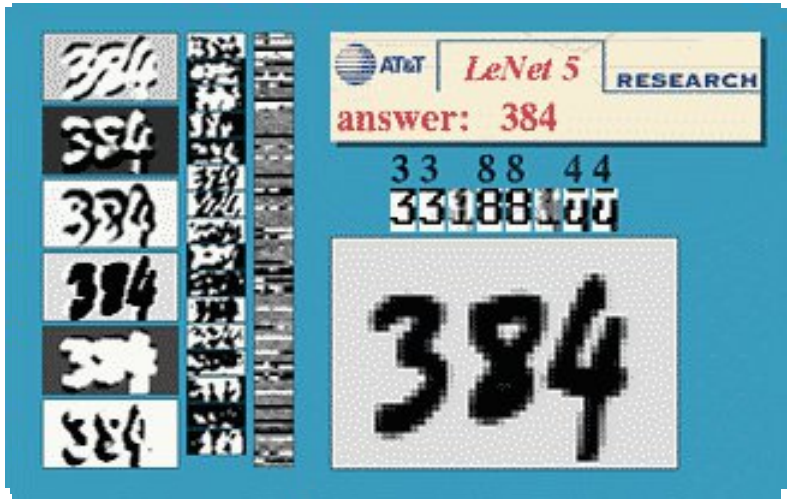


Photo MRI CT

Iris verification



Hand-written digits



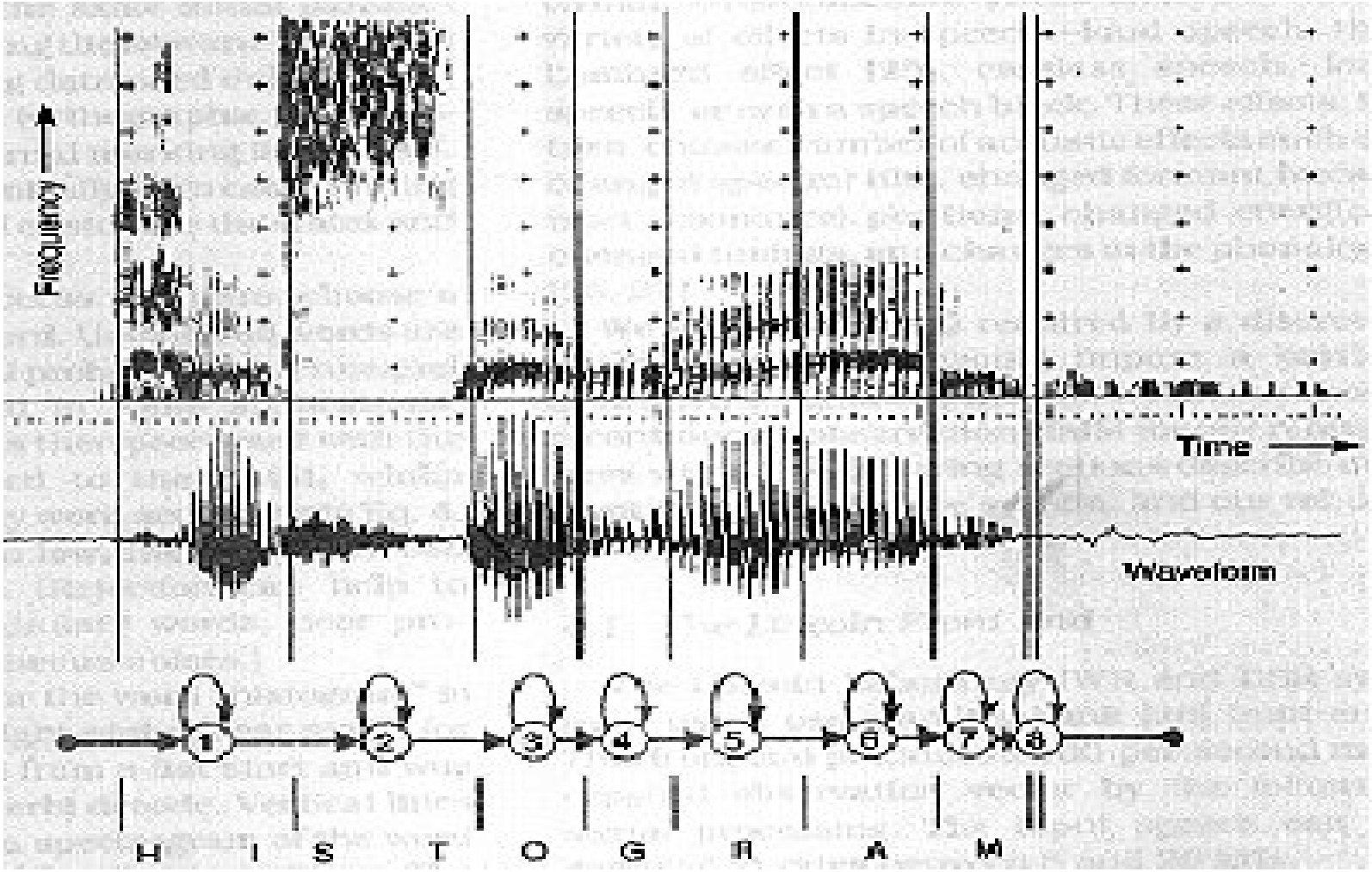




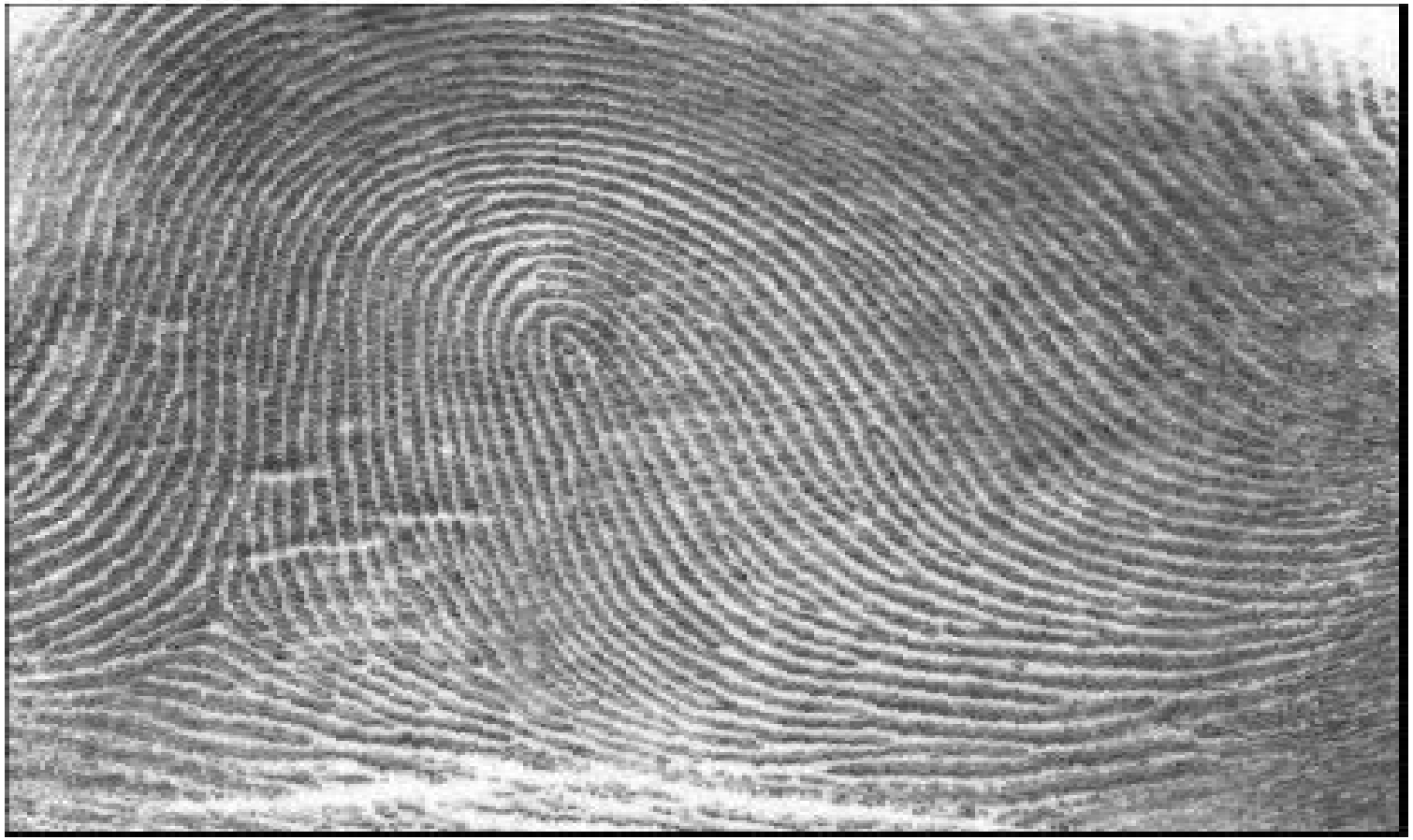
Radar Imaging



Speech Recognition

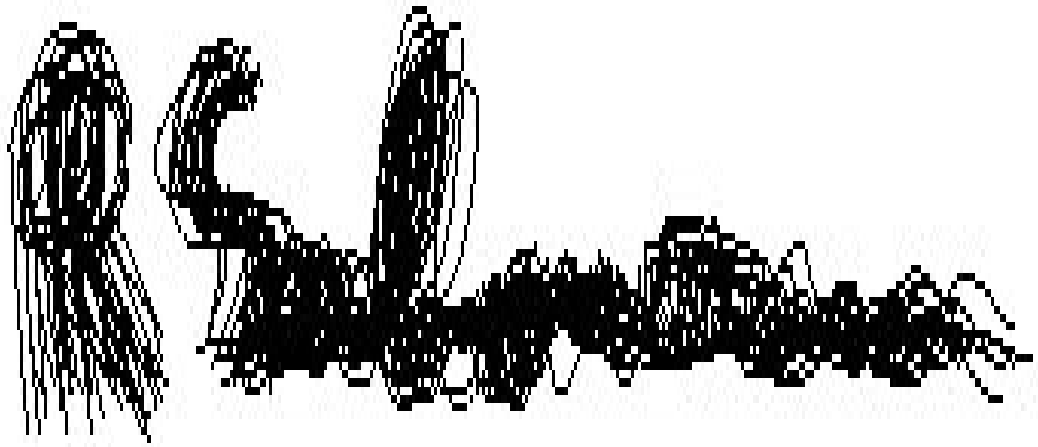


Finger print



fingerprint image

Signature Verification

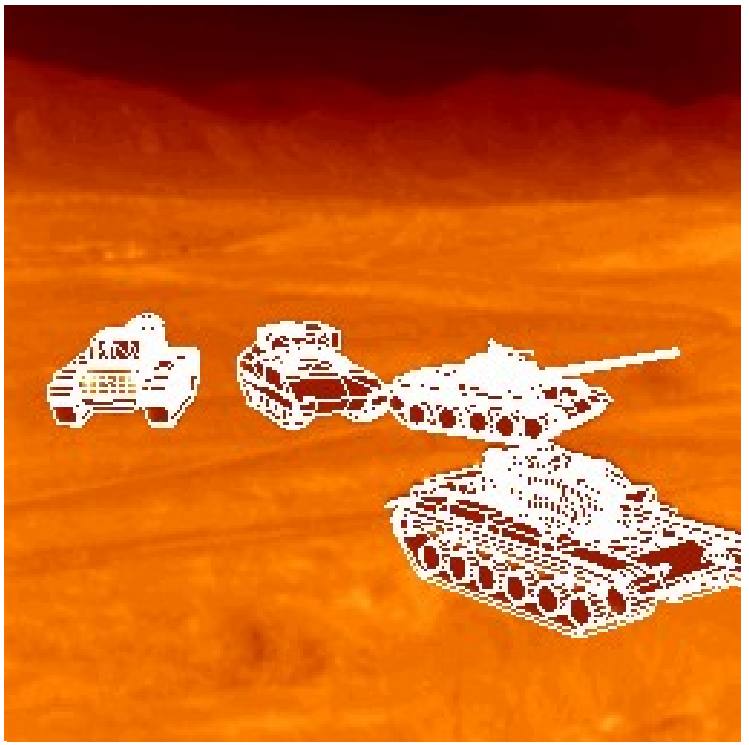


Face Recognition



Target Recognition

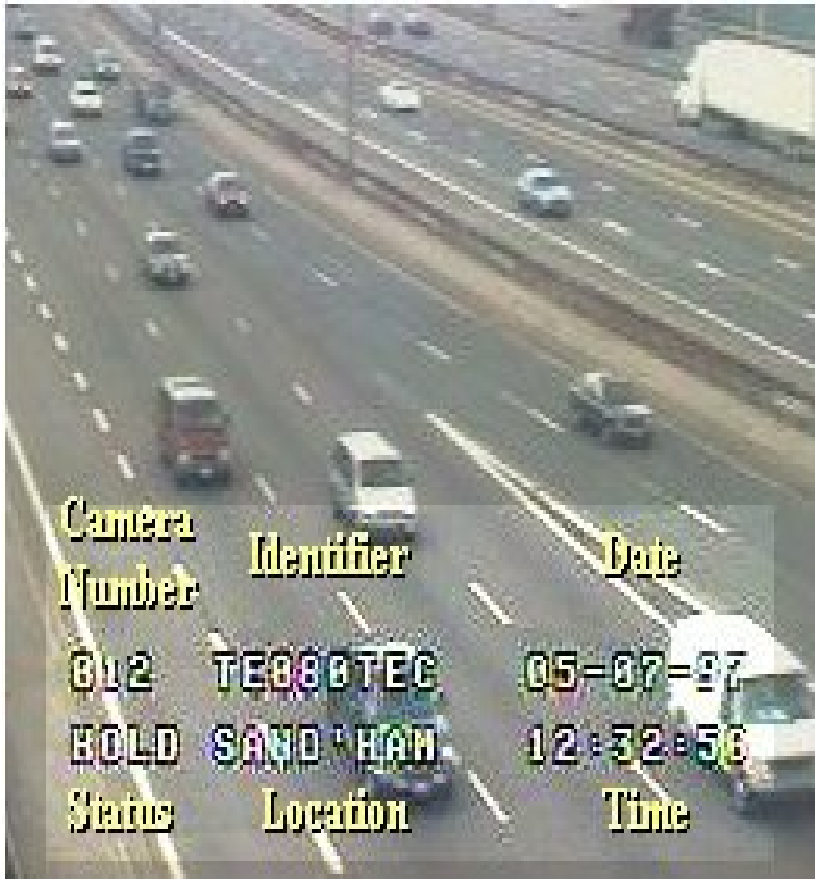


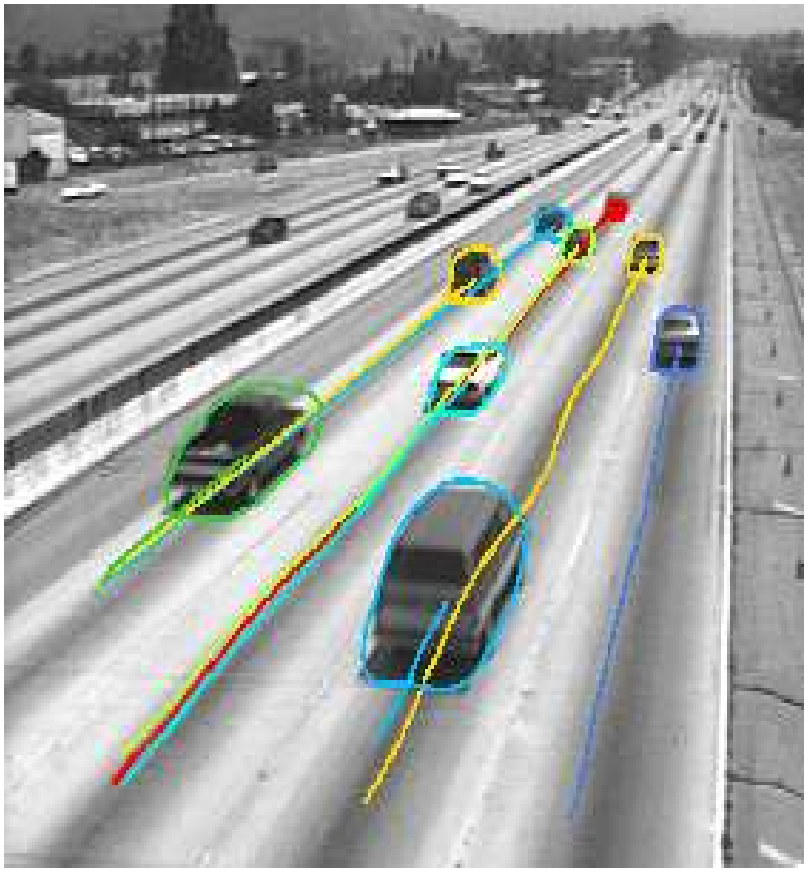


Robotics vision



Traffic Monitoring





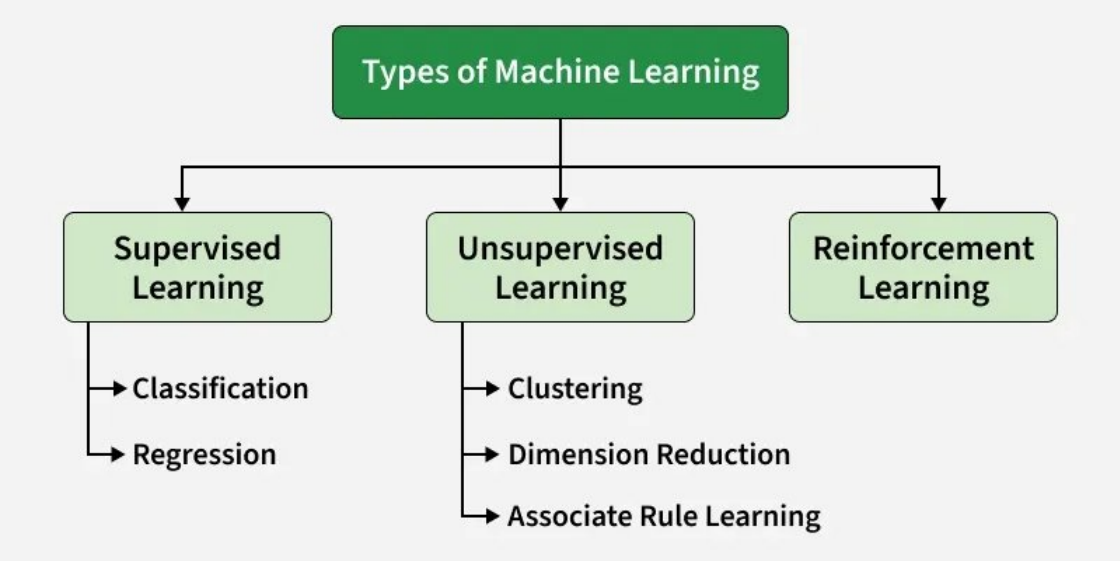
Types of Machine Learning

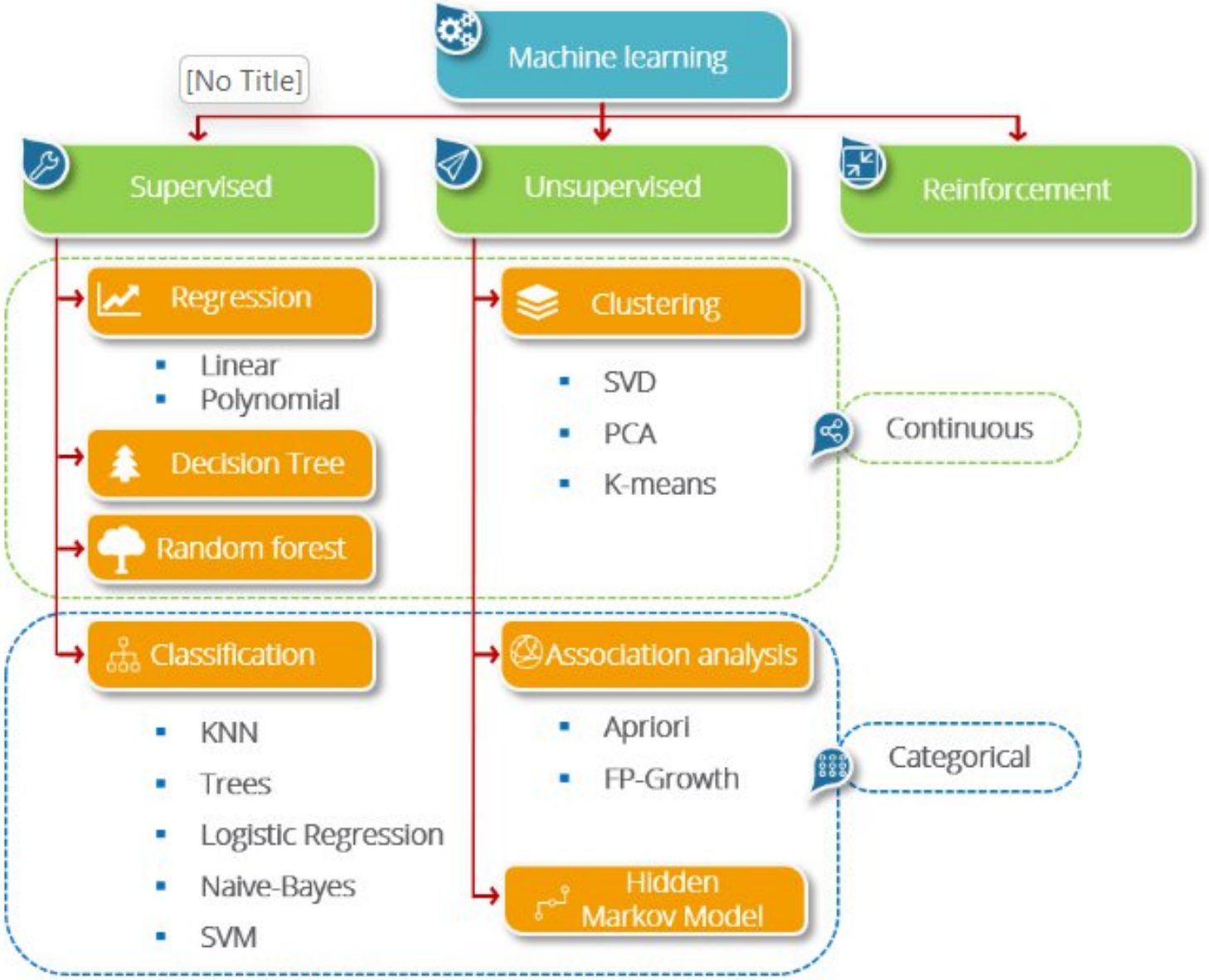
1. **Supervised** learning
   A series of examples are used for feedback

1. **Unsupervised** learning
   No feedback

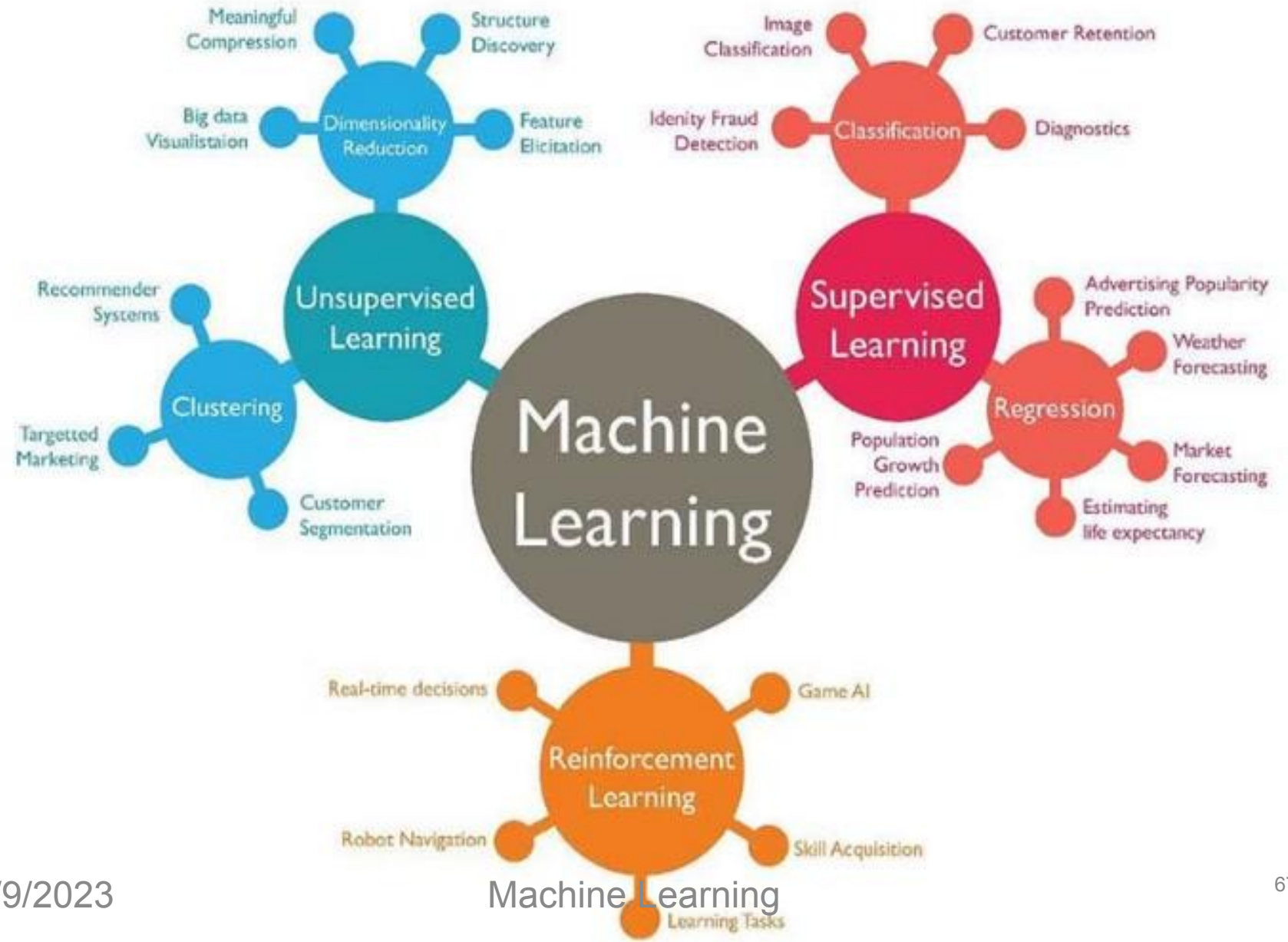
1. **Reinforcement** learning
   Indirect feedback after experience

Additionally, there is a more specific category called Semi-Supervised Learning and Self-Supervised Learning, which combines elements of both supervised and unsupervised learning




[No Title]
Machine learning
Supervised
Unsupervised
Reinforcement
Regression
Linear
Polynomial
Decision Tree
Random forest
Clustering
SVD
PCA
K-means
Continuous
Classification
KNN
Trees
Logistic Regression
Naive-Bayes
SVM
Association analysis
Apriori
FP-Growth
Categorical
Hidden Markov Model

Learning


Machine Learning
Unsupervised Learning
Dimensionality Reduction
Meaningful Compression
Structure Discovery
Big data Visualistaion
Feature Elicitation
Clustering
Recommender Systems
Targetted Marketing
Customer Segmentation
Supervised Learning
Classification
Image Classification
Customer Retention
Idenity Fraud Detection
Diagnostics
Regression
Advertising Popularity Prediction
Weather Forecasting
Population Growth Prediction
Market Forecasting
Estimating life expectancy
Reinforcement Learning
Real-time decisions
Game AI
Robot Navigation
Skill Acquisition
Learning Tasks

Supervised Learning

- A learning task involves a set of input variables and a set of output variables.
- The set of possible relationships (hypotheses) between input and output variables is known as the **hypothesis space**.
- Hypothesis have representations such as numerical functions, symbolic rules, decision trees and artificial neural nets.
- The learning algorithm attempts to find the best hypothesis that maps input to output using *"feedback"*.
- **Feedback** consists of a set of points (training data) for which values of input and output variables are known.

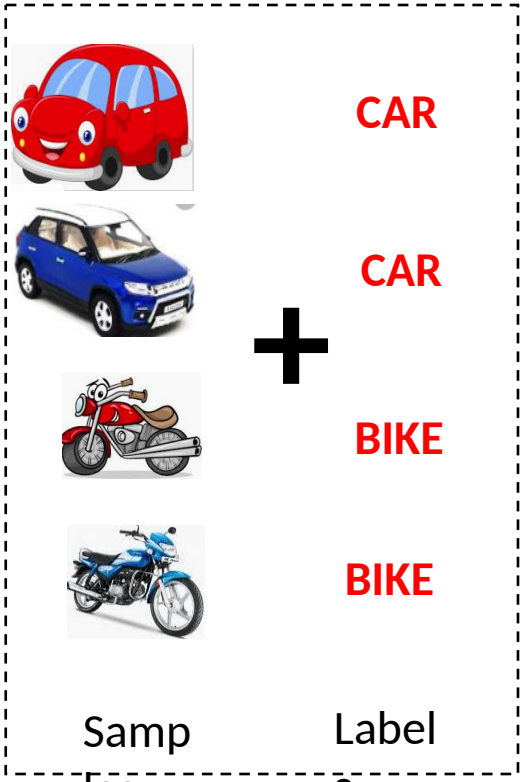
Unsupervised Learning

- In **unsupervised learning**, output variables are not known.
- Unsupervised learning algorithms identify trends in data and make inferences without knowledge of correct answers.

Reinforcement Learning

- It is a feedback-based machine learning technique, whereby an agent learns to behave in an environment by observing his mistakes and performing the actions.
- Reinforcement learning applies the method of learning via Interaction and feedback
- Reinforcement learning trains an agent to make a sequence of decisions through trial and error. The agent interacts with the environment, receives feedback in the form of rewards or penalties and learns optimal actions over time.

What is Supervised Learning?



+

=

Training Dataset

In supervised learning, we need a Labelled Training Dataset

What is Supervised Learning?



= Training Dataset

$f($  $)=$

Given a labelled dataset, the task is to devise a function which takes the dataset, and a new sample, and produces an output value.

What is Supervised Learning?



= Training Dataset

$f($  ,  $)=$

Given a labelled dataset, the task is to devise a function which takes the dataset, and a new sample, and produces an output value.

What is Supervised Learning?



= Training Dataset

$f($  $)=$ CAR

[Given a labelled dataset, the task is to devise a function which takes the dataset, and a new sample, and produces an output value.]

Classification



= Training Dataset

Classification

$f($  ,  $)$ = CAR

If the possible output values of the function are predefined and discrete/categorical, it is called Classification

Supervised Learning



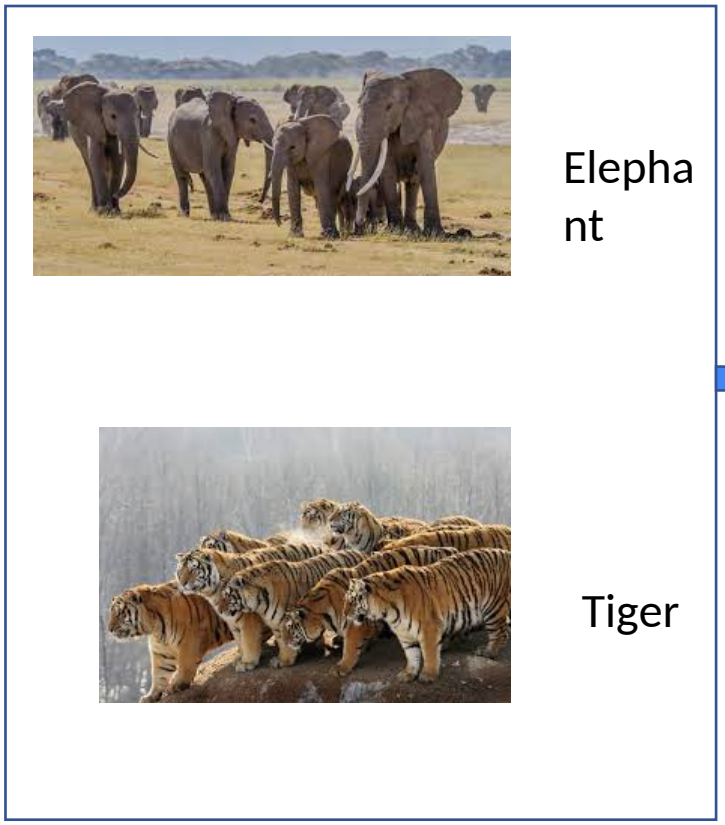
= Training Dataset

Classification

$f($  ,  $)$ = CAR

Predefined classes means, it will produce output only from the labels defined in the dataset. For example, even if we input a bus, it will produce either CAR or BIKE

Classifier

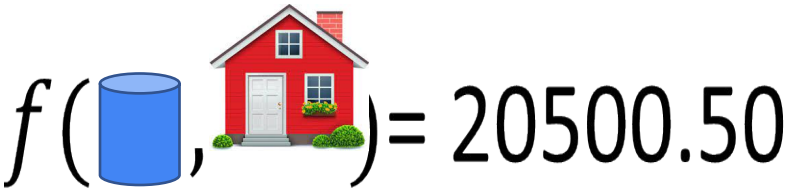

Elepha
nt
Tiger
Dataset
Elepha
nt
Classifie
r
Identify the
Animal ?

Regression



Dataset

Regression



If the possible output values of the function are continuous real values, then it is called Regression

The classification and Regression problems are supervised, because the decision depends on the characteristics of the ground truth labels or values present in the dataset, which we define as experience

Supervised Learning- Summary

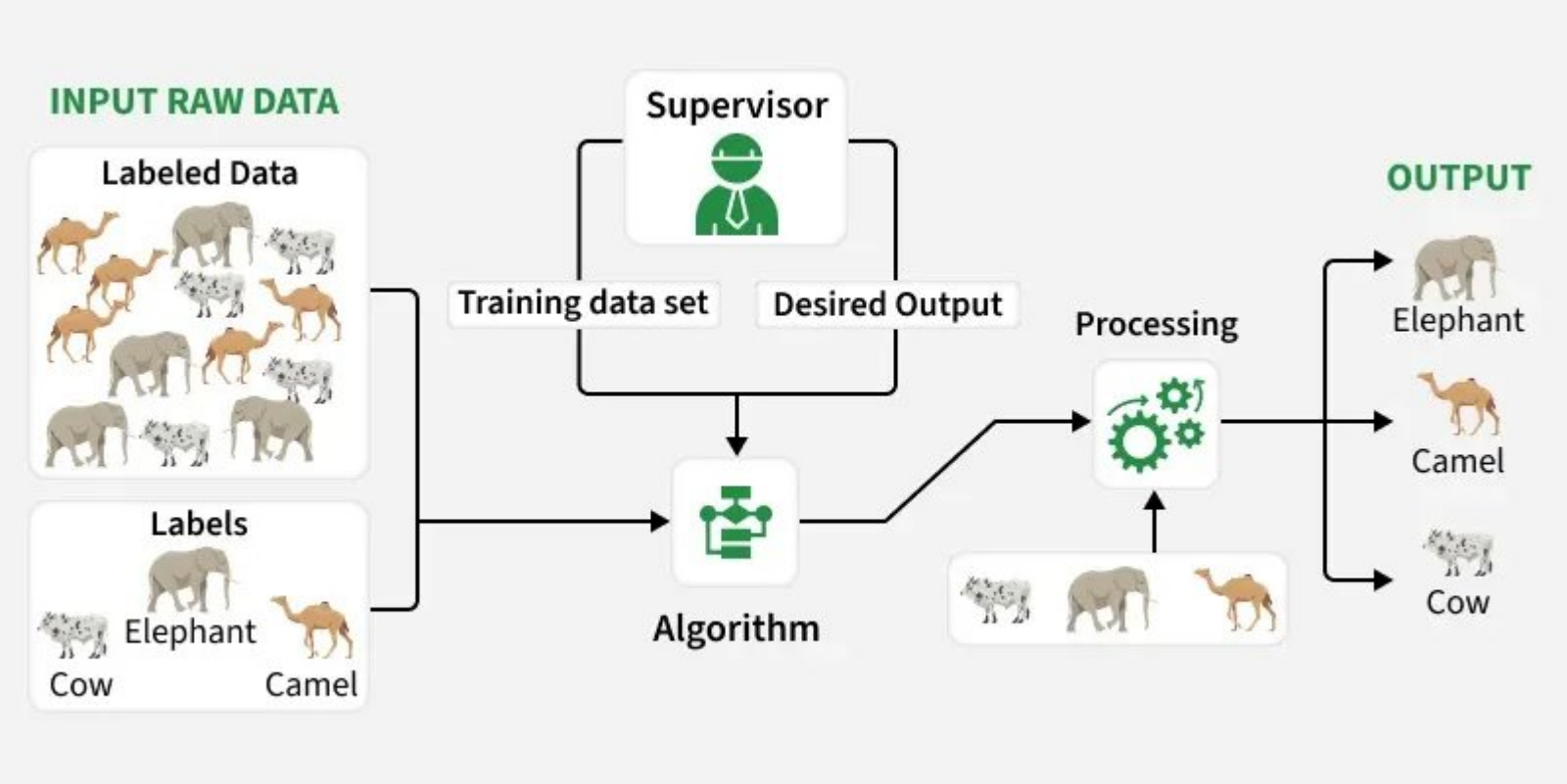
Machines are trained using labeled data to predict results

- Labeled data ➔ group of data tagged with one or more names, and the data is already known to machines.
- The algorithm measures its accuracy through the loss function, adjusting until the error has been sufficiently minimized.

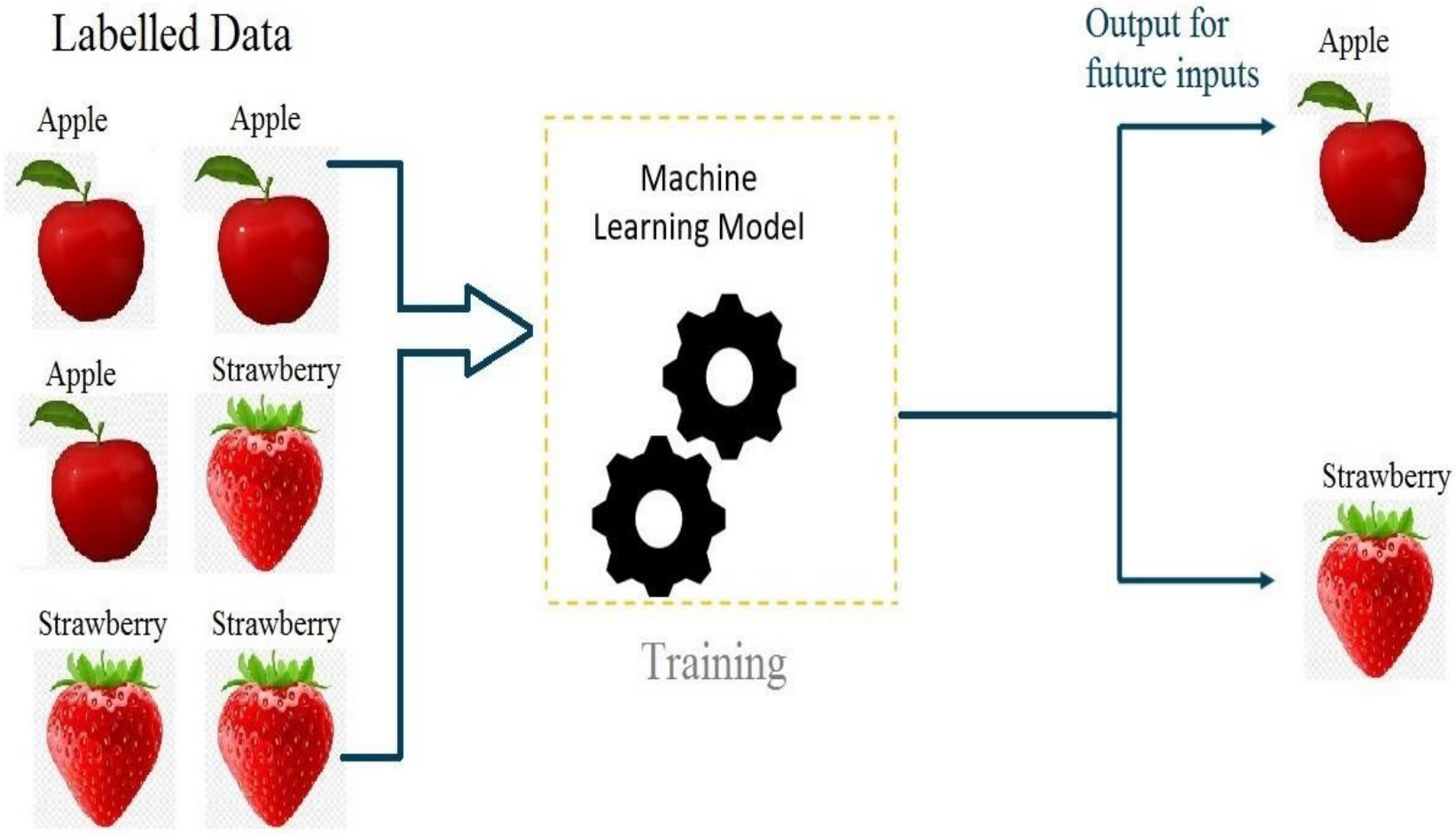
This is similar to a teacher-student scenario.

- There is a teacher who guides the student to learn from books and other materials.
- The student is then tested and if correct, the student passes.
- Else, the teacher tunes the student and makes the student learn from the mistakes that he or she had made in the past.

In Supervised Learning algorithms learn to map points between inputs and correct outputs. It has both training and validation datasets labelled.



Supervised learning



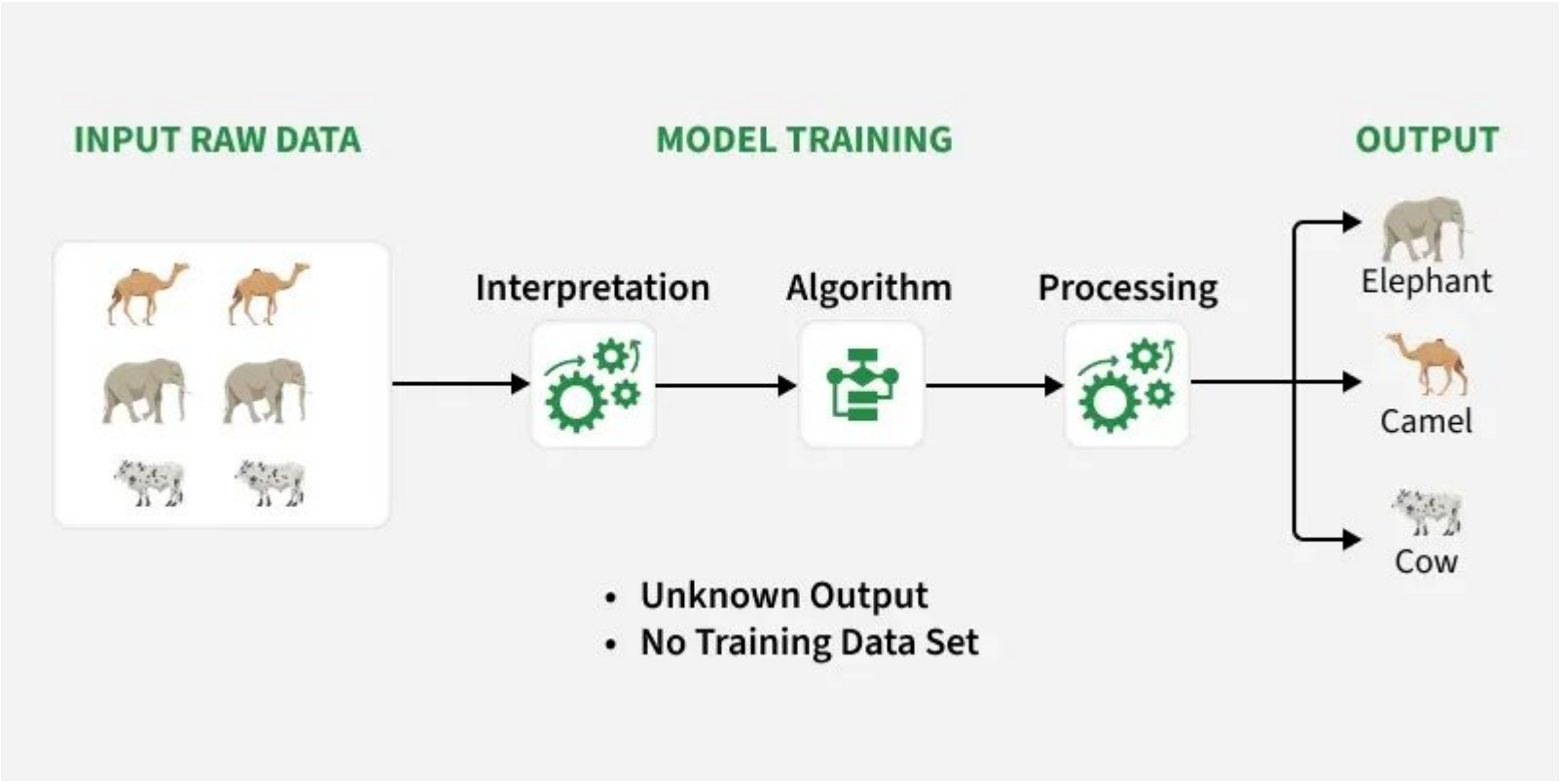
Unsupervised Learning

- Unsupervised Learning works with unlabeled data, meaning there are no predefined outputs. The algorithm finds hidden patterns, groups or relationships within the data on its own.
- Eg : A student who has textbooks and all the required material to study but has no teacher to guide. Ultimately, the student will have to learn by himself or herself to pass the exams.

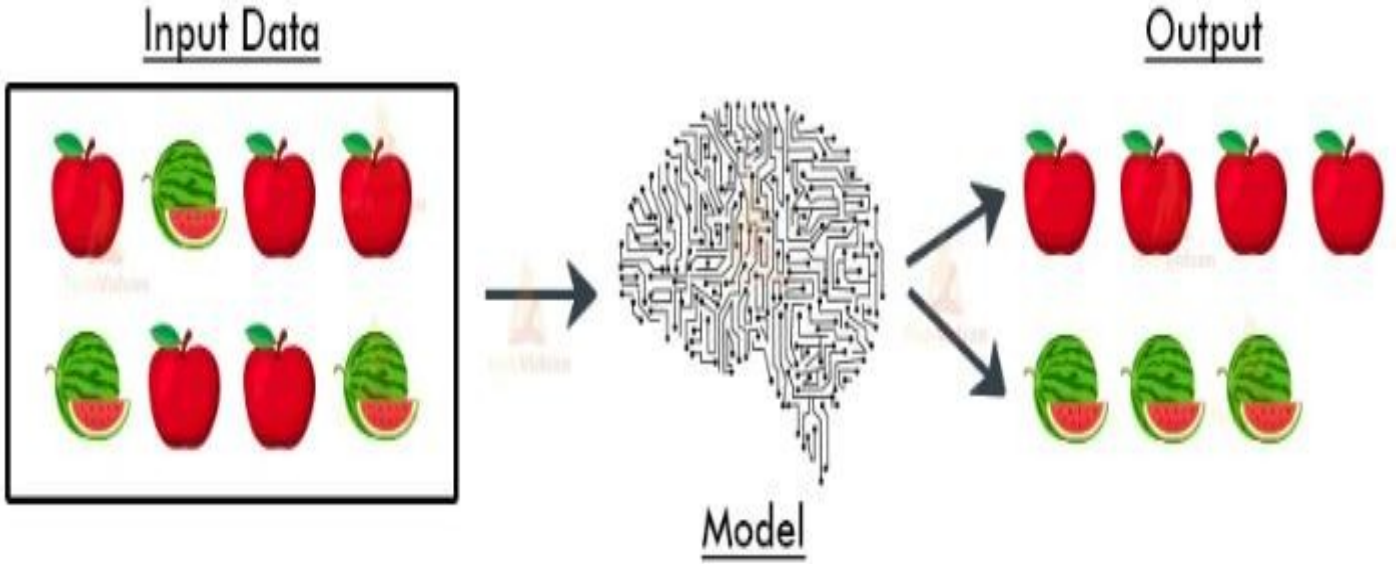
✹Classified into

- ■ Clustering
- ■ Association

Unsupervised Learning



Unsupervised Learning



What is Unsupervised Learning


~~CAR~~


~~CAR~~


~~BIKE~~


~~BIKE~~

Dataset

In the unsupervised learning, we do not need to know the labels or Ground truth values

Clustering



Dataset



Clustering

The task is to identify the patterns like group the similar objects together

Association Rules Mining

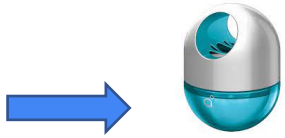
















Association Rules Mining

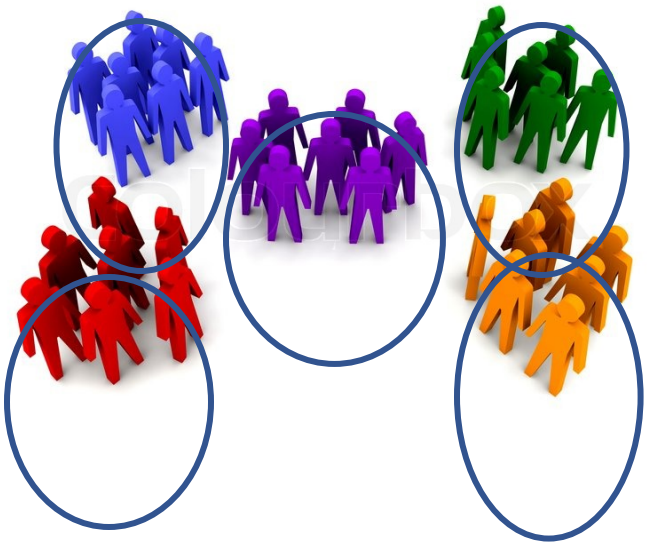
Dataset

[Association rules like]

More Example Unsupervised Learning- Clustering



Dataset



More Example Unsupervised Learning- Association







Applications of unsupervised learning

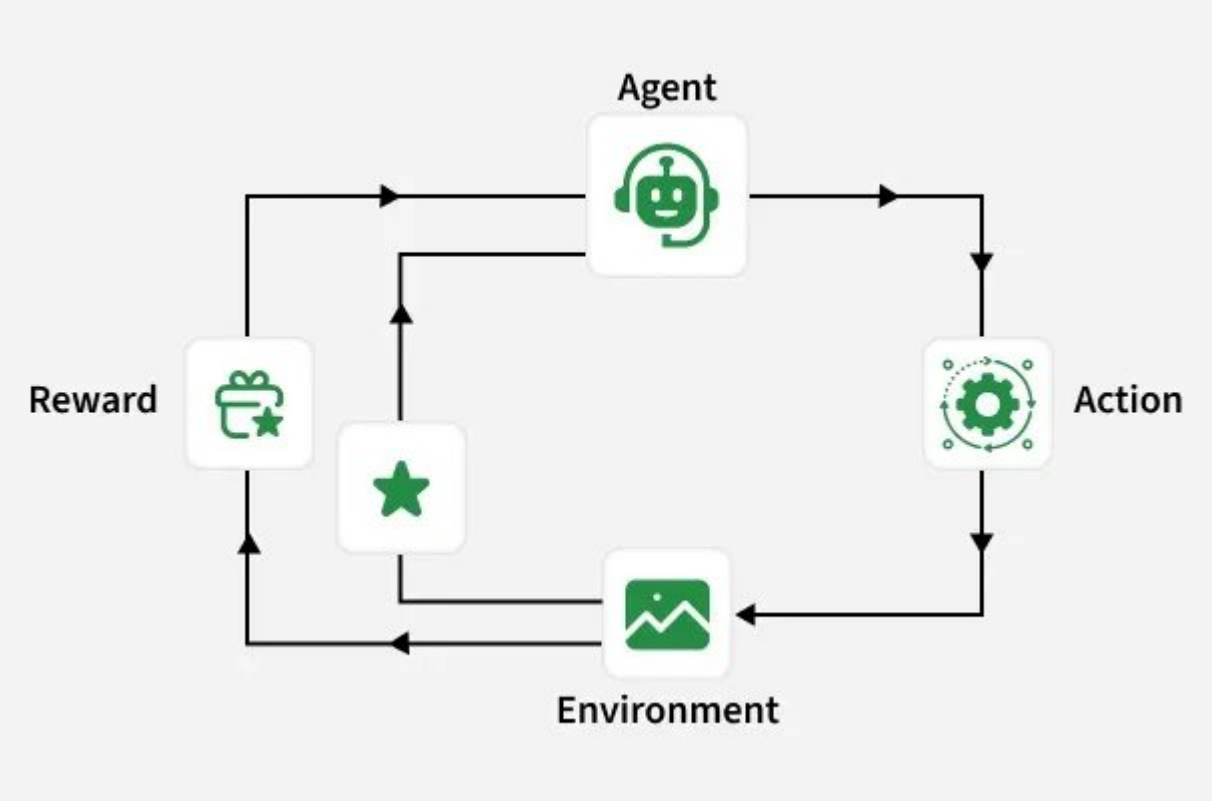
- News Sections
- Computer vision
- Medical imaging
- Anomaly detection
- Recommendation Engines

Reinforcement Learning

- It is a feedback-based machine learning technique, whereby an agent learns to behave in an environment by observing his mistakes and performing the actions.
- Reinforcement learning applies the method of learning via Interaction and feedback
- Reinforcement learning trains an agent to make a sequence of decisions through trial and error. The agent interacts with the environment, receives feedback in the form of rewards or penalties and learns optimal actions over time.

Reinforcement Learning

Learning from trials and errors



Reinforcement Learning

Agent: It is the learner or the decision-maker performing actions to receive a reward.

Environment: It is the scenario where an agent learns and performs future tasks.

Action: actions that are performed by the agent.

State: current situation

Policy: Decision-making function of an agent whereby the agent decides the future action based on the current state.

Reward: Returns provided by the environment to an agent for performing each action.

Example: An AI agent learning to play chess gets positive feedback for good moves and negative for poor ones. Over time, it learns strategies to win more often.

- Agent get a reward for each right action and a penalty for each wrong action.
- Agent learns automatically with feedback and improve its performance.

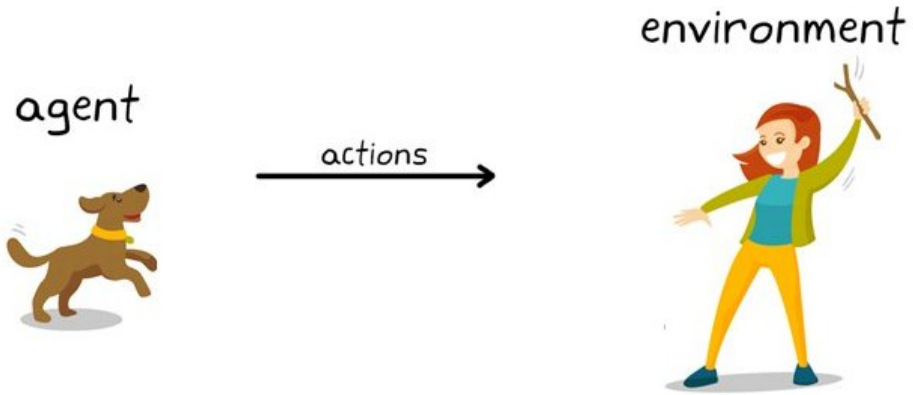
Here are some of most common reinforcement learning algorithms:

- **Q-learning**
- **SARSA (State-Action-Reward-State-Action)**
- **Deep Q-learning**

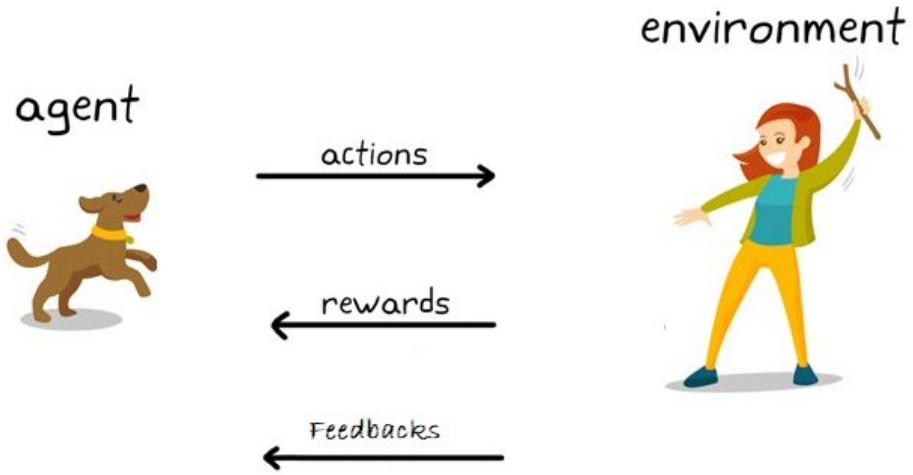
What is Reinforcement Learning



What is Reinforcement Learning



What is Reinforcement Learning



Example



Agent





Task



Environment

Reinforcement Learning









Punishment

Reinforcement Learning









Reward

Baby Learn from the Trials and Errors

How Does Reinforcement Learning Work- Example

- Cats don’t understand any form of language and therefore a different strategy has to be followed to communicate with the cat.
- A situation is created where the cat acts in various ways. The cat is rewarded with fish if it is the desired way. Therefore the cat behaves in the same way whenever it faces that situation expecting more food as a reward.
- The scenario defines the process of learning from positive experiences.
- Lastly, the cat also learns what not to do through negative experiences.

Where to Use Reinforcement Learning

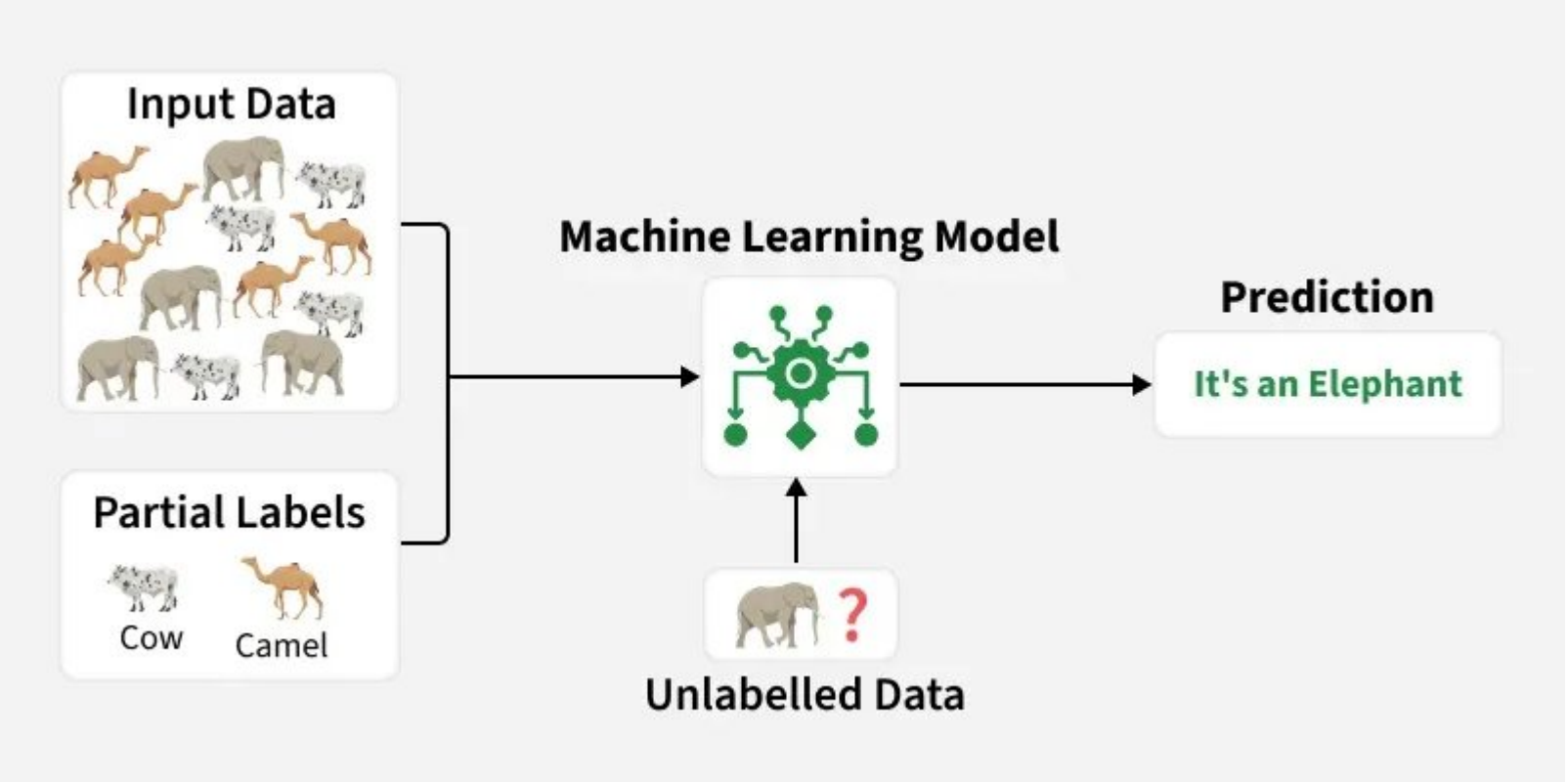
- When you need an agent to learn by interacting with an environment.
- Best for decision-making or optimization tasks involving trial and feedback loops.
- Used when long-term performance or adaptive behavior is more important than immediate accuracy.

Applications of Reinforcement Learning

- **Gaming and simulation:** Teaching agents or NPCs to play and adapt intelligently.
- **Robotics and automation:** Enabling robots to perform tasks autonomously.
- **Autonomous vehicles:** Helping self-driving cars make real-time decisions.
- **Healthcare and finance:** Optimizing treatment plans, trading and resource allocation.
- **Recommendation and personalization:** Improving user experience through adaptive suggestions.
- **Industrial and energy management:** Optimizing control systems and energy use.

Semi-Supervised Learning: Supervised + Unsupervised Learning

- Semi-Supervised Learning combines both Supervised and Unsupervised approaches.
- It uses a small set of labeled data and a large set of unlabeled data for training
- Useful when labeling is costly or time-consuming.



Semi-Supervised Learning: Supervised + Unsupervised Learning

Example: Consider that we are building a language translation model, having labeled translations for every sentence pair can be resources intensive.

It allows the models to learn from labeled and unlabeled sentence pairs, making them more accurate.

This technique has led to significant improvements in the quality of machine translation services.

Popular Techniques

- **Graph-based Learning:** Spreads label information through data relationships.
- **Label Propagation:** Iteratively assigns labels to unlabeled data.
- **Co-training:** Uses two models to train and label each other's data.
- **Self-training:** Uses model predictions as pseudo-labels.
- **Generative Adversarial Networks (GANs):** Generates synthetic data to improve learning.

Where to Use Semi-Supervised Learning

- When you have limited labeled data but plenty of unlabeled data.
- Useful for domains with high labeling costs, such as medical, NLP or image datasets.
- Ideal when unlabeled data still holds valuable information that can improve learning performance.

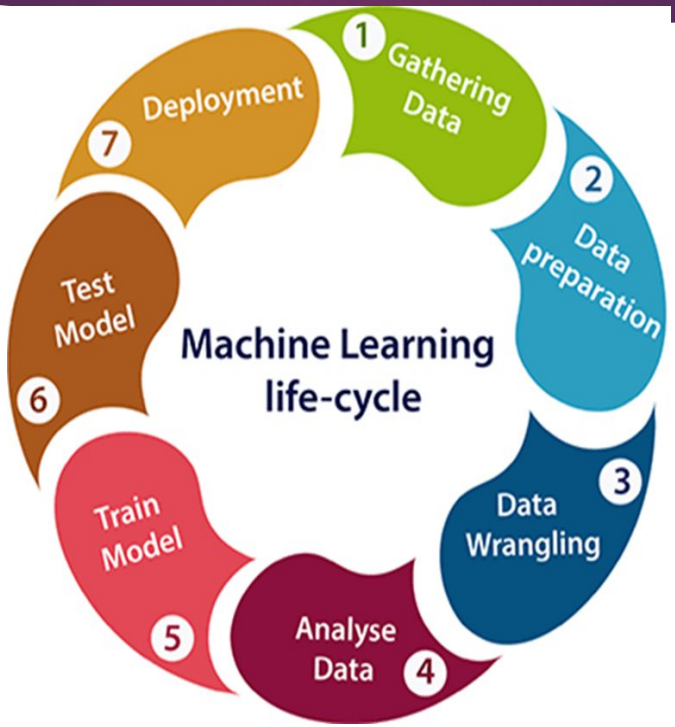
Applications

- **Image Classification:** Combine small labeled and large unlabeled image datasets to improve accuracy.
- **Natural Language Processing (NLP):** Enhance language models by using a mix of labeled and vast unlabeled text data.
- **Speech Recognition:** Boost accuracy by leveraging limited transcribed audio and more unlabeled speech data.
- **Recommendation Systems:** Improve recommendations using sparse labeled data and abundant unlabeled user behavior.
- **Healthcare & Medical Imaging:** Improve medical image analysis with a mix of labeled and

Comparison

Type	Data Requirement	Label Availability	Learning Goal	Common Use Case
Supervised	High	Labeled	Predict outputs	Spam detection, Price prediction
Unsupervised	Medium	Unlabeled	Find hidden patterns	Customer segmentation
Reinforcement	High	Reward feedback	Learn best actions	Robotics, Games

Machine learning Life cycle



Gathering Data

- Data Gathering is the first step of the machine learning life cycle. The goal of this step is to identify and obtain all data-related problems.
- Need to identify the different data sources, as data can be collected from various sources such as files, database, internet, or mobile devices.
- The quantity and quality of the collected data will determine the efficiency of the output.
- The more will be the data, the more accurate will be the prediction.

Gathering Data(cont.)

- This step includes the below tasks:
  - **Identify various data sources**
  - **Collect data**
  - **Integrate the data obtained from different sources**
- By performing the above task, we get a coherent set of data, also called as a **dataset**. It will be used in further steps

Data preparation

- After collecting the data, prepare it for further steps. Data preparation is a step where we put our data into a suitable place and prepare it to use in our machine learning training.
- Put all data together, and then randomize the ordering of data.

Further divided into two processes:

- **Data exploration:**
  Nature of data that we have to work with. We need to understand the characteristics, format, and quality of data.
  A better understanding of data leads to an effective outcome. In this, we find Correlations, general trends, and outliers.
- **Data pre-processing:**
  Now the next step is pre-processing of data for its analysis.

Data Wrangling

- Process of cleaning and converting raw data into a useable format.
- It is not necessary that data we have collected is always of our use as some of the data may not be useful. In real-world applications, collected data may have various issues, including:
  - Missing Values
  - Duplicate data
  - Invalid data
  - Noise
- Use various filtering techniques to clean the data.
- It is mandatory to detect and remove the above issues because it can negatively affect the quality of the outcome

Data Analysis

- Cleaned and prepared data is passed on to the analysis step. This step involves:
- Selection of analytical techniques
- Building models
- Review the result
- The aim is to build a machine learning model to analyze the data using various analytical techniques and review the outcome.
- Use Classification, Regression, Cluster analysis, Association, etc. then build the model using prepared data, and evaluate the model.

Train Model

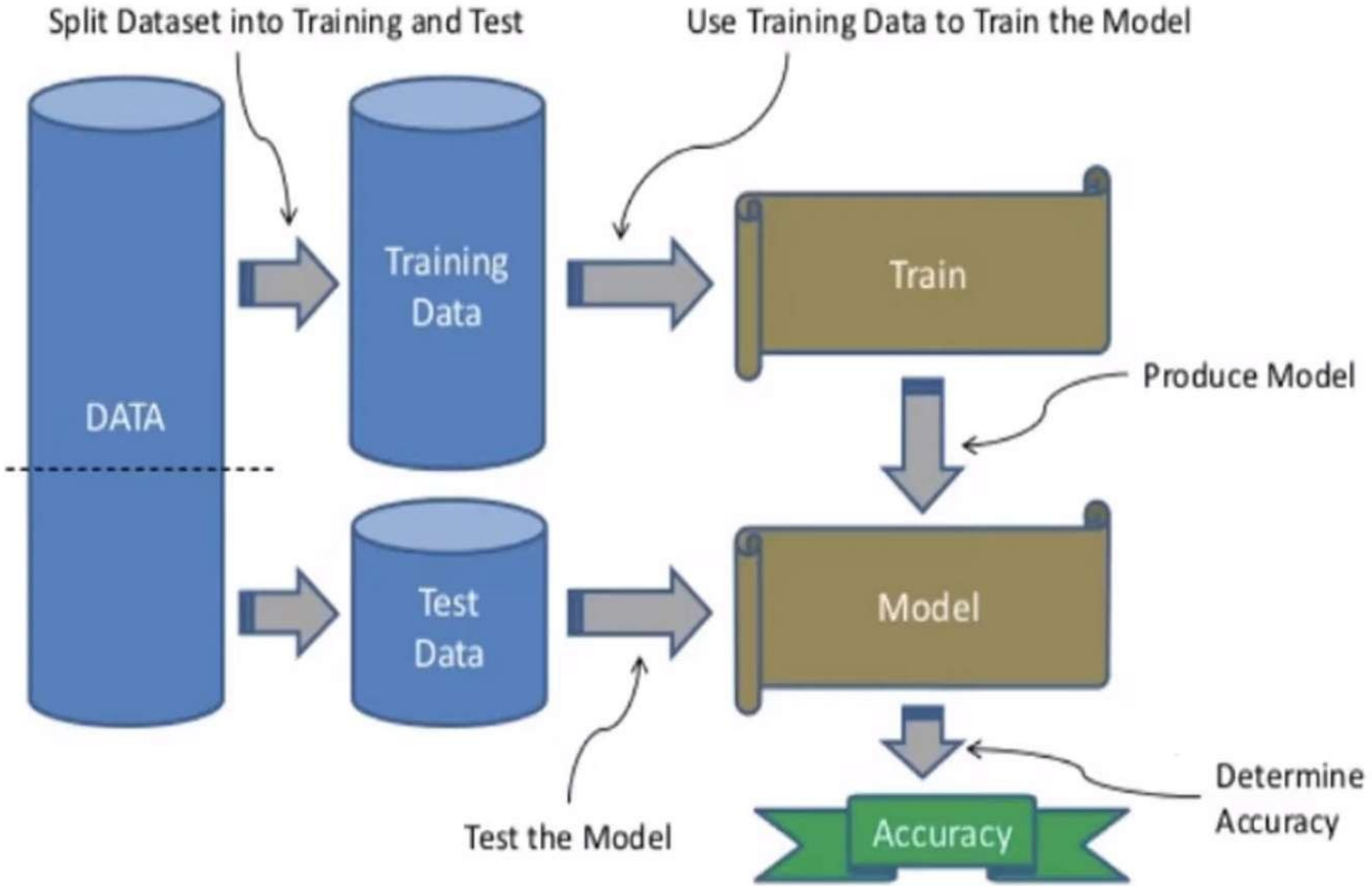
- Train the model to improve its performance for better outcome of the problem.

- Use datasets to train the model using various machine learning algorithms. Training a model is required so that it can understand the various patterns, rules, and, features.

Test Model

- Once machine learning model has been trained on a given dataset, then test the model. In this step, check for the accuracy of the model by providing a test dataset to it.

- Testing the model determines the percentage accuracy of the model as per the requirement of project or problem.


Split Dataset into Training and Test
Use Training Data to Train the Model
DATA
Training Data
Test Data
Train
Produce Model
Model
Test the Model
Accuracy
Determine Accuracy

Deployment

- The last step of machine learning life cycle is deployment, where we **deploy the model in the real-world system.**
- If the above-prepared model is producing an accurate result as per our requirement with acceptable speed, then we deploy the model in the real system.
- Similar to making the final report for a project.

Implementation for model Training

1. Import Libraries

```
import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.pipeline import Pipeline

from sklearn.compose import ColumnTransformer

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score
```

2. Load and Prepare the data

```
# Load dataset
df = pd.read_csv("https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv")


# Select relevant features
features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
df = df[features + ['Survived']].dropna()  # Drop rows with missing values


# Display the first few rows
print(df.head())
```

```
   Pclass     Sex   Age  SibSp  Parch     Fare Embarked  Survived
0       3    male  22.0      1      0   7.2500        S         0
1       1  female  38.0      1      0  71.2833        C         1
2       3  female  26.0      0      0   7.9250        S         1
3       1  female  35.0      1      0  53.1000        S         1
4       3    male  35.0      0      0   8.0500        S         0
```

3. Define Preprocessing Steps

```
# Define numerical and categorical features
num_features = ['Age', 'SibSp', 'Parch', 'Fare']
cat_features = ['Pclass', 'Sex', 'Embarked']

# Define transformers
num_transformer = StandardScaler()  # Standardization for numerical features
cat_transformer = OneHotEncoder(handle_unknown='ignore')  # One-hot encoding for categorical features

# Combine transformers into a preprocessor
preprocessor = ColumnTransformer([('num', num_transformer, num_features),
    ('cat', cat_transformer, cat_features)])
```

4. Split the data for training and Testing

```
# Define target and features

X = df[features]

y = df['Survived']

# Split into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Display the shape of the data

print(f"Training set shape: {X_train.shape}")

print(f"Testing set shape: {X_test.shape}")
```

Output:

```
Training set shape: (567, 7)

Testing set shape: (143, 7)
```

5. Build and Train model

```
# Define the pipeline
pipeline = Pipeline([
    ('preprocessor', preprocessor),  # Data transformation
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))  # ML model
])
# Train the model
pipeline.fit(X_train, y_train)
print("Model training complete!")
```

Output:

```
Model training complete!
```

6. Evaluate the Model

```
# Make predictions
y_pred = pipeline.predict(X_test)

# Compute accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")
```

Output:

```
Model Accuracy: 0.76
```

7. Save and Load the Model

```
import joblib

# Save the trained pipeline

joblib.dump(pipeline, 'ml_pipeline.pkl')

# Load the model

loaded_pipeline = joblib.load('ml_pipeline.pkl')

# Predict using the loaded model

sample_data = pd.DataFrame([{'Pclass': 3, 'Sex': 'male', 'Age': 25, 'SibSp':
0, 'Parch': 0, 'Fare': 7.5, 'Embarked': 'S'}])

prediction = loaded_pipeline.predict(sample_data)

print(f"Prediction: {'Survived' if prediction[0] == 1 else 'Did not Survive'}")
```

Output:

```
Prediction: Did not Survive
```

```
Data Sample:
    Pclass     Sex   Age  SibSp  Parch     Fare Embarked  Survived
0       3    male  22.0      1      0   7.2500        S         0
1       1  female  38.0      1      0  71.2833        C         1
2       3  female  26.0      0      0   7.9250        S         1
3       1  female  35.0      1      0  53.1000        S         1
4       3    male  35.0      0      0   8.0500        S         0
Training set shape: (569, 7)
Testing set shape: (143, 7)
Model training complete!
Model Accuracy: 0.76
Prediction for Sample Data: Did not Survive
```

Learning

"The action or process of obtaining information or ability through studying, practicing, being instructed, or experiencing something,"

We may split learning techniques into five categories based on this:

1. **Rote Learning (Memorizing):** Memorizing things without understanding the underlying principles or rationale.
2. **Instructions (Passive Learning):** Learning from a teacher or expert.
3. **Analogy (Experience)**: We may learn new things by applying what we've learned in the past.
4. **Inductive Learning (Experience):** Formulating a generalized notion based on prior experience.
5. **Deductive Learning:** Getting new information from old information.

Inductive Learning

- Inductive Learning is the process of forming a generalized notion after viewing many examples of the concept.
- For instance, suppose a child is requested to write the solution 2*8=? They can memorize the answer using the rote learning approach or use inductive Learning to construct a notion to compute the outcomes using examples such as 2*1=2, 2*2=4, and so on. In this approach, the child will be able to answer questions of a similar nature utilizing the same notion.
- Similarly, we may train our computer to learn from previous data and recognize whether an object is in a given category of interest.

The hypothesis of Inductive Learning:

- The ultimate aim of concept learning is to find a hypothesis ‘h’ that is identical to the target notion c over data set X, with the only knowledge about c being its value over X.
- “Any hypothesis that approximates the target value well over a suitably large collection of training instances will likewise approximate the target value well over other unseen cases,”
- Consider whether a person goes to the movies or not based on four binary characteristics with two potential values (true or false):
  - 1. Is Rich -> true, false
  - 2. Is There Any Free Time -> true or false
  - 3. It’s a Holiday -> true or false
  - 4. Has work pending -> true, false
- We also have training data, with two data items serving as positive samples and one serving as a negative sample:
  - **x1**: <true, true, false, false> : **+ve**
  - **x2**: <true, false, false, true> : **+ve**
  - **x3:** <true, false, false, true> : **-ve**

Concept Learning

What if machines could learn the same way we do, by forming concepts from examples?

- In machine learning, the process of identifying a general rule or concept from specific examples is called concept learning.
- "Inferring a boolean-valued function from training examples of its input and output'
- Formal Definition: It is the process of inferring a generalised description (concept) from a set of positive and negative examples using logical expressions or hypothesis space.

Concept Learning Example:

Let’s say you’re building a model to identify “fruit”.

You provide examples like:

Colour	Shape	Taste	Fruit?
Red	Round	Sweet	Yes
Yellow	Long	Sweet	Yes
Green	Spiky	Sour	No
Orange	Round	Sweet	Yes

From these, the model tries to find a pattern, maybe “sweet and round” means fruit. This is a basic concept learning task in machine learning. The machine tries to find the common features in all “Yes” examples while rejecting the “No” ones.

The Concept Learning Task

The actual concept learning task involves:

1. **Hypothesis Space (H):** All possible concepts the model could learn.
2. **Training Examples (D):** Positive and negative data samples.
3. **Target Concept (C):** The actual rule that classifies inputs correctly.
4. **Learning Algorithm:** The method to search and narrow down hypotheses.

The Goal: Find the hypothesis $h \in H$ such that $h(x) = C(x)$ for every training example x.

concept learning **-**"Problem of searching through a predefined space of potential hypotheses for the hypothesis that best fits the training examples"

A Concept Learning Task – Enjoy Sport Training Examples

Example	**Sky**	**AirTemp**	**Humidity**	**Wind**	**Water**	**Forecast**	**EnjoySport**
1	Sunny	Warm	Normal	Strong	Warm	Same	**YES**
2	Sunny	Warm	High	Strong	Warm	Same	**YES**
3	Rainy	Cold	High	Strong	Warm	Change	**NO**
4	Sunny	Warm	High	Strong	Warm	Change	**YES**

ATTRIBUTES

CONCEPT

- A set of example days, and each is described by six attributes.
- The task is to learn to predict the value of EnjoySport for arbitrary day, based on the values of its attribute values.

A Concept Learning Task – Enjoy Sport

- Now the target concept is **EnjoySport : X →{0,1}.**
- With this, a learner task is to learn to predict the value of EnjoySport for arbitrary day, based on the values of its attribute values.
- When a new sample with the values for attributes **<Sky, Air Temperature, Humidity, Wind, Water, Forecast>** is given, the value for EnjoySport (ie. 0 or 1) is predicted based on the previous learning.

A Concept Learning Task – Enjoy Sport

Let H denote the set of all possible hypotheses that the learner may consider regarding the identity of the target concept.

So H = {h1, h2, …. }.

Let each hypothesis be a vector of six constraints, specifying the values of the six attributes **<Sky, Air Temperature, Humidity, Wind, Water, Forecast>**.

In hypothesis representation, value of each attribute could be either

- "?' — that any value is acceptable for this attribute,
- specify a single required value (e.g., Warm) for the attribute, or
- "0" that no value is acceptable.

A Concept Learning Task – Enjoy Sport

For Example:

- The hypothesis that Sachin enjoys his favorite sport only on cold days with high humidity (independent of the values of the other attributes) is represented by the expression **< ?,cold, High,?, ? ,?>**
- The most general hypothesis — < ?, ? , ? , ?, ? , ?> that every day is a positive example
- The most specific hypothesis — < 0, 0, 0, 0, 0, 0>that no day is a positive example.

EnjoySport concept learning task requires

- learning the sets of days for which EnjoySport=yes and then describing this set by a conjunction of constraints over the instance attributes.

A Concept Learning Task – Enjoy Sport

1. **X** — The set of items over which the concept is defined is called the set of instances, which we denote by X.
2. **c** — The concept or function to be learned is called the target concept. In the current example, the target concept corresponds to the value of the attribute EnjoySport (i.e, c(x)=1 if EnjoySport=Yes, and c(x)=0 if EnjoySport= No).
3. **(x, c(x))** — When learning the target concept, the learner is presented by a set of training examples, each consisting of an instance x from X, along with its target concept value c(x). Instances for which c(x) = 1 are called positive examples and instances for which c(x) = 0 are called negative examples. We will often write the ordered pair (x, c(x)) to describe the training example consisting of the instance x and its target concept value c(x).
4. **D** — We use the symbol D to denote the set of available training examples.
5. **H** — We use the symbol H to denote the set of all possible hypotheses that the learner may consider regarding the identity of the target concept.
6. **h(x)** — In general, each hypothesis h in H represents a Boolean-valued function defined over X; that is, h : X →{0, 1}. The goal of the learner is to find a hypothesis h such that h(x) = c(x) for all x in X.

A Concept Learning Task – Enjoy Sport

Given

- ***Instances X :*** set of all possible days, each described by the attributes
  - Sky – (values: Sunny, Cloudy, Rainy)
  - AirTemp – (values: Warm, Cold)
  - Humidity – (values: Normal, High)
  - Wind – (values: Strong, Weak)
  - Water – (values: Warm, Cold)
  - Forecast – (values: Same, Change)
- ***Target Concept (Function) c :*** EnjoySport : X → {0,1}
- ***Hypotheses H :*** Each hypothesis is described by a conjunction of constraints on the attributes.
- ***Training Examples D :*** positive and negative examples of the target function

Determine

- A hypothesis h in H such that h(x) = c(x) for all x in D.

Inductive learning hypothesis

For a given collection of examples, in reality, learning algorithm return a function h (hypothesis) that approximates c (target concept). But the expectation is, the learning algorithm to return a function h (hypothesis) that equals c (target concept) ie. h(x) = c(x) for all x in D.

Inductive learning hypothesis is any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over any other unobserved examples.

Hypothesis Space- EnjoySport learning task.

X -denote the instances

H -hypotheses

Attributes	Values	Count
1. Sky	Sunny, Cloudy, Rainy	3
2. AirTemp	Warm, Cold	2
3. Humidity	Normal, High	2
4. Wind	Strong, Weak	2
5. Water	Warm, Cool	2
6. Forecast	Same, Change	2

$$Distinct\ Observations\ in\ X = 3.2.2.2.2.2 = 96$$

$$Distinct\ hypothesis\ in\ H = 5.4.4.4.4.4 = 5120$$

Hypothesis h is a vector of six constraints, specifying the values of the six attributes **<Sky, Air Temperature, Humidity, Wind, Water, Forecast>**.

In this hypothesis representation, value of each attribute could be either “?” or “0” other than defined values.

Hypothesis Space- EnjoySport learning task.

The number of combinations: 5×4×4×4×4×4 = 5120 syntactically distinct hypotheses. (Two more values for attributes: ? and 0)

They are syntactically distinct but not semantically.

For example, the below 2 hypothesis says the same but they look different.

h1 = <Sky=0 AND Temp=warm AND Humidity=? AND Wind=strong AND Water=warm AND Forecast=same >

h2 = <Sky=sunny AND Temp=warm AND Humidity=? AND Wind=strong AND Water=0 AND Forecast=same>

Neither of these hypotheses accept any "day", so semantically the same. All such hypothesis having same semantic is counted as 1. So we can have total number of combinations as below.

1 (hypothesis with one or more 0) + 4×3×3×3×3×3 (add ? to each attribute) = 973 semantically distinct hypotheses [Only one more value for attributes: ?, and one hypothesis representing empty set of instances.]

General-to-Specific Ordering of Hypotheses

Consider two hypotheses, -

h1 = (Sunny, ?, ?, Strong, ?, ?)

h2 = (Sunny, ?, ?, ?, ?, ?)

The hypothesis h2 has fewer constraints on attributes than hypothesis h1.

The sets of instances that are classified positive by h1 will be less than sets of instances that are classified positive by h2 as h2 imposes fewer constraints on the instance.

Any instance classified positive by hl will also be classified positive by h2.

Therefore, we say that h2 is more general than hl. In reverse, we can also say that, h1 is more specific than h2.

General-to-Specific Ordering of Hypotheses

Hypothesis h1 and h2 classifies the instance x as positive can written as h1(x) = 1 and h2(x) = 1.

Now h2 is more general than h1, so it can be written as if h1(x) = 1 implies h2(x) = 1. In symbols .

$$h_2 \geq_g h_1$$

Definition for **more-general-than-or-equal-to** –

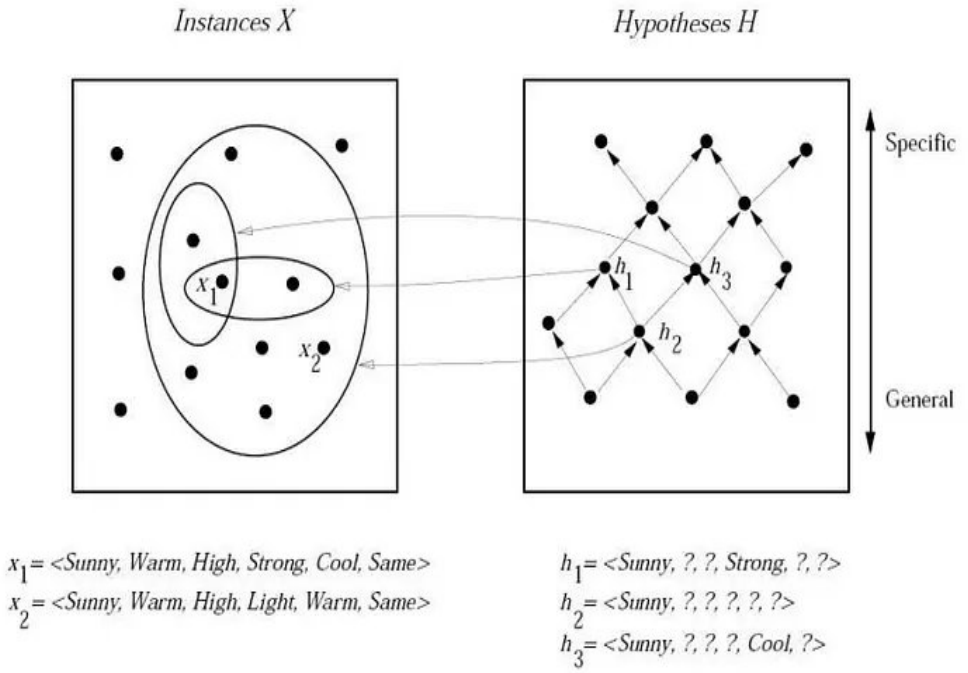
Definition: Let h_j and h_k be Boolean-valued functions defined over X. Then h_j is **more general-than-or-equal-to** h_k (written $h_j \geq_g h_k$) if and only if

$$(\forall x \in X)[h_k(x) = 1 \rightarrow h_j(x) = 1]$$

In similar way, **more-specific-than**, also be defined as below.

The h_j is **more-general-than** h_k ($h_j > h_k$) if and only if $h_j \geq_g h_k$ is true and $h_k \geq_g h_j$ is false. We also say h_k is **more-specific-than** h_j.

General-to-Specific Ordering of Hypotheses



- The box on the left represents the set X of all instances, the box on the right the set H of all hypotheses.
- Each hypothesis corresponds to some subset of X, that is, the subset of instances that it classifies positive. In the figure, there are 2 instances x1 and x2, and 3 hypothesis h1, h2 and h2.
- h1 classifies — x1, h2 classifies — x1 and x2, and h3 classifies — x1. This indicates, h2 is more-general-than h1 and h3.
- The arrows connecting hypotheses represent the more-general-than relation, with the arrow pointing toward the less general hypothesis.
- Note the subset of instances characterized by h2 subsumes the subset characterized by hl ,hence h2 is more-general–than h1

Many learning algorithms for concept learning organize the search through the hypothesis space by relying on a general-to-specific ordering of hypotheses.

Find-s: finding a maximally specific hypothesis

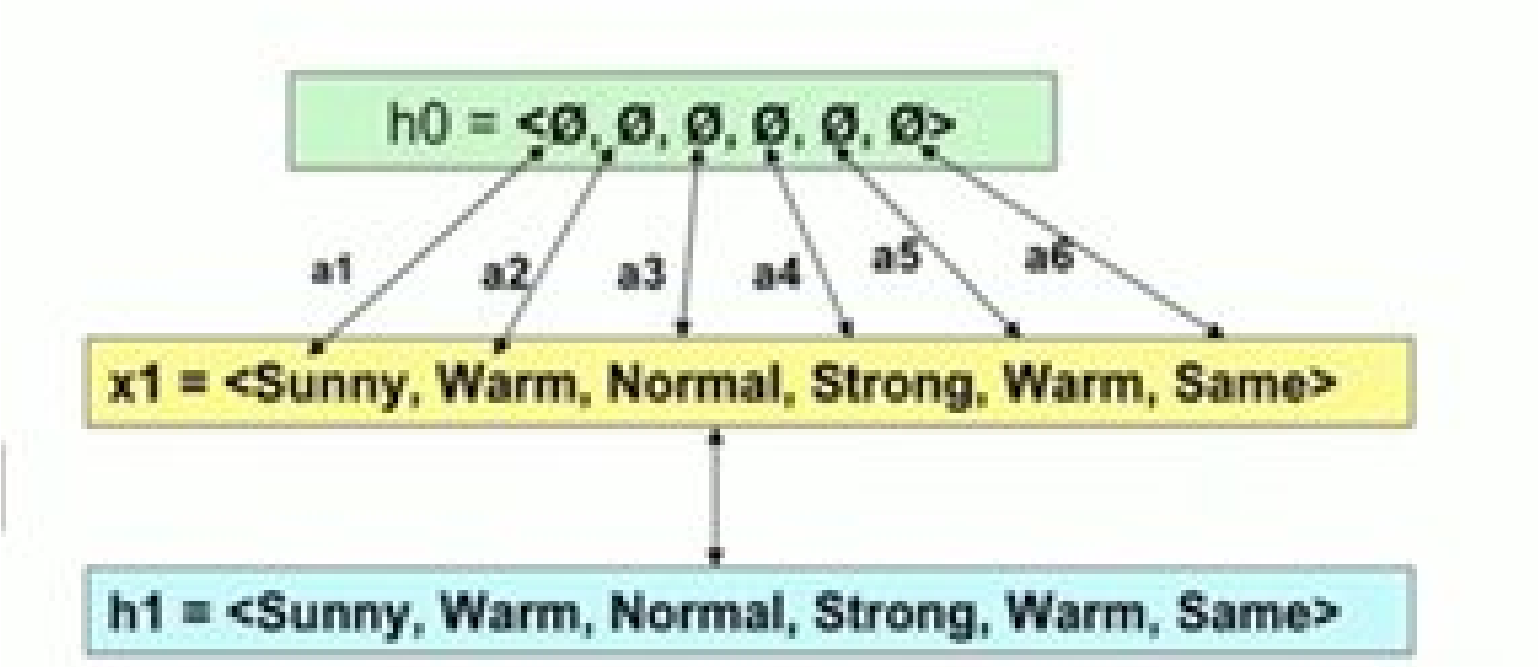
1.Initialize h to the most specific hypothesis in H.
2.For each positive training instance x
    For each attribute constraint a_i in h
        If the constraint a_i is satisfied by x
        Then do nothing
        Else replace a_i in h by the next more general constraint that is satisfied by x
3. Output hypothesis h

Example:

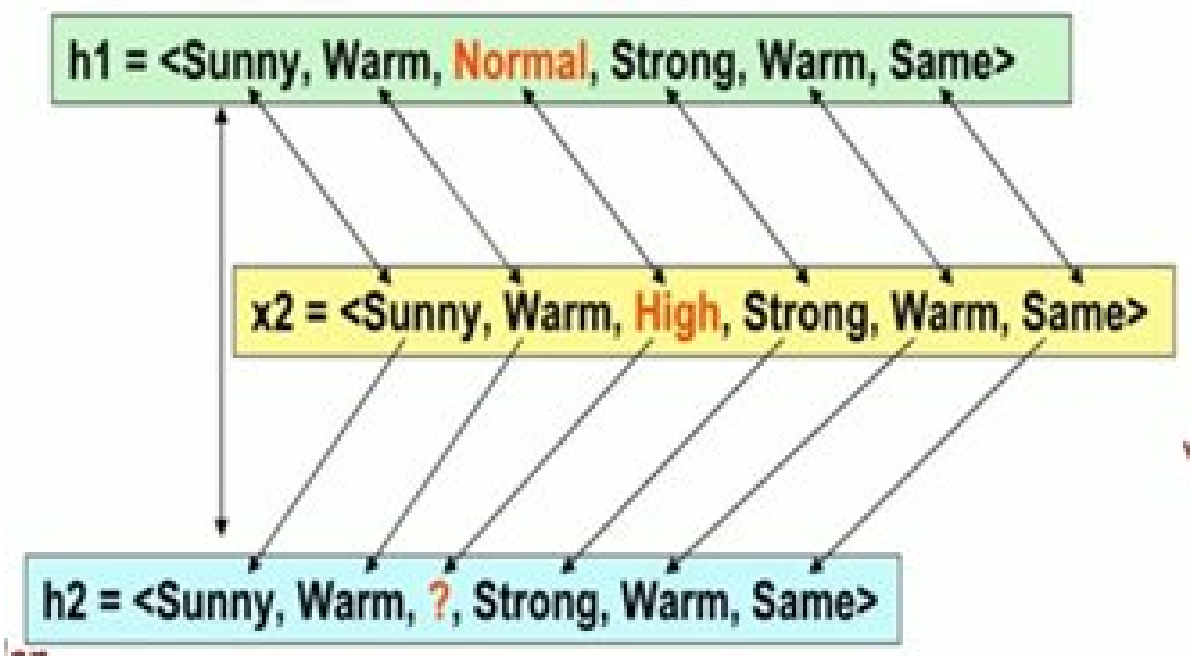
Assume the learner is given the sequence of training examples from the EnjoySport task

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Iteration 1


h0 = <∅, ∅, ∅, ∅, ∅, ∅>
a1
a2
a3
a4
a5
a6
x1 = <Sunny, Warm, Normal, Strong, Warm, Same>
h1 = <Sunny, Warm, Normal, Strong, Warm, Same>

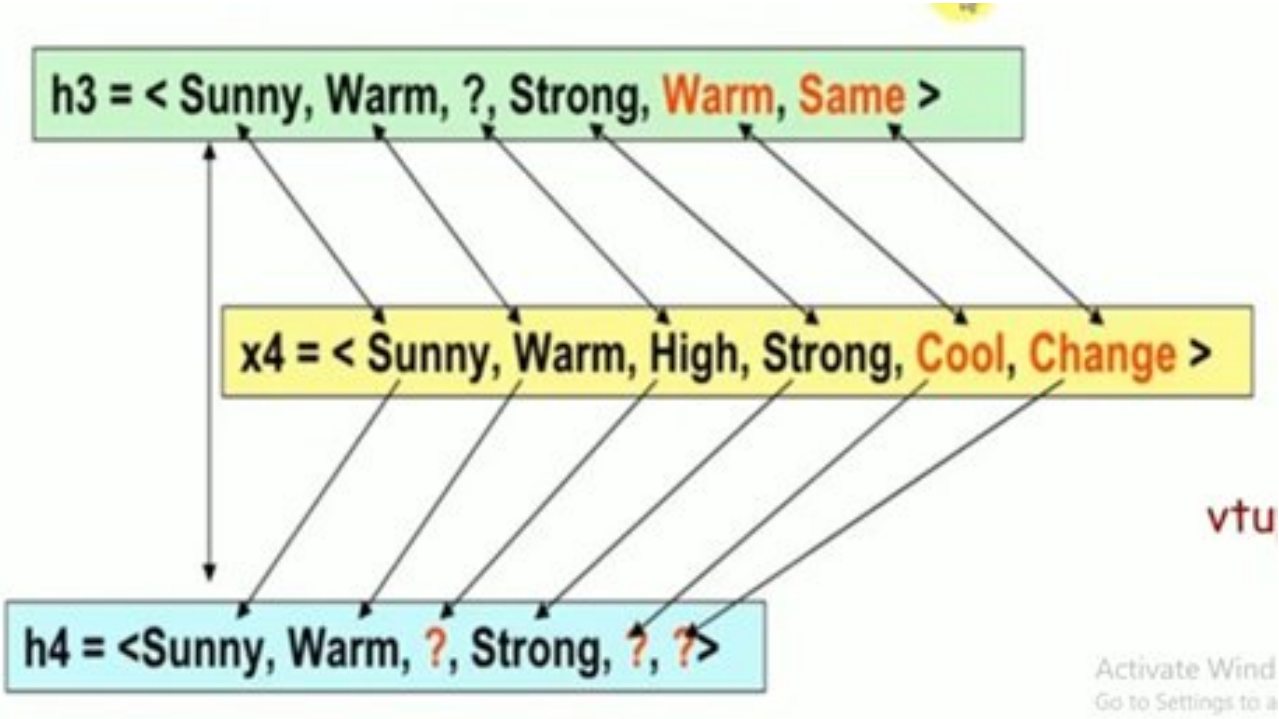
Iteration 2


h1 = <Sunny, Warm, Normal, Strong, Warm, Same>
x2 = <Sunny, Warm, High, Strong, Warm, Same>
h2 = <Sunny, Warm, ?, Strong, Warm, Same>

Iteration 3



Iteration 4


h3 = < Sunny, Warm, ?, Strong, Warm, Same >
x4 = < Sunny, Warm, High, Strong, Cool, Change >
h4 = <Sunny, Warm, ?, Strong, ?, ?>
vtu
Activate Wind
Go to Settings to a

Find-s

1. Initialize h to the most specific hypothesis in H **h — (Ø, Ø, Ø, Ø, Ø, Ø)**

2. First training example

x1 = < Sunny, Warm, Normal, Strong ,Warm ,Same>, EnjoySport = +ve.

Observing the first training example, it is clear that hypothesis h is too specific. None of the "Ø" constraints in h are satisfied by this example, so each is replaced by the next more general constraint that fits the example

h1 = < Sunny, Warm, Normal, Strong ,Warm, Same>.

Find-s

3. Consider the second training example

x2 = < Sunny, Warm, High, Strong, Warm, Same>, EnjoySport = +ve.

The second training example forces the algorithm to further generalize h, this time substituting a “?” in place of any attribute value in h that is not satisfied by the new example.

Now h2 =< Sunny, Warm, ?, Strong, Warm, Same>

Find-s

4. Consider the third training example

x3 =< Rainy, Cold, High, Strong, Warm, Change>,EnjoySport = — ve.

The FIND-S algorithm simply ignores every negative example. So the hypothesis remain as before,

so **h3 =< Sunny, Warm, ?, Strong, Warm, Same>**

Find-s

5. Consider the fourth training example

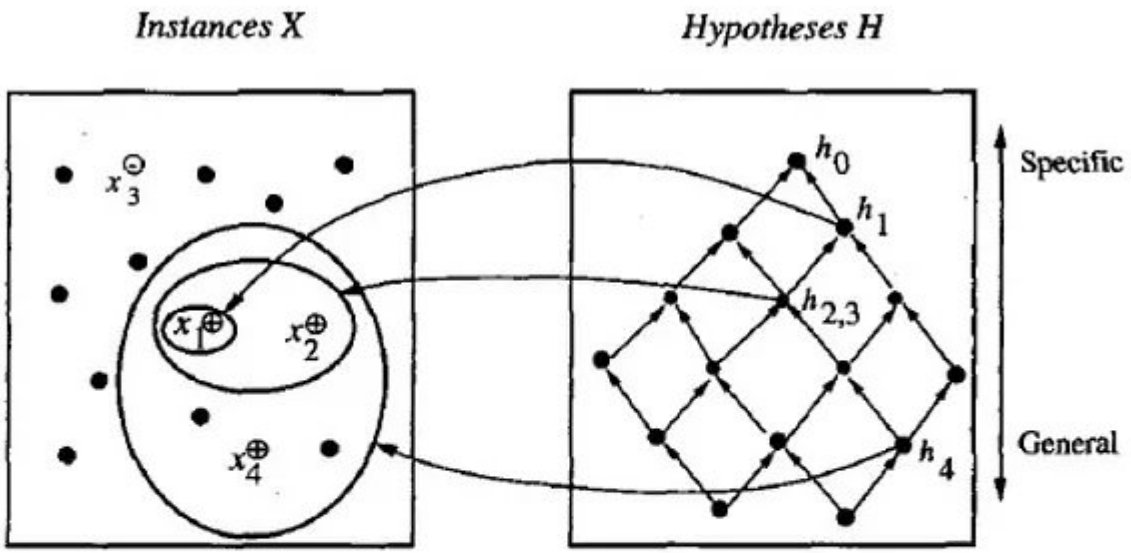
x4 =<Sunny,Warm,High,Strong, Cool,Change>, EnjoySport =+ve.

The fourth example leads to a further generalization of h as

h4 =< Sunny, Warm, ?, Strong, ?, ?>

6. So the final hypothesis is **< Sunny, Warm, ?, Strong, ?, ?>**

Find-s



x_1 = <Sunny Warm Normal Strong Warm Same>, +

x_2 = <Sunny Warm High Strong Warm Same>, +

x_3 = <Rainy Cold High Strong Warm Change>, -

x_4 = <Sunny Warm High Strong Cool Change>, +

h_0 = <∅, ∅, ∅, ∅, ∅, ∅>

h_1 = <Sunny Warm Normal Strong Warm Same>

h_2 = <Sunny Warm ? Strong Warm Same>

h_3 = <Sunny Warm ? Strong Warm Same>

h_4 = <Sunny Warm ? Strong ? ? >

The key property of the FIND-S algorithm

- FIND-S is guaranteed to output the most specific hypothesis within H that is consistent with the positive training examples.
- FIND-S algorithm's final hypothesis will also be consistent with the negative examples provided the correct target concept is contained in H, and provided the training examples are correct.

Unanswered questions by FIND-S

1. **Has the learner converged to the correct target concept ?.** Although FIND-S will find a hypothesis consistent with the training data, it has no way to determine whether it has found the only hypothesis in H consistent with the data (i.e., the correct target concept), or whether there are many other consistent hypotheses as well.
2. **Why prefer the most specific hypothesis ?.** In case there are multiple hypotheses consistent with the training examples, FIND-S will find the most specific. It is unclear whether we should prefer this hypothesis over, say, the most general, or some other hypothesis of intermediate generality.

Unanswered questions by FIND-S

3. Are the training examples consistent ?. In most practical learning problems there is some chance that the training examples will contain at least some errors or noise. Such inconsistent sets of training examples can severely mislead FIND-S, given the fact that it ignores negative examples.

4. What if there are several maximally specific consistent hypotheses?. There can be several maximally specific hypotheses consistent with the data. Find S finds only one.

Consistent Hypothesis

A hypothesis h is consistent with a set of training examples D if and only if h(x) = c(x) for each example (x, c(x)) in D.

$$Consistent\ (h, D) \equiv (\forall\ \langle x, c(x)\rangle \in D)\ h(x) = c(x))$$

Difference between consistent and satisfies

A training example x is said to satisfy hypothesis h when h(x) = 1, regardless of whether x is a positive or negative example of the target concept.

An example x is said to consistent with hypothesis h iff h(x) = c(x)

Consistent Hypothesis- Example 1

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Consistent Hypothesis- Example 1

We found the hypothesis h 1= **< Sunny, Warm, ?, Strong, ?, ?>.** Now for each example (x, c(x)) in D, we will evaluate h(x) equals c(x).

1. (<Sunny, Warm, Normal, Strong, Warm, Same>,<yes>) → h(x)=c(x)
2. (<Sunny, Warm, High, Strong, Warm, Same>,<yes>) → h(x)=c(x)
3. (<Rainy, Cold, High, Strong, Warm, Change>,<No>) → h(x)=c(x)
4. (<Sunny, Warm, High, Strong, Cool, Change>,<yes>) → h(x)=c(x)

Now we can say, hypothesis h1 is consistent with a set of training examples D

Qn: Hypothesis h2 = < ?, ?, ?, Strong, ?, ?>, is this hypothesis consistent with set of training example D ?

In case of training example (3), h(x) != c(x). So hypothesis h1 is not consistent with D.

Qn: We have a hypothesis h3= < ?, Warm, ?, Strong, ?, ?>, is this hypothesis consistent with set of training example D ?

All the training examples hold h(x) = c(x). So hypothesis h2 is consistent with D.

Consistent Hypothesis- Example 2

Example	Citations	Size	InLibrary	Price	Editions	Buy
1	Some	Small	No	Affordable	One	No
2	Many	Big	No	Expensive	Many	Yes

h1 = (?, ?, No, ?, Many) – Yes —- is a consistent hypothesis

h2 = (?, ?, No, ?, ?) – yes —- is inconsistent hypothesis

Check consistent/not- identify the violating example(s)

1. x1=(<Sunny,Hot,Normal,Strong,Warm,Same>,<Yes>) → h(x)=c(x)
2. x2=(<Sunny,Cool,High,Weak,Warm,Change>,<No>) → h(x)=c(x)
3. x3=(<Rainy,Mild,Normal,Strong,Cool,Same>,<Yes>) → h(x)=c(x)
4. x4=(<Overcast,Hot,High,Weak,Warm,Same>,<Yes>) → h(x)=c(x)

h1 = <Sunny, ?, ?, ?, ?, ?>

h2 = < ?, ?, ?, Strong, ?, ?>

h3 = <Sunny, Hot, ?, ?, ?, ?>

h4 = < ?, Cool, ?, ?, ?, ?>

h5 = < ?, ?, Normal, ?, ?, ?>

h6 = <Overcast, ?, ?, ?, ?, ?>

h7 = < ?, ?, High, Weak, ?, ?>

h8 = <Rainy, ?, ?, Strong, ?, ?>

h9 = <Sunny, ?, Normal, ?, ?, ?>

h10 =< ?, Hot, ?, ?, Warm, ?>

Version space

The version space, denoted VS_H,D with respect to hypothesis space H and training examples D, is the subset of hypotheses from H consistent with the training examples in D

$$VS_{H,D} \equiv \{h \in H \mid Consistent(h, D)\}$$

In the previous example, we have two hypothesis from H and they are consistent with D.

h1=< Sunny, Warm, ?, Strong, ?, ?>

and h3=< ?, Warm, ?, Strong, ?, ?>

So this set of hypothesis { h1, h3} is called a Version Space.

List-Then-Eliminate algorithm

1. ***VersionSpace*** ***c*** *a* list containing every hypothesis in H
2. For each training example, ***(x, c(x))***

   remove from ***VersionSpace*** any hypothesis ***h*** for which ***h(x) ≠ c(x)***
3. Output the list of hypotheses in ***VersionSpace***

List-Then-Eliminate algorithm- Example1

Example: List-Then-Eliminate algorithm

F1 – > A, B

F2 – > X, Y

Instance Space: (A, X), (A, Y), (B, X), (B, Y) – **4 Examples**

Hypothesis Space: (A, X), (A, Y), (A, **ø**), (A, ?), (B, X), (B, Y), (B, **ø**), (B, ?), (**ø**, X), (**ø**, Y), (**ø**, **ø**), (**ø**, ?), (**?**, X), (**?**, Y), (**?**, **ø**), (**?**, ?) – **16 Hypothesis**

Semantically Distinct Hypothesis : (A, X), (A, Y), (A, ?), (B, X), (B, Y), (B, ?), (**?**, X), (**?**, Y (**?**, ?), (**ø**, **ø**) – **10**

List-Then-Eliminate algorithm- Example1

Solution:

Version Space: (A, X), (A, Y), (A, ?), (B, X), (B, Y), (B, ?), (?, X), (?, Y) (?, ?), (ø, ø),

Training Instances

F1 F2 Target

A X Yes

A Y Yes

Consistent Hypothesis are: (A, ?), (?, ?)

Problems: List-Then-Eliminate algorithm

1. The hypothesis space must be finite
2. Enumeration of all the hypothesis, in practice it is not feasible.thereforer inefficient

List-Then-Eliminate algorithm- Example2

For the enjoysport training examples D, we can output the below list of hypothesis which are consistent with D. In other words, the below list of hypothesis is a version space.

h1	Sunny	?	?	?	?	?
h2	?	Warm	?	?	?	?
h3	Sunny	?	?	Strong	?	?
h4	Sunny	Warm	?	?	?	?
h5	?	Warm	?	Strong	?	?
h6	Sunny	Warm	?	Strong	?	?

List-Then-Eliminate algorithm- Example

In the list of hypothesis, there are two extremes representing general (h1 and h2) and specific (h6) hypothesis. Lets define these 2 extremes as general boundary G and specific boundary S.

General boundary G

The **general boundary** G, with respect to hypothesis space H and training data D, is the set of maximally general members of H consistent with D.

Specific boundary S

The **specific boundary** S, with respect to hypothesis space H and training data D, is the set of minimally general (i.e., maximally specific) members of H consistent with D.

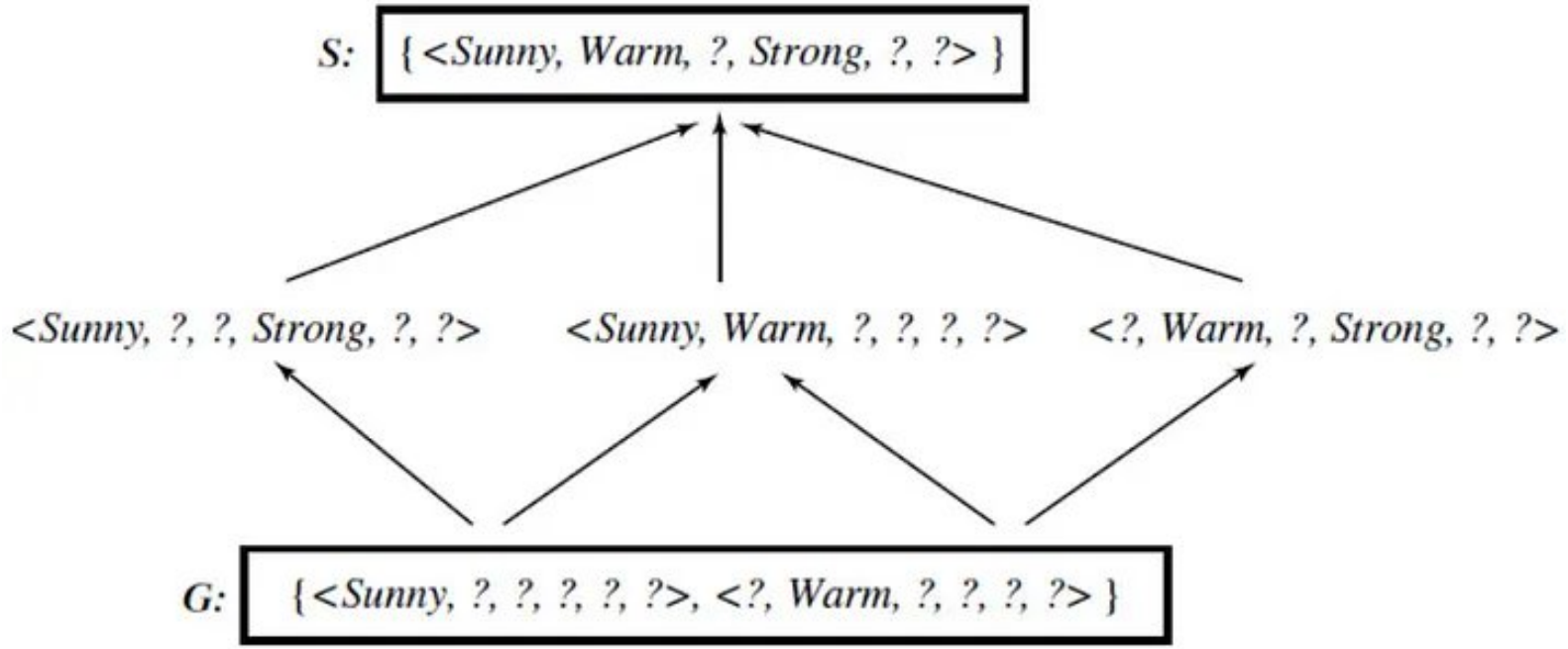
Version Space representation theorem

Let X be an arbitrary set of instances and Let H be a set of Boolean-valued hypotheses defined over X. Let c: X →{O, 1} be an arbitrary target concept defined over X, and let D be an arbitrary set of training examples {(x, c(x))). For all X, H, c, and D such that S and G are well defined,

$$VS_{H,D}=\{h \in H \mid (\exists s \in S)\ (\exists g \in G)\ (g \geq_g h \geq_g s)\}$$

The below figure shows the version space for enjoysport concept learning including both general and specific boundary sets.

Version Space representation theorem



From Version Space representation theorem, there exist a hypothesis s belongs to specific boundary set S, and there exist a hypothesis g belongs to general boundary set G, a hypothesis h belongs hypothesis space H holding the relationship "g is **more-general-than-or-equal-to** h and h is **more-general-than-or-equal-to** s" forms a version space.

Candidate Elimination algorithm

Initialize G to the set of maximally general hypotheses in H

Initialize S to the set of maximally specific hypotheses in H

For each training example d, do

- If d is a positive example
  - Remove from G any hypothesis inconsistent with d
  - For each hypothesis s in S that is not consistent with d
    - Remove s from S
    - Add to S all minimal generalizations h of s such that
      - h is consistent with d, and some member of G is more general than h
    - Remove from S any hypothesis that is more general than another hypothesis in S

- If d is a negative example
  - Remove from S any hypothesis inconsistent with d
  - For each hypothesis g in G that is not consistent with d
    - Remove g from G
    - Add to G all minimal specializations h of g such that
      - h is consistent with d, and some member of S is more specific than h
    - Remove from G any hypothesis that is less general than another hypothesis in G

Candidate Elimination algorithm

- The Candidate-Elimination algorithm computes the version space containing all hypotheses from H that are consistent with an observed sequence of training examples.
- It begins by initializing the version space to the set of all hypotheses in H; that is, by initializing the G boundary set to contain the most general hypothesis in H as

  G0 ← { <?, ?, ?, ?, ?, ?> }

  and initializing the S boundary set to contain the most specific hypothesis as

  S0 ← { <0, 0, 0, 0, 0, 0> }

Candidate Elimination algorithm

- These two boundary sets delimit the entire hypothesis space, because every other hypothesis in H is both more general than S0 and more specific than G0.
- As each training example is considered, the S and G boundary sets are generalized and specialized, respectively, to eliminate from the version space any hypotheses found inconsistent with the new training example.
- After all examples have been processed, the computed version space contains all the hypotheses consistent with these examples and only these hypotheses.

Candidate Elimination algorithm- Example

Training examples D

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Candidate Elimination algorithm- Example

Write most specific and generic boundaries

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: $\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset. \emptyset \rangle$

Activate Windov
Go to Settings to acti

G_0: $\langle ?, ?, ?, ?, ?, ? \rangle$

Iteration 1

- Consider first example- +ve.
- First goto generic and check hypo at generic boundary is consistent/not
- if consistent

  Retain it

- else

  Write next general

- In this case,consistent, therefore retain

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: ⟨∅, ∅, ∅, ∅, ∅. ∅⟩

G_0: G_1: ⟨?, ?, ?, ?, ?, ?⟩

Iteration 1

Then goto specific->

if not consistent,write next specific

Here not consistent, therefore write next specific

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: ⟨∅, ∅, ∅, ∅, ∅. ∅⟩

S_1: ⟨*Sunny,Warm, Normal, Strong, Warm, Same*⟩

G_0: G_1: ⟨?, ?, ?, ?, ?, ?⟩

Iteration 2

second example, +ve

first goto generic->Consistent

Retain

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: ⟨∅, ∅, ∅, ∅, ∅. ∅⟩

S_1: ⟨Sunny, Warm, Normal, Strong, Warm, Same⟩

G_0 G_1 G_2 ⟨?, ?, ?, ?, ?, ?⟩

Iteration 2

Then goto specific->Inconsistent

write new specific

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: ⟨∅, ∅, ∅, ∅, ∅. ∅⟩

S_1: ⟨*Sunny,Warm, Normal, Strong, Warm, Same*⟩

S_2: ⟨*Sunny,Warm, ?, Strong, Warm, Same*⟩

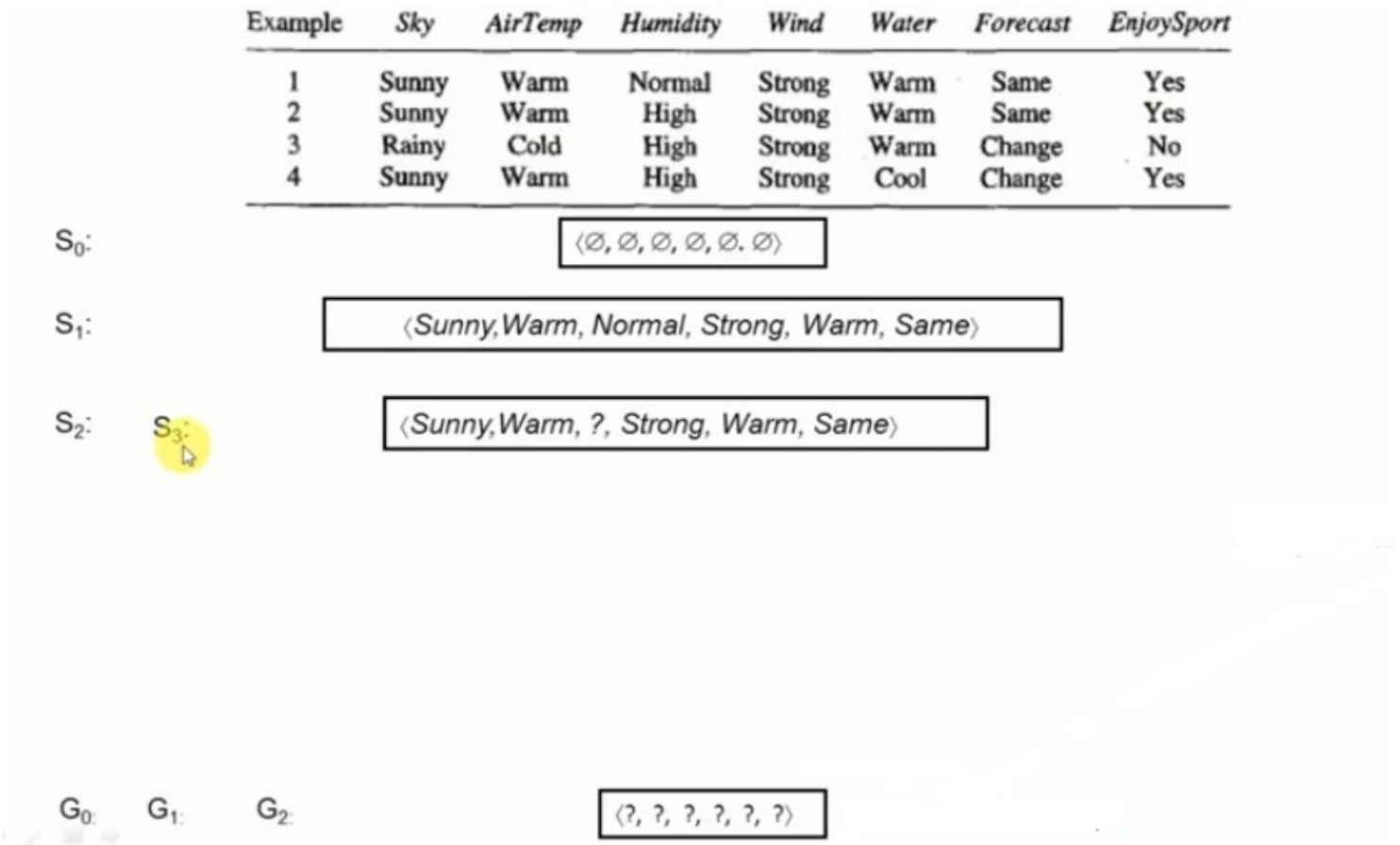
G_0 G_1 G_2: ⟨?, ?, ?, ?, ?, ?⟩

Iteration 3

Third example, -ve

First goto specific->consistent(No and expected also No). Therefore retain

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes



Iteration 3

goto generic->inconsistent

write all hypothesis which are consistent till now

replace each ? mark with opposite of the value in the example

check whether all of these are consistent/not with all examples till now(upto 3rd eg)

if not, remove them (in red color)

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S_0: ⟨∅, ∅, ∅, ∅, ∅. ∅⟩

S_1: ⟨Sunny,Warm, Normal, Strong, Warm, Same⟩

S_2: S_3: ⟨Sunny,Warm, ?, Strong, Warm, Same⟩

G_3: ⟨Sunny,?,?,?,?,?⟩ ⟨?,Warm,?,?,?,?⟩ ⟨?,?,Normal,?,?,?⟩ ⟨?, ?,?,?,Cool,?⟩ ⟨?,?,?,?,?,Same⟩

G_0: G_1: G_2: ⟨?, ?, ?, ?, ?, ?⟩

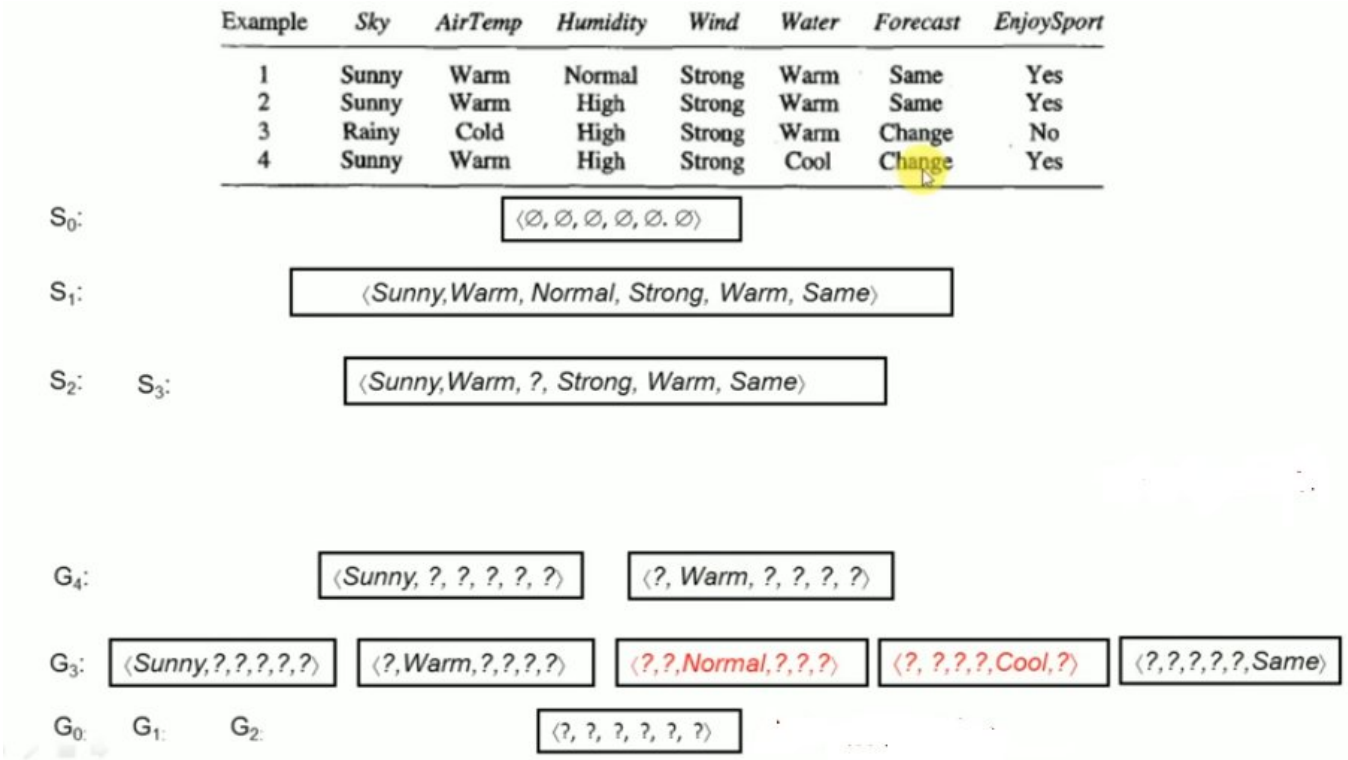
Iteration 4

Last example- +ve

First goto generic

retain all consistent

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

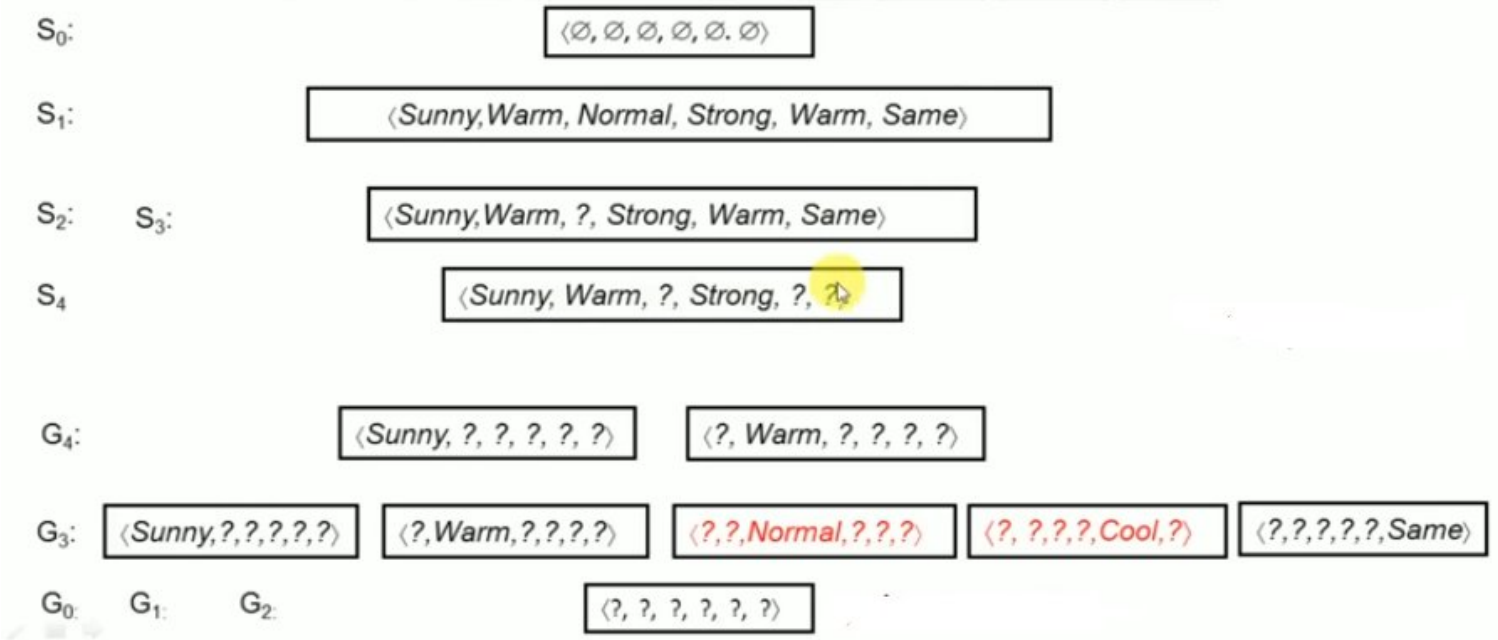


Iteration 4

Then goto specific->Inconsistent

replace with ? in the specific

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

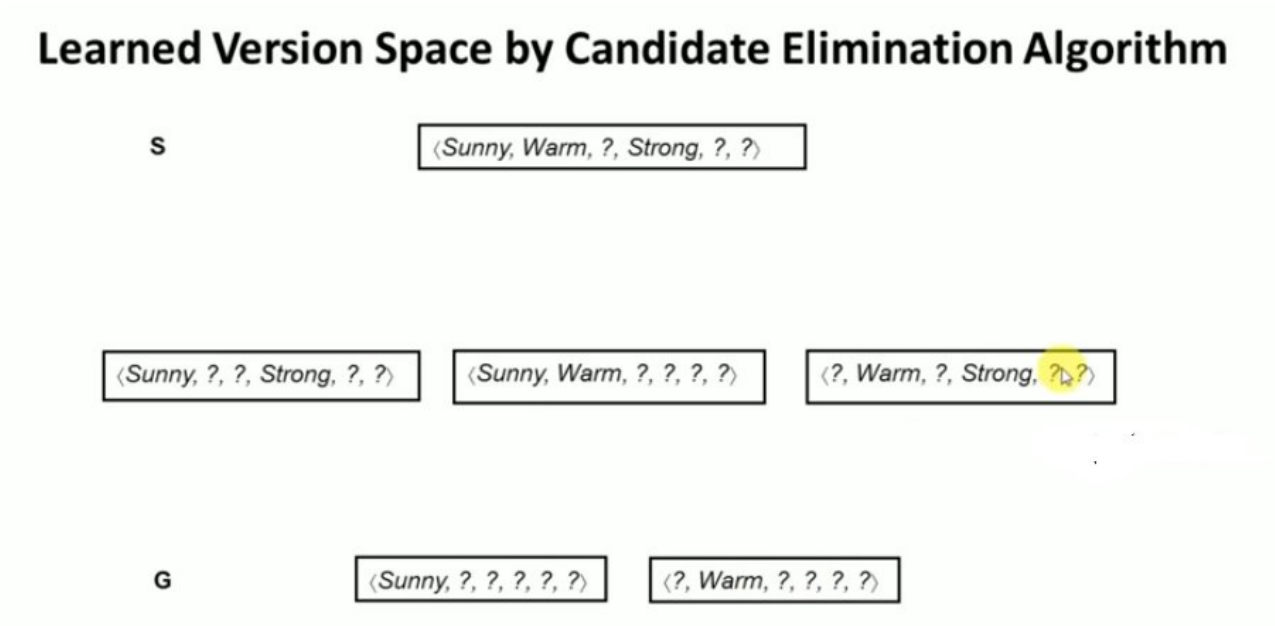


Learned Version Space by Candidate Elimination Algorithm

S4 and G4 are generic and specific boundaries

since more than one hypothesis,write some more possibilities

consider one from each and compare for the each mismatch and replce ? with specific



Total consistent hypothesis for the given set=6

ML Key Concepts

Data: Features

Features: Measurable data that helps make predictions

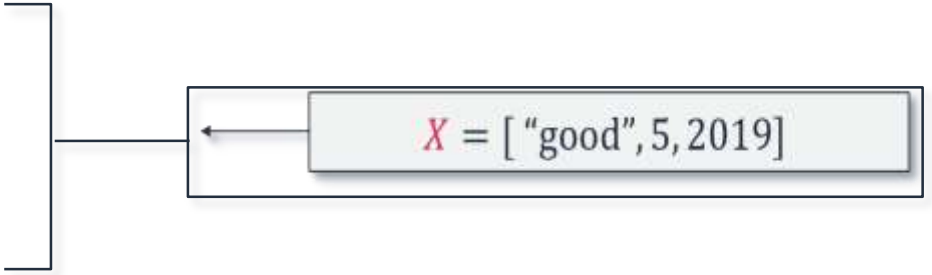
- Single feature: x
- Collection of features: Vector X

Example book feature: Year sold



Example set of book features:

- Book condition
- Month sold
- Year sold

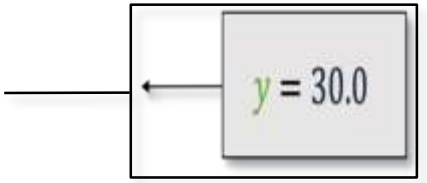


Data: Label

Label: The output that you want to predict

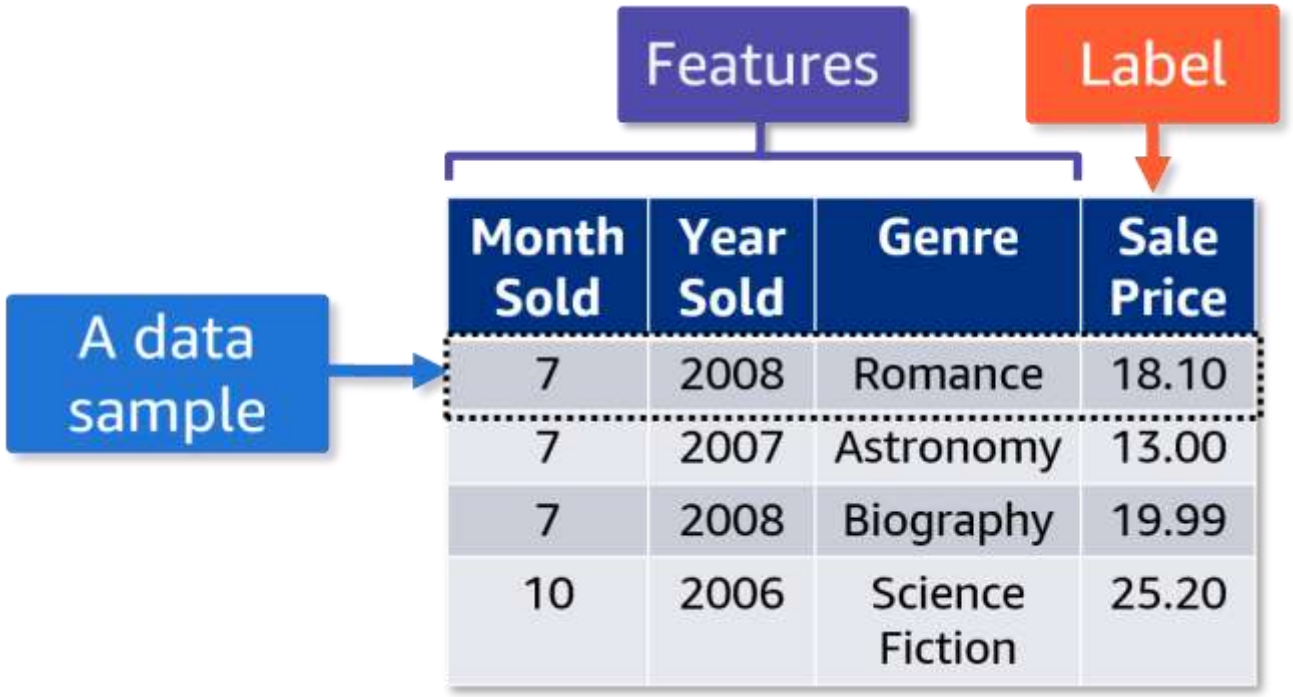
- Sometimes called the *target*
- For a single data sample, usually denoted with the variable y

A particular book target: **book sale price**



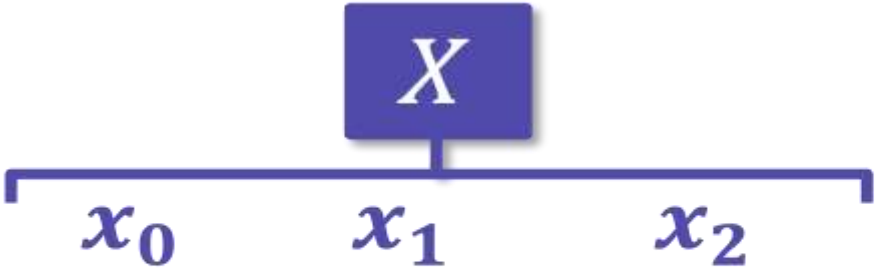
Tabular data: Features and labels

- Rows represent each data sample.
- Columns represent each feature and the label.



Tabular data: Vectors and variables

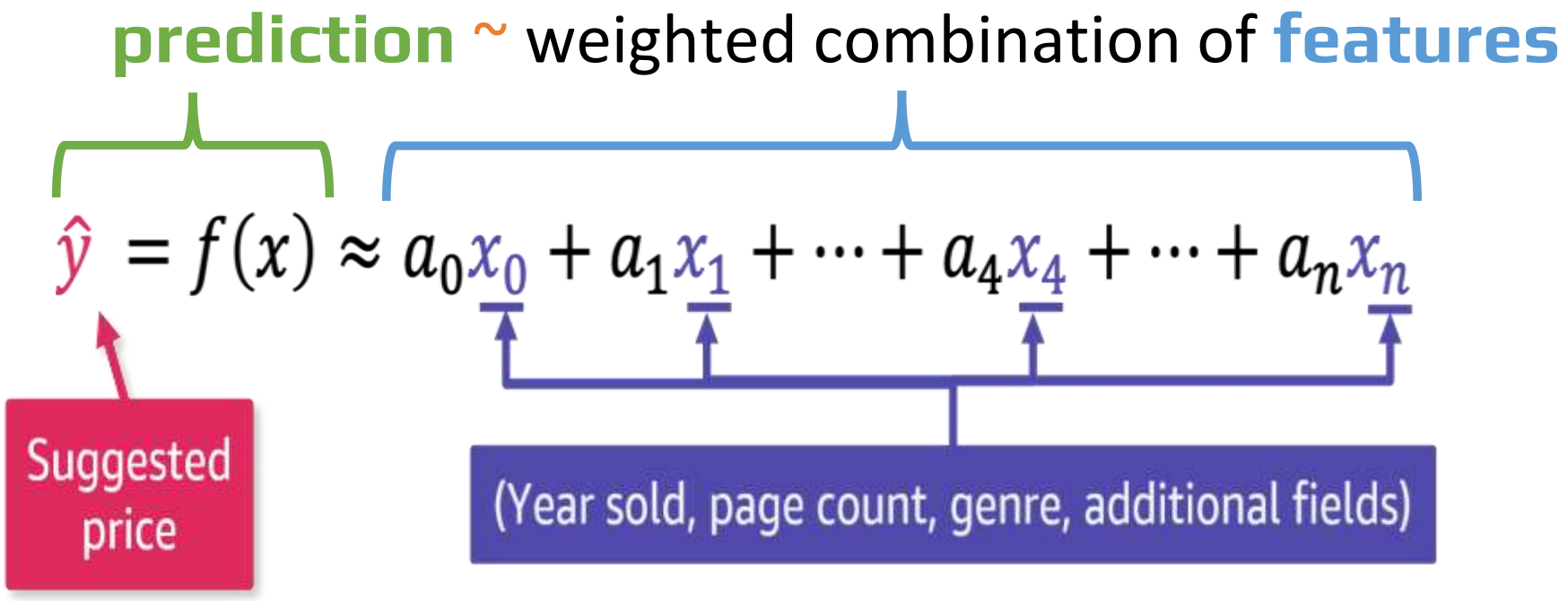
- Features are represented by a vector X.
- Labels are represented by a variable y.



Month Sold	Year Sold	Genre	Sale Price
7	2008	Romance	18.10
7	2007	Astronomy	13.00
7	2008	Biography	19.99
10	2006	Science Fiction	25.20

ML model: An example

The goal is to predict a good suggested book price.

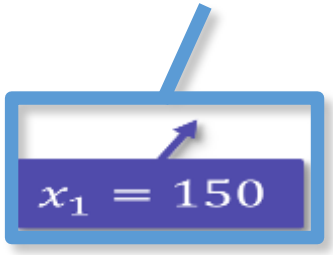


ML model: Feature 1

- x_1: Number of pages for a given book

prediction ~ weighted combination of **features**

$$\hat{y} = f(x) \approx a_0 x_0 + a_1 * 150 + \cdots + a_4 x_4 + \cdots + a_n x_n$$



ML model: Feature 4

- x_1: Number of pages for a given book
- x_4: Is this book fiction?

prediction ~ weighted combination of **features**

$$\hat{y} = f(x) \approx a_0 x_0 + a_1 * 150 + \cdots + a_4 * 1 + \cdots + a_n x_n$$

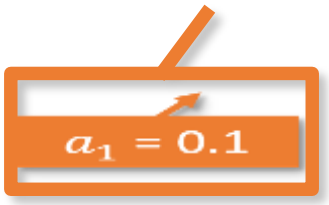
$x_1 = 150$

x_4 = yes = 1

ML model: Weight 1

- **Weights**, in this model, determine how much influence a given feature has on the output (**prediction**)
  - a_1: Weight for "Number of pages" feature x_1

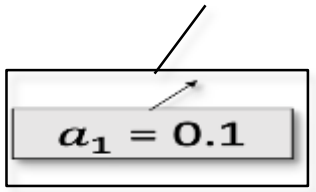
$$\hat{y} = f(x) \approx a_0 x_0 + 0.1 * 150 + \cdots + a_4 * 1 + \cdots + a_n x_n$$

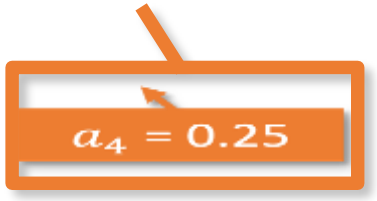


ML model: Weight 4

- **Weights**, in this model, determine how much influence a given feature has on the output (**prediction**)
  - a_1: Weight for "Number of pages" feature x_1
  - a_4: Weight for "Is fiction" feature x_4

$$\hat{y} = f(x) \approx a_0 x_0 + 0.1 * 150 + \cdots + 0.25 * 1 + \cdots + a_n x_n$$





ML model: Prediction

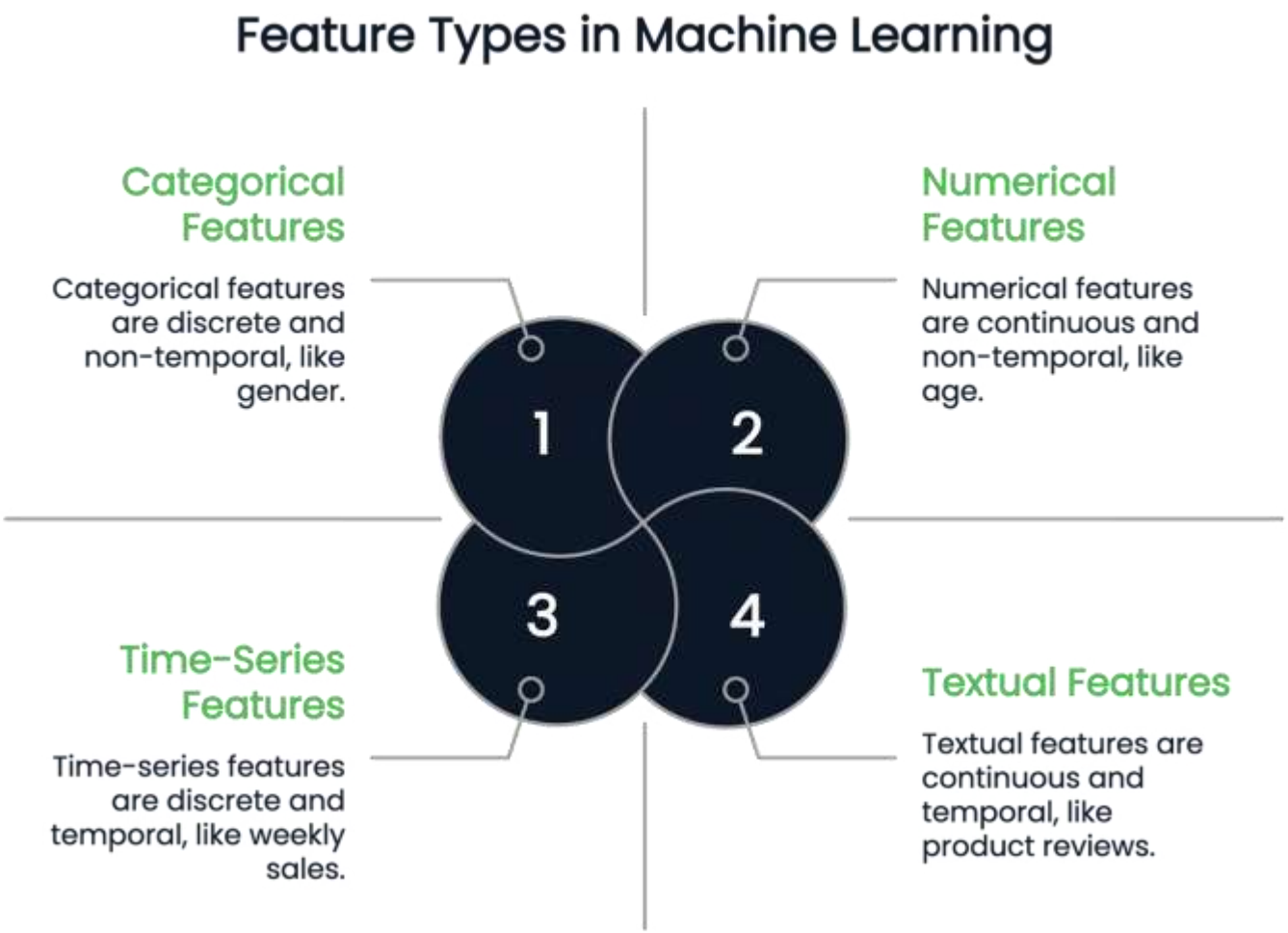
For this model, use these two features to determine a **price to suggest**.

$$\hat{y} = f(x) \approx 0.1 * 150 + 0.25 * 1 = 15.25$$

Suggested price of a 150-page book in the fiction genre is $15.25

Note: This example shows how a model can make predictions; the method to learn these weights from data will be explained later.

Types of Features in Machine Learning



Types of Features in Machine Learning

1. **Numerical features**

- Numerical features represent data in numbers.
- They are continuous quantitative variables.
- Examples include height, age, and salary.

1. **Categorical features**

- A categorical column can contain only discrete features.
- For example, a person's gender is a categorical column.
- Categorical variables are further divided into: binary and non-binary types.
  - Binary variables have two possible categories
  - Non-binary features can have multiple categories

Types of Features in Machine Learning

3. Textual

- Textual columns contain only text data.
  - Examples : product reviews or product description columns in a retail dataset.

4.Time-series features

- Time-series features represent timely data
  - Example:Weekly sales or stock price fluctuations over a year.

Machine Learning Pipeline

Step 1: Data Collection and Preprocessing

Step 2: Feature Engineering

Step 3: Data splitting

Step 4: Model Selection & Training

Step 5: Model evaluation & Optimization

Step 6: Model Deployment

Step 7: Continuous learning & Monitoring

Defining the problem

Define the objective:

A specific outcome that you want to use ML to achieve.

For example, to help someone achieve something or some business goal.

Increase revenue by ensuring all products in catalog have prices & can be surfaced to customer.

Translate the objective to an ML problem:

Express the objective in ML terms; for example, predicting or calculating the probability.

Predict the price (continuous numerical value) for all products in the catalog where the price is missing.

Step 1: Data Collection and Preprocessing

1. The first step is to collect data from relevant sources, which might include databases, CSV files, APIs, or web scraping. The quality and quantity of the data you collect will significantly impact the performance of your machine learning model.
2. After collecting the data, understand the features (columns), the target variable (what you want to predict), the types of data (numeric, categorical, text), and identify potential issues such as missing values, outliers, or inconsistencies.
3. Data preprocessing is the process of transforming raw data into a clean and usable format.

Step 2: Feature Engineering

Feature Engineering is the process of selecting, creating or modifying features like input variables or data to help machine learning models learn patterns more effectively.
It involves transforming raw data into meaningful inputs that improve model accuracy and performance.

Step 3: Data splitting

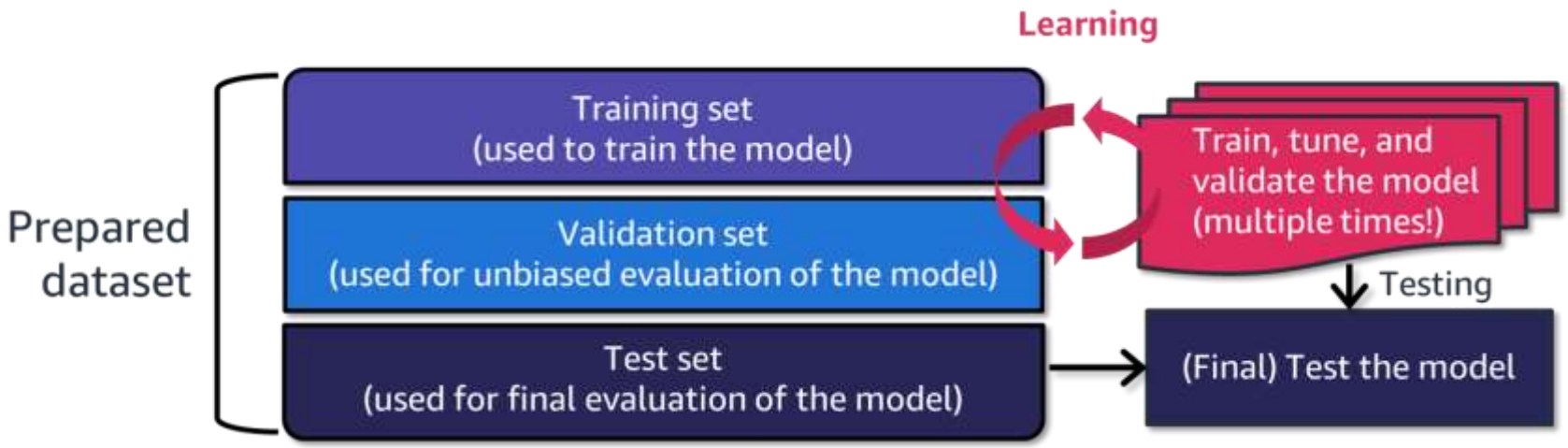
Train-Test Split:

- **Split Data**: To evaluate the performance of a model, the dataset is typically split into a training set (used to train the model) and a testing set (used to evaluate the model's performance). A common split ratio is 70–30 or 80–20, depending on the size of the dataset.
- **Why Split Data?**: Splitting the data helps prevent overfitting, where a model performs well on training data but poorly on unseen data. By testing the model on a separate set of data (the test set), you can get a better estimate of its real-world performance.

Step 4: Model Selection & Training

- **Supervised models:** Train (learn) by comparing the error between predictions and true values (labels).
- **Unsupervised models:** Train (learn) by comparing the results of grouping similar things together. Similarity can be defined in different ways; for example, distance or intersections of features.

Step 5: Model evaluation & Optimization



Step 6: Model Deployment

Model Deployment is the process of taking a trained machine learning model and making it available in a real-world environment, so it can receive new data and make predictions for users or applications.

Step 7: Continuous learning & Monitoring

Continuous Learning: Retrain or update the model periodically using new data/feedback to keep it accurate as real-world patterns change.

Monitoring: Continuously track model performance (accuracy, drift in data/predictions) on live data to detect when the model starts degrading.

Data processing: Preparing the data

- Data preprocessing is the process of transforming raw data into a clean and usable format.
- This involves a series of tasks aimed at improving the quality of the data, making it more suitable for machine learning algorithms.

- Convert all values to numbers.

Sale Price	Type	Description	imgID
6.99	Grocery	Sugar Free …	51Q696oAVL
26.39	Health	Sports Recovery …	31WGV435lL
9.79	Grocery	Simple Jam…	41c3R441GfL
49	Beauty	Therapy Oil …	51Mv1232QK

Sale Price	Type	Description	imgID
6.99	26	424	1068
26.39	40	863	284
9.79	26	1039	653
49	51	492	942

Tables 1 and 2: Example of conversion from an original dataset to only numerical values

Why Preprocess the Data?

Noisy: Containing errors or outliers.

E.g., Salary = "-10" Noisy data may come from

- Human or computer error at data entry.
- Errors in data transmission.

Missing: lacking certain attribute values or containing only aggregate data. **E.g., Occupation = "".** Missing (Incomplete) may data come from

- "Not applicable" data value when collected.
- Human/hardware/software problems.

Inconsistent: Data inconsistency meaning is that different versions of the same data appear in different places.Inconsistent data may come from

- Errors in data entry.
- Merging data from different sources with varying formats.
- Differences in the data collection process.

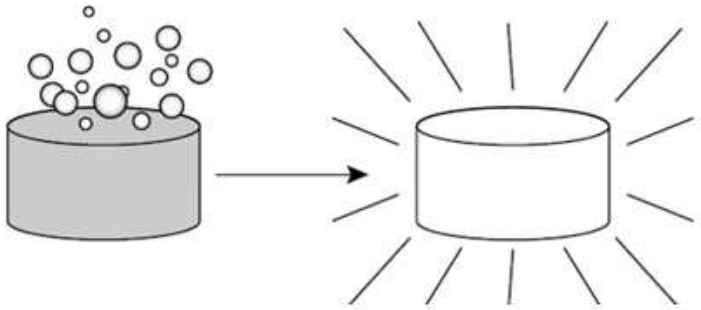
Steps in Data Preprocessing

1. Data Cleaning

Data cleaning is the process of identifying and correcting errors or inconsistencies in the data to ensure it is accurate and complete. The objective is to address issues that can distort analysis or model performance.

For example:

- **Handling missing values:** Using strategies like mean/mode imputation, deletion, or predictive models to fill in or remove missing data.
- **Removing duplicates:** Eliminating duplicate records to ensure each entry is unique and relevant.
- **Correcting inconsistent formats:** Standardizing formats (e.g., date formats, string cases) to maintain consistency.



Missing Values

Handling missing data : Missing data can negatively impact the performance of a machine learning model or analysis. There are several strategies to handle missing values effectively:

- **Imputation:** This technique involves filling in missing values with a calculated estimate, such as the mean, median, or mode of the available data. Advanced methods include predictive modeling, where missing values are predicted based on relationships within the data.
- **Deletion:** Removing rows or columns with missing values is a straightforward solution. However, it should be used cautiously as it can lead to loss of valuable data, especially if many entries are missing.
- **Modeling missing values:** In cases where the missing data pattern is more complex, machine learning models can predict the missing values based on the rest of the dataset. This can improve accuracy by incorporating relationships between different variables.

Impute Missing Data:

- **Mean/Median Imputation**: Replace missing values with the mean or median of the column, which is useful for numerical data.
- **Mode Imputation**: For categorical data, replace missing values with the most frequent value (mode).
- **Predictive Imputation**: Use models like k-Nearest Neighbors (KNN) to predict and fill in missing values based on other available data.
- **Forward/Backward Fill**: In time series data, missing values can be filled based on previous (forward fill) or subsequent (backward fill) values.

Duplicate Data Removal:

Identify Duplicates: Duplicates can occur due to data entry errors or merging datasets. They can be identified using functions like `pandas.DataFrame.duplicated()` in Python.

Remove Duplicates: Once identified, duplicates should be removed to prevent them from skewing the analysis or model results.

Outlier detection and removal

Outliers are extreme values that deviate significantly from the rest of the data, which, like missing values, can distort analysis and model performance. Various techniques can be used to detect and handle outliers:

- **Z-Score method:** This approach measures how many standard deviations a data point is from the mean. Data points beyond a certain threshold (e.g., ±3 standard deviations) can be considered outliers.
- **Interquartile range (IQR)**: IQR is the range between the first quartile (Q1) and the third quartile (Q3). Values beyond 1.5 times the IQR above Q3 or below Q1 are considered outliers.
- **Visual techniques:** Visualization methods like box plots, scatter plots, or histograms can help detect outliers in a dataset. Once identified, outliers can either be removed or transformed, depending on their influence on the analysis.

Noisy Data

Noise is a random error or variance in a measured variable.

Data smoothing techniques are used to eliminate noise and extract the useful patterns.

Noisy Data- The different techniques used for data smoothing are:

1.Binning: Binning methods smooth a sorted data value by consulting its "neighbourhood," that is, the values around it. The sorted values are distributed into several "buckets," or bins.

There are three kinds of binning. They are:

- Smoothing by Bin Means:In this method, each value in a bin is replaced by the mean value of the bin.
- Smoothing by Bin Medians:In this method, each value in a bin is replaced by the median value of the bin.
- Smoothing by Bin Boundaries:In this method, the minimum and maximum values in each bin are identified as the bin boundaries. Each bin value is then replaced by the closest boundary value.

Example:

Sorted data for price (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34

Partition into (equal-frequency) bins:

Bin 1: 4, 8, 15

Bin 2: 21, 21, 24

Bin 3: 25, 28, 34

Smoothing by bin means:

Bin 1: 9, 9, 9

Bin 2: 22, 22, 22

Bin 3: 29, 29, 29

Smoothing by bin medians:

Bin 1: 8, 8, 8

Bin 2: 21, 21, 21

Bin 3: 28, 28, 28

Smoothing by bin boundaries:

Bin 1: 4, 4, 15

Bin 2: 21, 21, 24

Bin 3: 25, 25, 34

2. Regression: Data Smoothing can also be done by regression, a technique that used to predict the numeric values in a given data set. It analyses the relationship between a target variable (dependent) and its predictor variable (independent).

- Regression is a form of a supervised machine learning technique that tries to predict any continuous valued attribute.

3. Clustering: It supports in identifying the outliers. The similar values are organized into clusters and those values which fall outside the cluster are known as outliers.

Data encoding

When working with categorical data, encoding is necessary to convert categories into numerical representations that machine learning algorithms can process.

Encoding Techniques:

- **Label Encoding**:
  - Label encoding converts categorical variables into integers.
  - For example, the categories “red”, “green”, “blue” could be encoded as 0, 1, and 2, respectively. This method is simple but can introduce ordinal relationships where none exist.
- **One-Hot Encoding**:
  - One-hot encoding converts categorical variables into a binary matrix, where each category is represented by a binary vector.
  - For example, “red”, “green”, “blue” would become three binary columns, with only one column being 1 for each category and the others being 0.

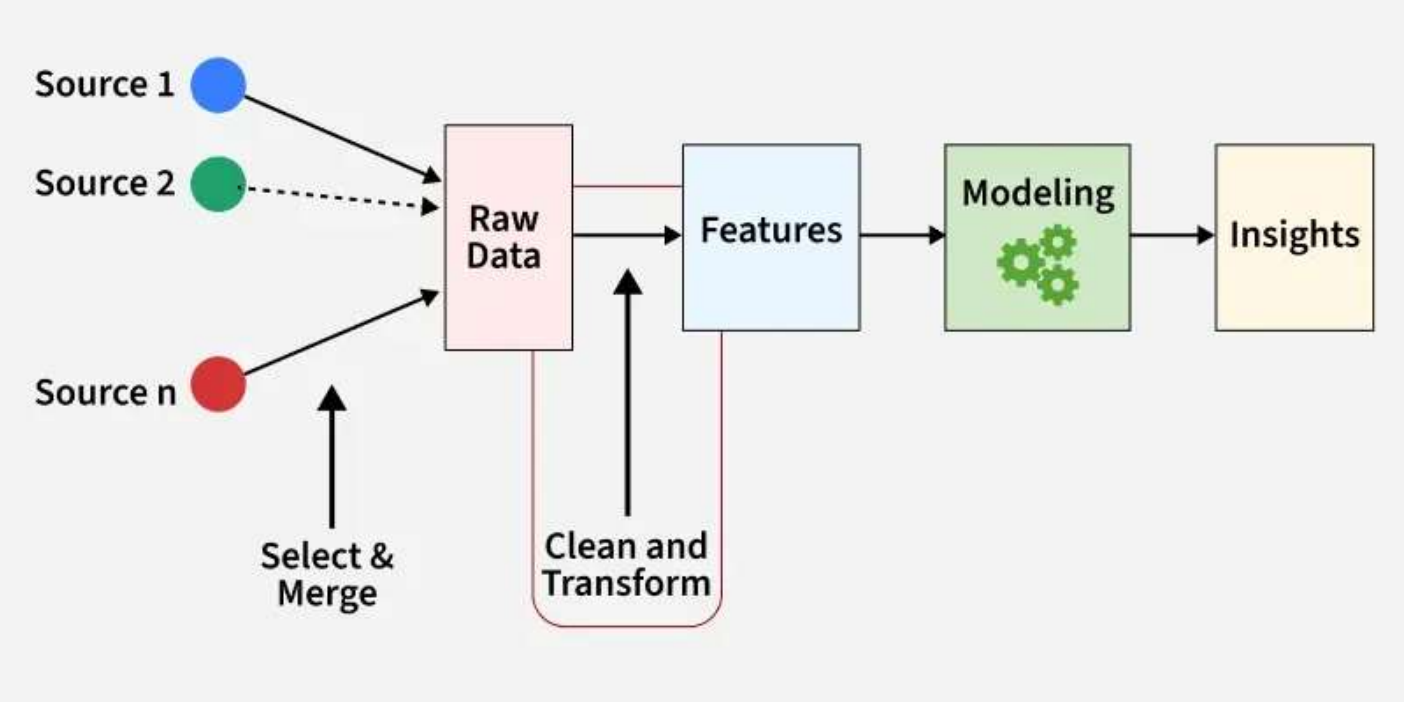
Data scaling and normalization

Scaling and normalization ensure that numerical features are on a similar scale.

- **Min-max scaling:** This technique scales data to a specified range, typically 0 to 1. It's useful when all features need to have the same scale.
- **Normalization**: Normalization scales the data to a range between 0 and 1 or -1 and 1. This is particularly useful when the model relies on distance metrics, like K-Nearest Neighbors (KNN) or Neural Networks.
- **Standardization**: Standardization rescales data so that it has a mean of 0 and a standard deviation of 1, which is important for algorithms like Support Vector Machines (SVM) or logistic regression.

FEATURE ENGINEERING

- Feature Engineering is the process of selecting, extracting, and transforming the most relevant features from the available data to build more accurate and efficient machine learning models.



Importance of Feature Engineering

- **Improve accuracy**: Choosing the right features helps the model learn better, leading to more accurate predictions.
- **Reduce overfitting**: Using fewer, more important features helps the model avoid memorizing the data and perform better on new data.
- **Boost interpretability**: Well-chosen features make it easier to understand how the model makes its predictions.
- **Enhance efficiency**: Focusing on key features speeds up the model's training and prediction process, saving time and resources.

FEATURE ENGINEERING

- Feature Creation
- Feature Transformation & Scaling
- Feature Extraction
- Feature Selection

1. Feature Creation:

Feature creation involves generating new features from domain knowledge or by observing patterns in the data. It can be:

- **Domain-specific**: Created based on industry knowledge like business rules.
- **Data-driven**: Derived by recognizing patterns in data.
- **Synthetic**: Formed by combining existing features.

2. Feature Scaling & Transformation

Transformation adjusts features to improve model learning:

- **Normalization & Scaling**: Adjust the range of features for consistency.
- **Encoding**: Converts categorical data to numerical form i.e one-hot encoding.
- **Mathematical transformations**: Like logarithmic transformations for skewed data.

Feature scaling

Feature scaling ensures that numerical features lie within a standardized range, preventing some features from dominating the learning process due to their larger values.

Eg: datasets in which different columns have different units – like one column can be in kilograms, while another column can be in centimeters.

Eg: we can have columns like income which can range from 20,000 to 100,000, while an age column which can range from 0 to 100.
When we feed these features to the model as is, there is every chance that the income will influence the result more due to its larger value. But this doesn't necessarily mean it is more important as a predictor. So, to give importance to both Age, and Income, we need feature scaling.

1. **Normalization (min-max scaling):**

This method scales all feature values to fall within 0 and 1. It subtracts the minimum value of the column from each data point and then divides it by the range of that column, which is the difference between the maximum and minimum values.

Scaled value =(datapoint - min(column))/(max(column) - min(column))

1. **Standardization (Z-score scaling):**

This transforms all features to have a mean of 0 and a standard deviation of 1. The formula is: the mean of a column is subtracted from each data point of that column, and the residue is divided by the standard deviation of that feature.

Scaled value =(datapoint - mean(column))/(std(column))

Feature Extraction

- In feature extraction new features are created from a combination of original features.
- Some of the commonly used operators for combining the original fatures include:

    1. For Boolean features: Conjunctions, Disjunctions etc.
    2. For nominal features: Cartesian product etc.
    3. For numerical features: Min, Max, addition etc.

Feature Extraction

Feat A	Feat B	Feat C	Feat D
34	34.5	23	233
44	45,56	11	3.44
78	22.45	21	4.5
22	65.22	11	322
11	122.35	63	23.2

Feat 1	Feat 2
41.25	185.80
54.20	53.12
43.73	35.79
37.02	239.42
113.39	167.74

3. Feature Extraction:

- **Dimensionality reduction**: Techniques like PCA reduce features while preserving important information.
- **Aggregation & Combination**: Summing or averaging features to simplify the model.

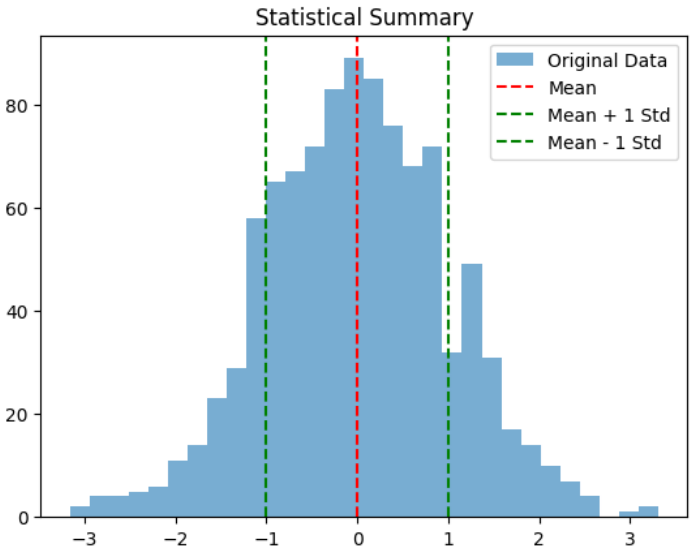
Importance of Feature Extraction

- Reduces computation by simplifying complex raw data.
- Improves model performance using relevant features.
- Provides better insights by removing noise.
- Helps prevent overfitting by reducing feature complexity.

Key Techniques for Feature Extraction

1. Statistical Methods

- **Mean:** The average value of a dataset.
- **Median:** The middle value when it is sorted in ascending order.
- **Standard Deviation:** A measure of the spread or dispersion of a sample.
- **Correlation and Covariance:** Measures of the linear relationship between two or more factors.



These statistical methods can be used to represent the center trend, spread and links within a collection.

2. Dimensionality Reduction

Dimensionality reduction reduces the number of features without losing important information. Some popular methods are:

- **Principal Component Analysis**: It transforms original features into new orthogonal components that capture maximum variance in the data.
- **Linear Discriminant Analysis (LDA):** It finds the best combination of features to separate different classes, maximizing class separability for better classification.

3. Feature Extraction for Textual Data

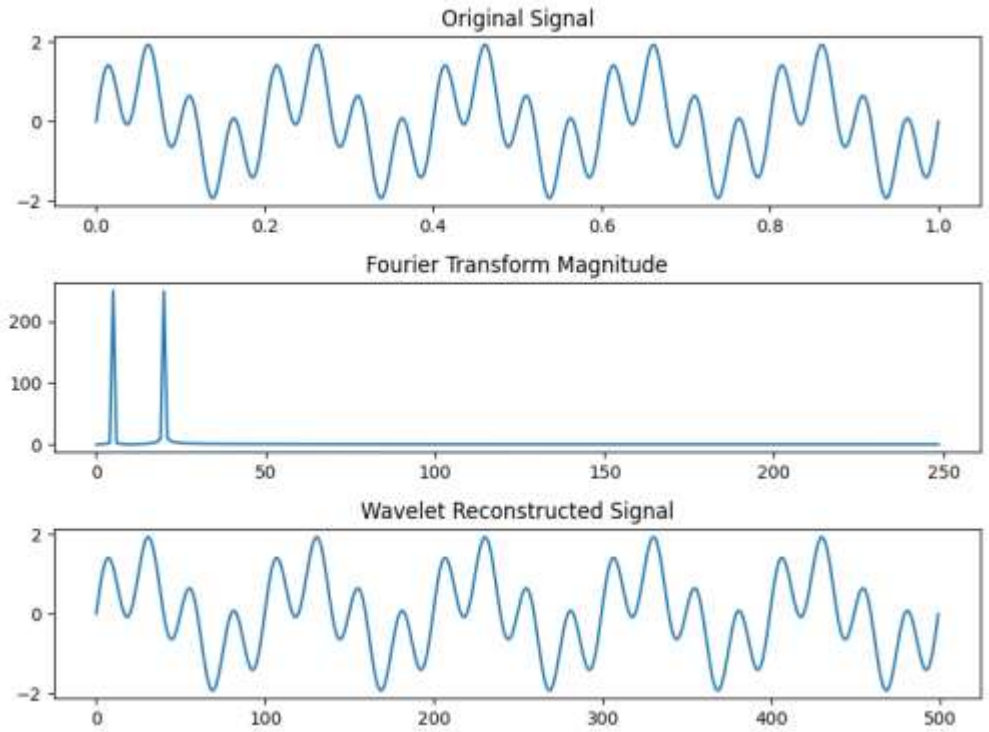
In Natural Language Processing (NLP), we often convert raw text into a format that machine learning models can understand.

1. **Bag of Words (BoW):** Represents a document by counting word frequencies, ignoring word order, useful for basic text classification.
2. **Term Frequency-Inverse Document Frequency (TF-IDF)**: Adjusts word importance based on frequency in a specific document compared to all documents, highlighting unique terms.

4. Signal Processing Methods

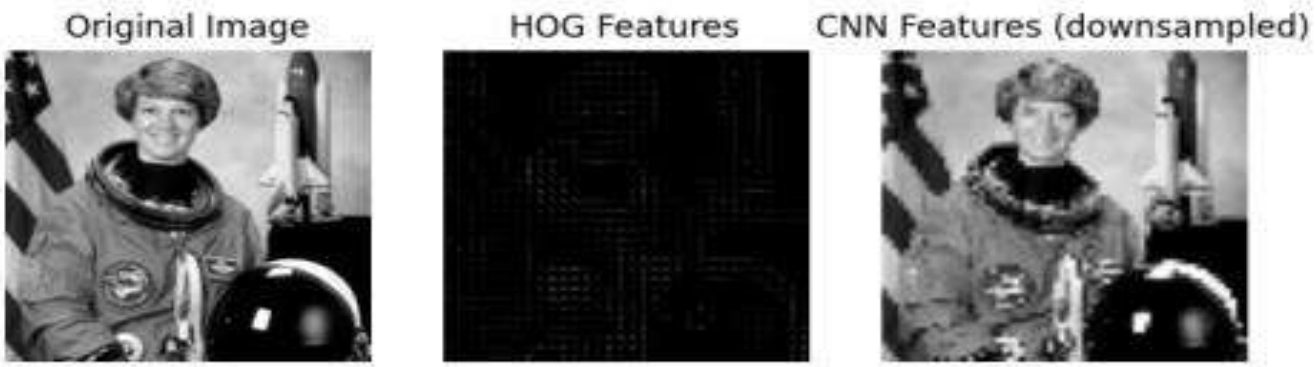
It is used for analyzing time-series, audio and sensor data:

1. **Fourier Transform:** It converts a signal from the time domain to the frequency domain to analyze its frequency components.
2. **Wavelet Transform:** It analyzes signals that vary over time, offering both time and frequency information for non-stationary signals.



5. Image Data Extraction

1. **Histogram of Oriented Gradients (HOG):** This technique finds the distribution of intensity gradients or edge directions in an image. It's used in object detection and recognition tasks.
2. **Convolutional Neural Networks (CNN) Features:** They learn hierarchical features from images through layers of convolutions, ideal for classification and detection tasks.



Advantages

- **Simplifies Data:** Reduces complex data into a manageable form for easier analysis and visualization.
- **Boosts Model Performance:** Removes irrelevant data, making algorithms faster and more accurate.
- **Highlights Key Patterns:** Filters out noise to focus on important features for quicker insights.
- **Improves Generalization:** Helps models perform better on new, unseen data by emphasizing informative features.
- **Speeds Up Training and Prediction:** Fewer features mean faster model training and real-time predictions.

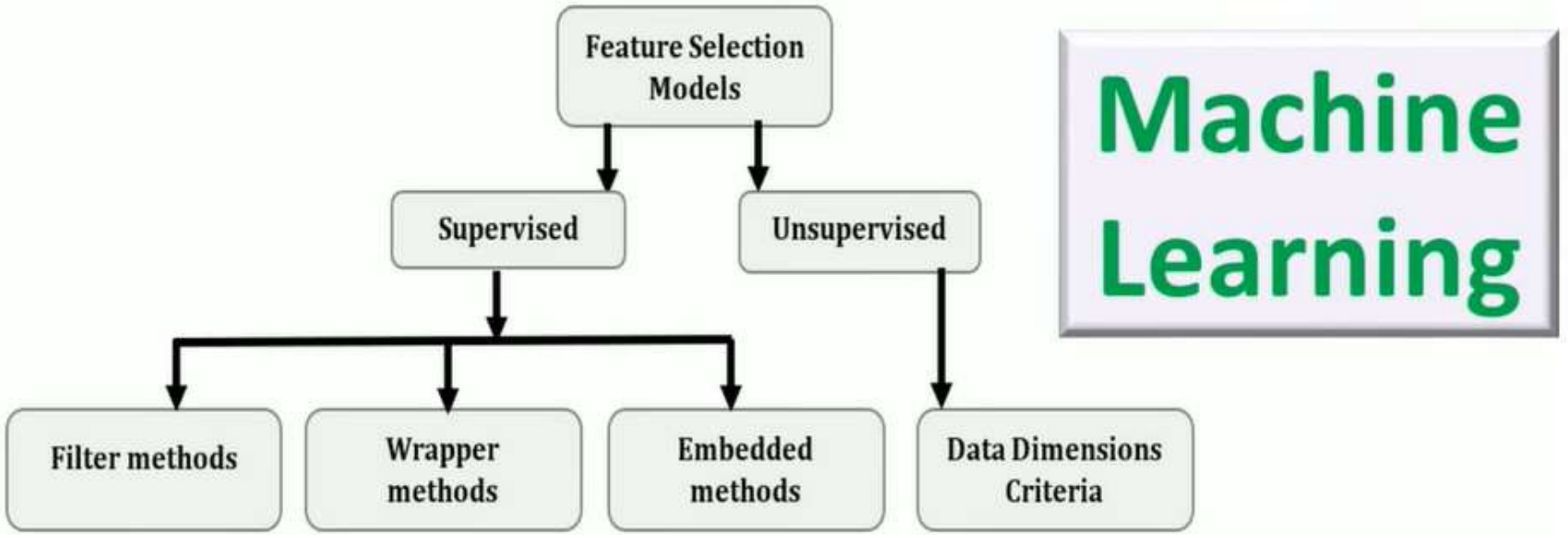
Challenges

- **Managing High-Dimensional Data:** Extracting relevant features from large, complex datasets can be difficult.
- **Risk of Overfitting or Underfitting:** Too many or too few features can hurt model accuracy and generalization.
- **Computational Costs:** Complex methods may require heavy resources, limiting use with big or real-time data.
- **Redundant or Irrelevant Features:** Overlapping or noisy features can confuse models and reduce efficiency.

4. Feature Selection:

Feature selection involves selecting the most relevant features and removing redundant or irrelevant ones, which helps reduce overfitting and improve efficiency.

Feature Selection



Feature Selection:

- Machine learning (ML) is a subfield of artificial intelligence that allows computers to learn without being explicitly programmed.
- Usually, machine learning model is built with help of dataset.
- A dataset is usually represented in a tabular form: rows, and columns are features.

Address	Number of Rooms	House Age	owner	price

Feature Selection:

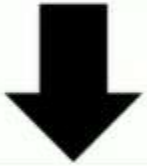
- In machine learning, feature selection selects the most relevant subset of features from the original feature set by dropping redundant, noisy, and irrelevant features.
- Feature selection is used today in many applications like Object detection, NLP, remote sensing, image retrieval, etc.

Why do we need to do feature selection

- If there are too many features, our model can may become weak or generate some misleading patterns.
- That happens because, usually, some features aren't correlated with the target variable and represent noise.
- If our model outputs predictions based on such features, its accuracy is likely to be subpar.
- Also, a dataset with many columns slows down the training process.

Why do we need to do feature selection

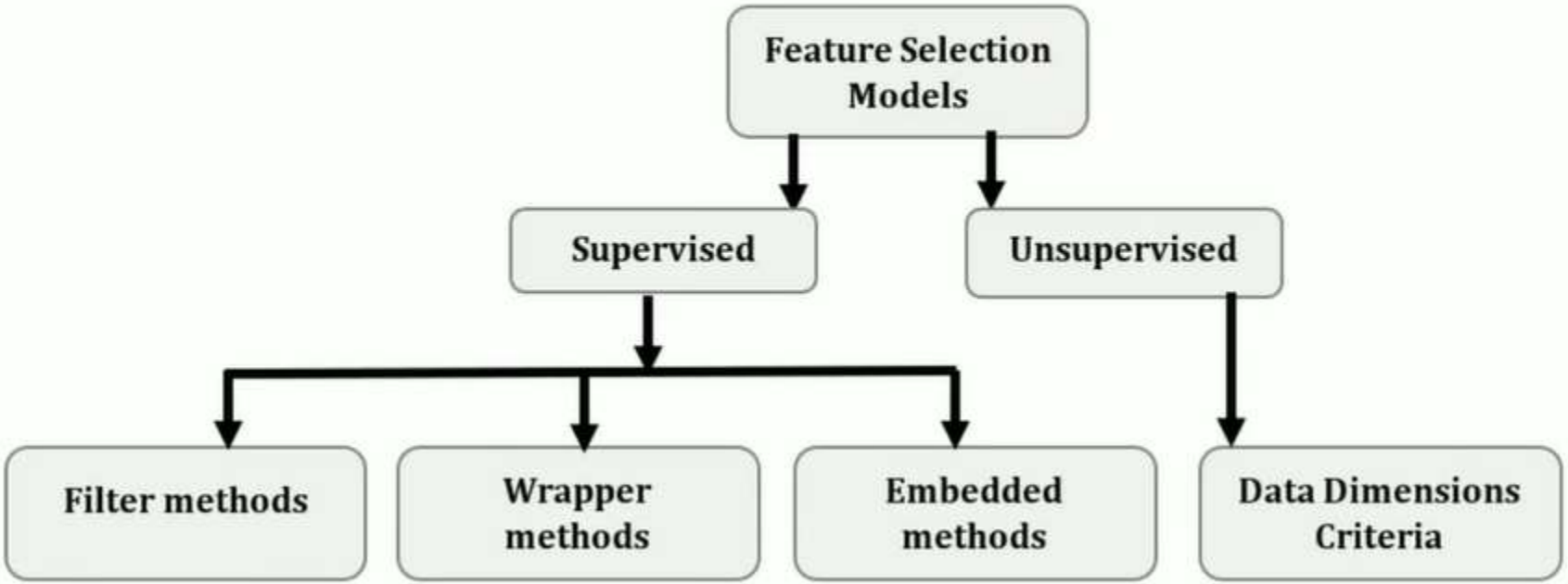
Address	Number of Rooms	House Age	owner	price



Address	Number of Rooms	House Age	Price

Feature Selection:

- There are several methods of doing feature selection:



Feature Selection:

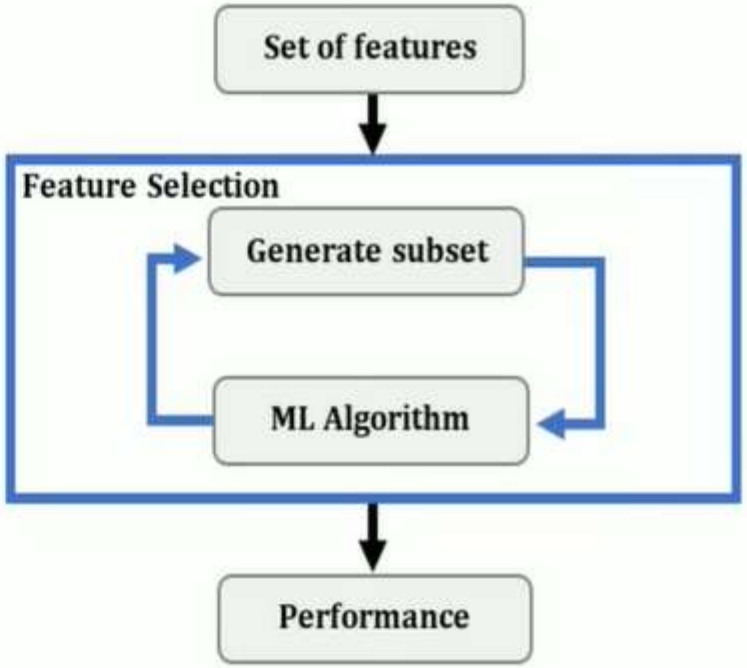
- **Unsupervised Methods**
- Unsupervised feature selection methods are applied to unlabeled data.
- An unsupervised selection method rates each feature dimension according to a number of factors, including entropy, variance, and the capacity to maintain local similarity.

Feature Selection:

- **Supervised Methods**
- On the other hand, we use supervised feature selection methods on labeled data.
- They determine the features that are expected to maximize the supervised model's performance.
- Supervised feature selection methods can be split into three primary categories based on the feature selection strategy.

Wrapper Method

- **Wrapper Methods**
- We use a wrapper method after choosing the ML algorithm to use.
- For each feature subset, we estimate the algorithm's performance by training and evaluating it using only the features in a subset.
- Then, we add or remove features based on the estimate.
- This is an iterative process.



Wrapper Method

We use a greedy strategy to form feature subsets.

- In **forward wrapper methods**, we start from an empty feature set and add the feature maximizing the performance in each step until no substantial improvement is observed.
- So, if there are **n** features, we build **n** ML models in the first iteration.
- Then, we select the feature corresponding to the model with the best performance.
- In the second iteration, we repeat the process with the remaining **n-1** features.
- We continue like this as long as there's a significant performance improvement between models with which we end successive iterations.

Wrapper Method

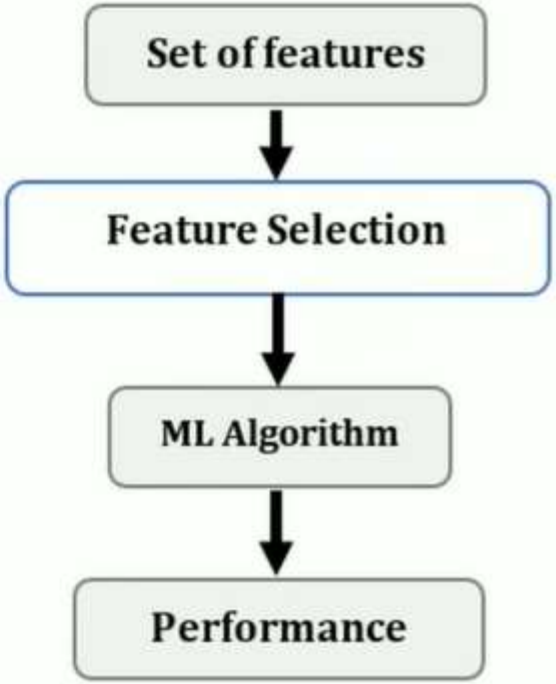
We use a greedy strategy to form feature subsets.

- **Backward methods** work the opposite way.
- They start from the full feature set and remove them one by one.
- Finally, stepwise methods reconsider features.
- So, in each iteration, they can remove a feature previously added as well as add a feature discarded in a previous step.

Filter Methods

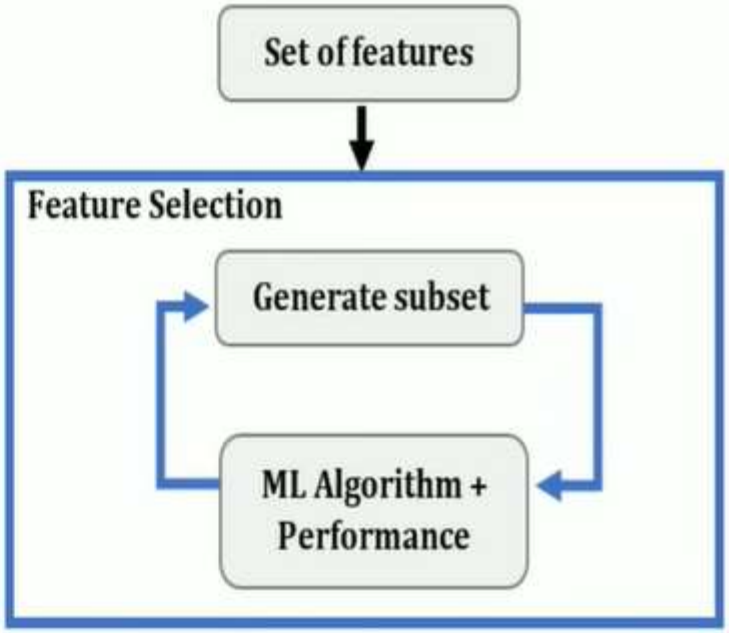
Filter Methods

- Filter methods use statistical tools to select feature subsets based on their relationship with the target.
- These methods remove features with low correlation with the target variable before training the final ML model.
- In doing so, they compute correlation and estimate the strength of the relationship using the, and other statistical tools.
- Chi-Square Test, Information Gain, Fisher's Score, Pearson correlation, ANOVA, variance thresholding



Embedded Methods

- **Intrinsic (or Embedded) Methods**
- Here selection of feature happens simultaneously with and is performed implicitly by the ML algorithm of our choice.
- During training, some steps of the ML algorithm do feature selection:
- For instance, this is the case with decision trees.
- At each node split, they choose the best feature to split the data by.
- Those choices represent feature selection.



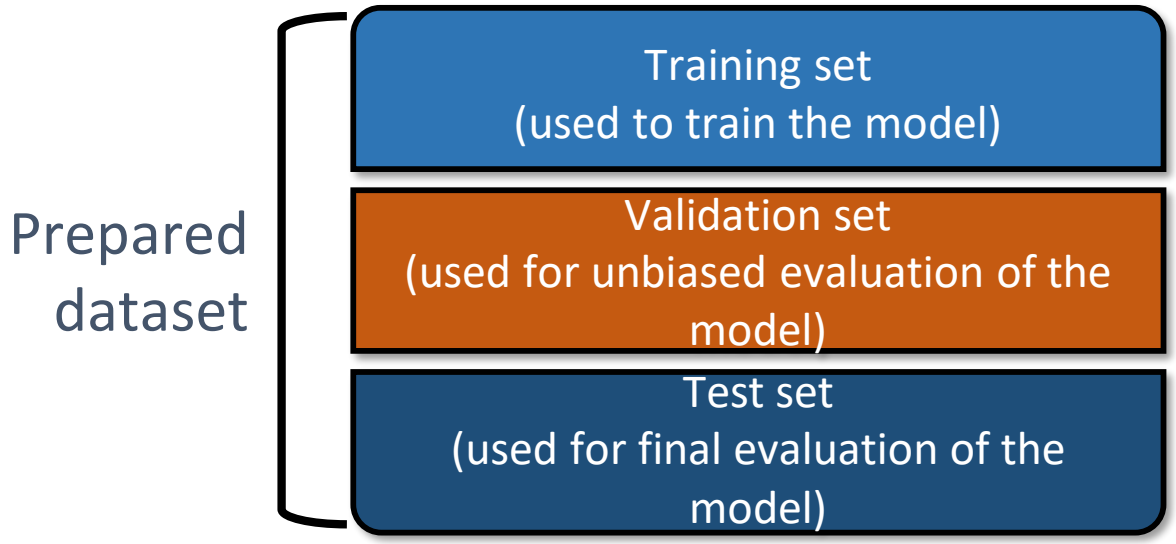
Advantages

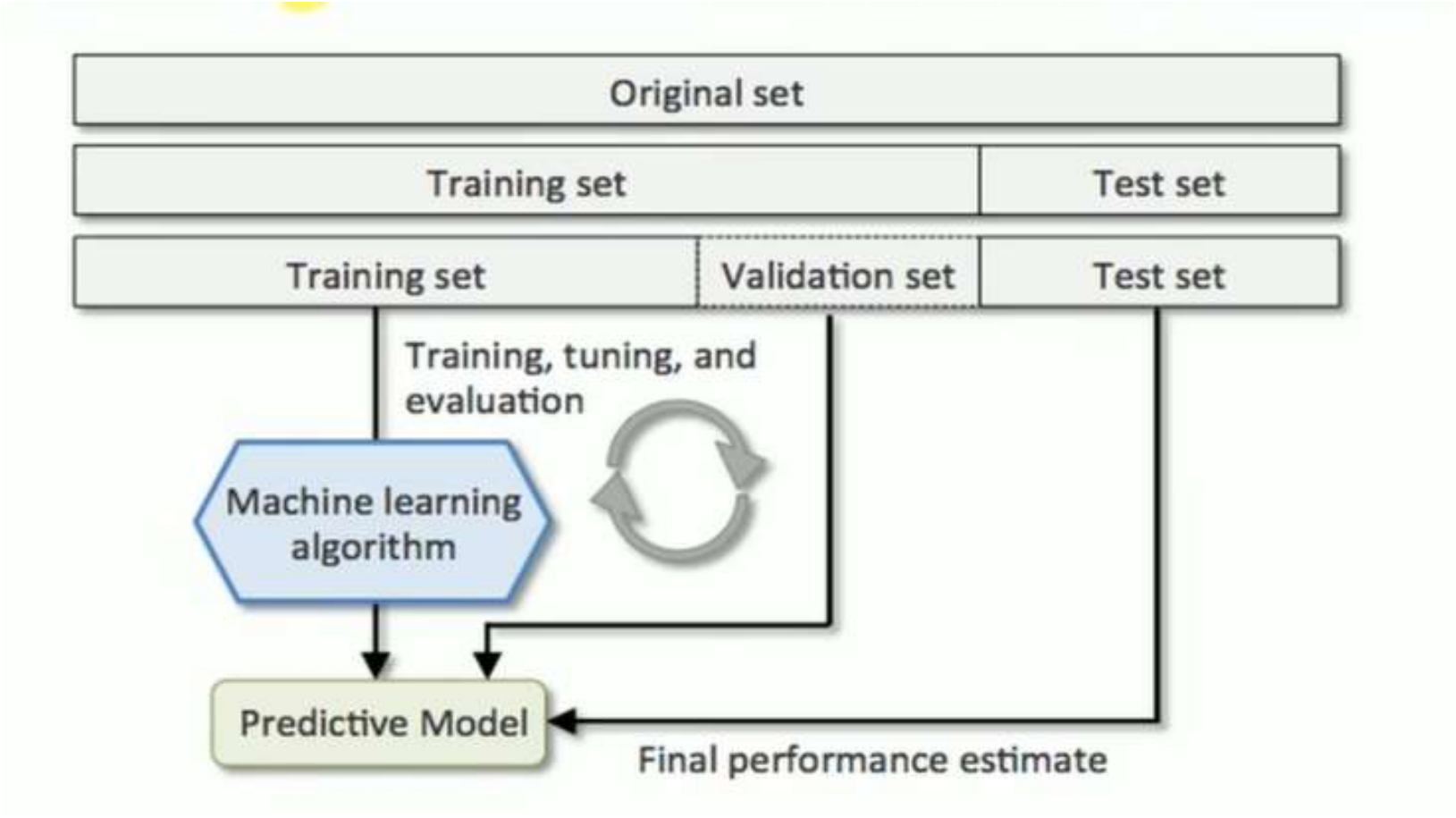
Feature selection methods allow us to:

- Reduce overfitting as less redundant data means less chance to make decisions based on noise;
- Improve accuracy by removing misleading and unimportant data;
- Reduce training time since data with fewer columns mean faster training.
- However, feature selection methods are hard to apply to high-dimensional data.
- The more features we have, the longer it takes for selection to complete.
- Also, there's the risk of overfitting when there aren't enough observations.

Data processing: Splitting datasets

When developing a machine learning model, one of the fundamental steps is to split your data into different subsets. These subsets are typically referred to as train, test, and validation data.




Original set
Training set
Test set
Training set
Validation set
Test set
Training, tuning, and evaluation
Machine learning algorithm
Predictive Model
Final performance estimate

Train Data

Training data is the subset of the dataset used to train the model. This is where the model learns patterns, relationships, and features of the data. During training, the algorithm adjusts its parameters based on this data to minimize error and improve its predictions.

- **Purpose**: To fit the model.
- **Process**: The model iteratively adjusts its parameters to better fit the training data, using algorithms like gradient descent.
- **Example**: If you're building a model to predict house prices, the training data might include numerous examples of houses with known prices, along with their features like size, location, and number of bedrooms.

Validation Data

Validation data is used to tune the model's hyperparameters and to provide an unbiased evaluation of the model while tuning. Hyperparameters are the aspects of the model that are not learned from the data but are set before the training process begins (e.g., learning rate, number of layers in a neural network).

- **Purpose**: To evaluate the model during training and assist in hyperparameter tuning.
- **Process**: After each training iteration (or epoch), the model's performance is evaluated on the validation set. This helps to prevent overfitting to the training data.
- **Example**: In the house price prediction example, the validation data would be a separate subset of house examples that are not seen by the model during training but are used to check the model's performance and tune hyperparameters.

Test Data

Test data is the subset of the dataset used to provide an unbiased evaluation of a final model fit on the training dataset. It is only used after the model has been trained (and validated).

- **Purpose**: To assess the final model's performance and generalization to unseen data.
- **Process**: Once the model has been trained and hyperparameters tuned, the test data is used to evaluate the model's performance. This step is crucial to understand how the model will perform in the real world.
- **Example**: For the house price prediction model, the test data would be another separate subset of houses that the model hasn't seen during training or validation, and their known prices would be compared against the model's predictions

- **The training set teaches the model, the validation set tunes it, and the test set delivers the final unbiased performance verdict.**
- **A common starting split is 80% training, 10% validation, and 10% test — adjust based on dataset size and use case.**
- **Reusing validation data as test data causes overfitting; keeping the test set completely unseen preserves evaluation integrity.**
- **K-fold cross-validation maximizes limited datasets by rotating which portion serves as the validation split across multiple runs.**

Data Part	Typical Percentage	Primary Purpose
Training Set	70% - 80%	Algorithm learning and weight optimization
Testing Set	20% - 30%	Final model evaluation and accuracy metrics
Validation Set	Optional	Hyperparameter tuning and architectural changes

Why Split Data?

1. **Prevent Overfitting**: By splitting the data, you ensure that the model isn't just memorizing the training data but is able to generalize well to unseen data.
2. **Unbiased Evaluation**: Using a separate test set provides an unbiased evaluation of the model's performance.
3. **Hyperparameter Tuning**: Validation sets help in tuning the model's hyperparameters to improve performance.

Cross Validation

Cross-validation is a crucial technique in machine learning used to evaluate the performance of a model. It helps ensure that the model generalizes well to unseen data. There are several types of cross-validation, each with its own methodology and use cases.

- The idea of cross-validation arises because of the *problems* with train test model or train test validation model.
- It basically wants to guarantee that the score of our model does not depend on the way we picked the train and test set.

Following are the type of Cross Validation Techniques

- K-folds
- Stratified K-folds
- Leave-one-out
- Leave-p-out

K- Fold Cross Validation

In K-Fold Cross-Validation, the dataset is divided into K equally-sized folds. The model is trained on K-1 folds and validated on the remaining fold. This process is repeated K times, with each fold used exactly once as the validation set. The final performance metric is the average of the K validation scores.

Advantages:

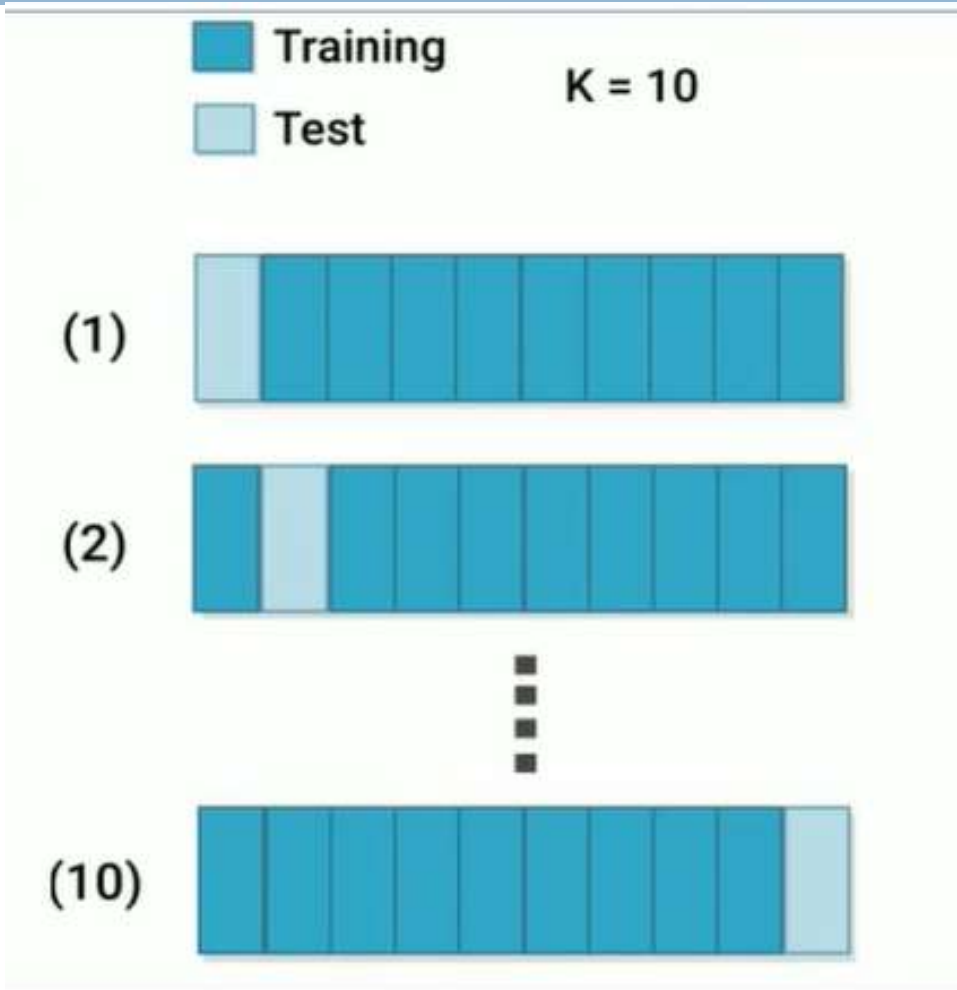
- More reliable estimate of model performance.
- Reduces variance compared to a single holdout set.

Disadvantages:

- Computationally more expensive than holdout validation.
- Not suitable for time-series data without modification.
- Not suitable for imbalanced datasets

K- Fold Cross Validation

- Entire dataset into divided into 10 folds
- Train model 10 times
- In each iteration 1 fold is considered for testing and remaining 9 for training
  - first iteration- fold 1 for testing
  - second iteration- fold 2 for testing & so on
- Take the avg-> final performance



Overfitting, Underfitting

When we're working on a machine learning project, we often face situations where we are encountering unexpected performance or error rate differences between the training set and the test set (as shown).

```
1 from sklearn.metrics import classification_report
2 print(classification_report(y_train, predicted_values))
```

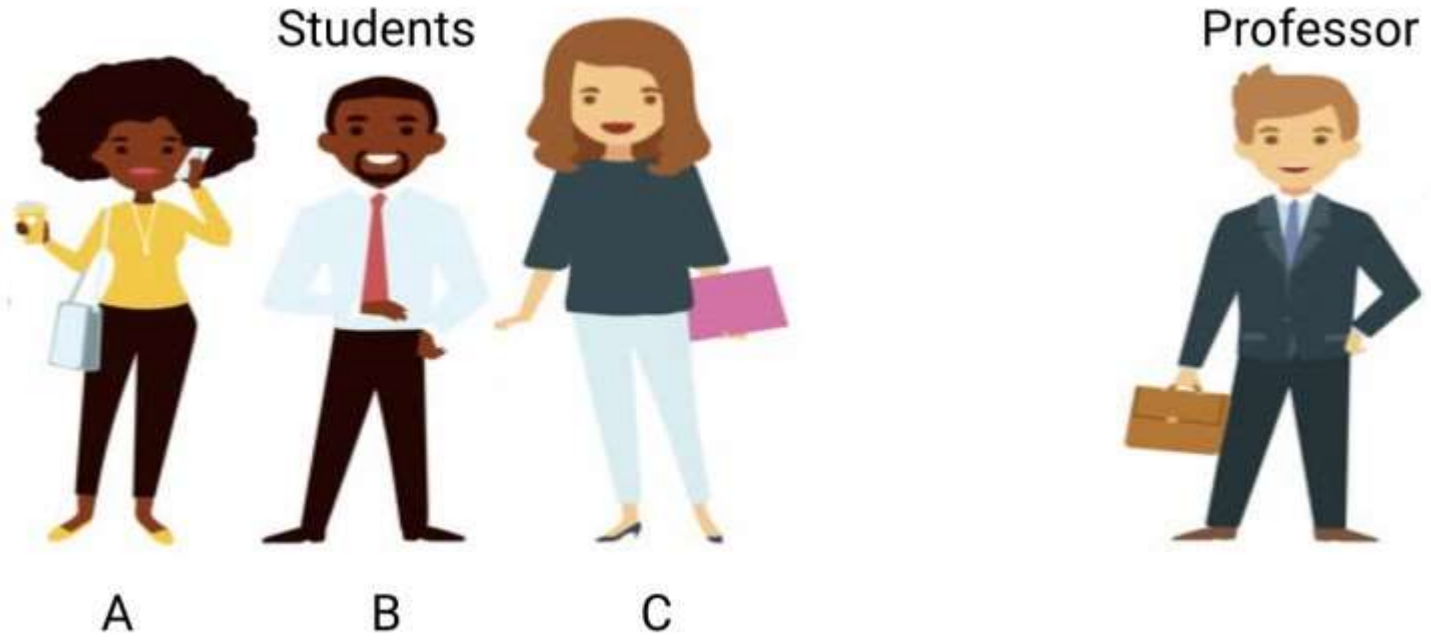
	precision	recall	f1-score	support
0	1.00	1.00	1.00	14234
1	1.00	1.00	1.00	3419
accuracy			1.00	17653
macro avg	1.00	1.00	1.00	17653
weighted avg	1.00	1.00	1.00	17653

```
1 predicted_values = classifier.predict(x_test)
2 print(classification_report(y_test, predicted_values))
```

	precision	recall	f1-score	support
0	0.87	0.86	0.87	3559
1	0.44	0.46	0.45	855
accuracy			0.78	4414
macro avg	0.66	0.66	0.66	4414
weighted avg	0.79	0.78	0.79	4414

Example

- Consider a math class consisting of 3 students and a professor.



Example

- Now, in any classroom, we can broadly divide the students into 3 categories. We'll talk about them one-by-one.



A

- Hobby = chating
- Not interested in class
- Doesn't pay much attention to professor

- Let's say that student A resembles a student who does not like math. She is not interested in what is being taught in the class and therefore does not pay much attention to the professor and the content he is teaching.

Example

- Let’s consider student B. He is the most competitive student who focuses on memorizing each and every question being taught in class instead of focusing on the key concepts. Basically, he isn’t interested in learning the problem-solving approach.



B

- Hobby = to be best in class.
- Mugs up everything professor says.
- Too much attention to the class work.

Example

- Finally, we have the ideal student C. She is purely interested in learning the key concepts and the problem-solving approach in the math class rather than just memorizing the solutions presented.

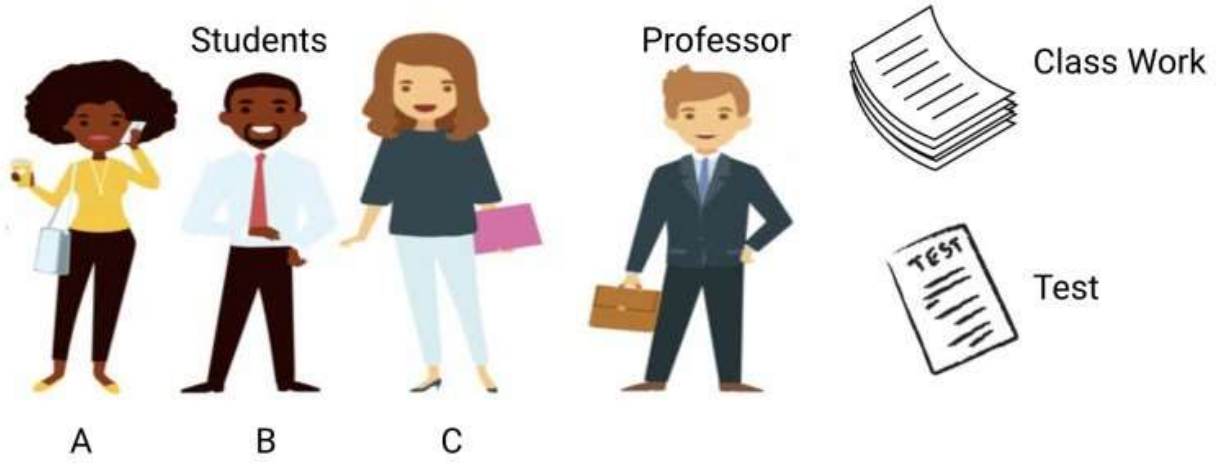


C

- Hobby = learning new things
- Eager to learn concepts.
- Pays attention to class and learns the idea behind solving a problem.

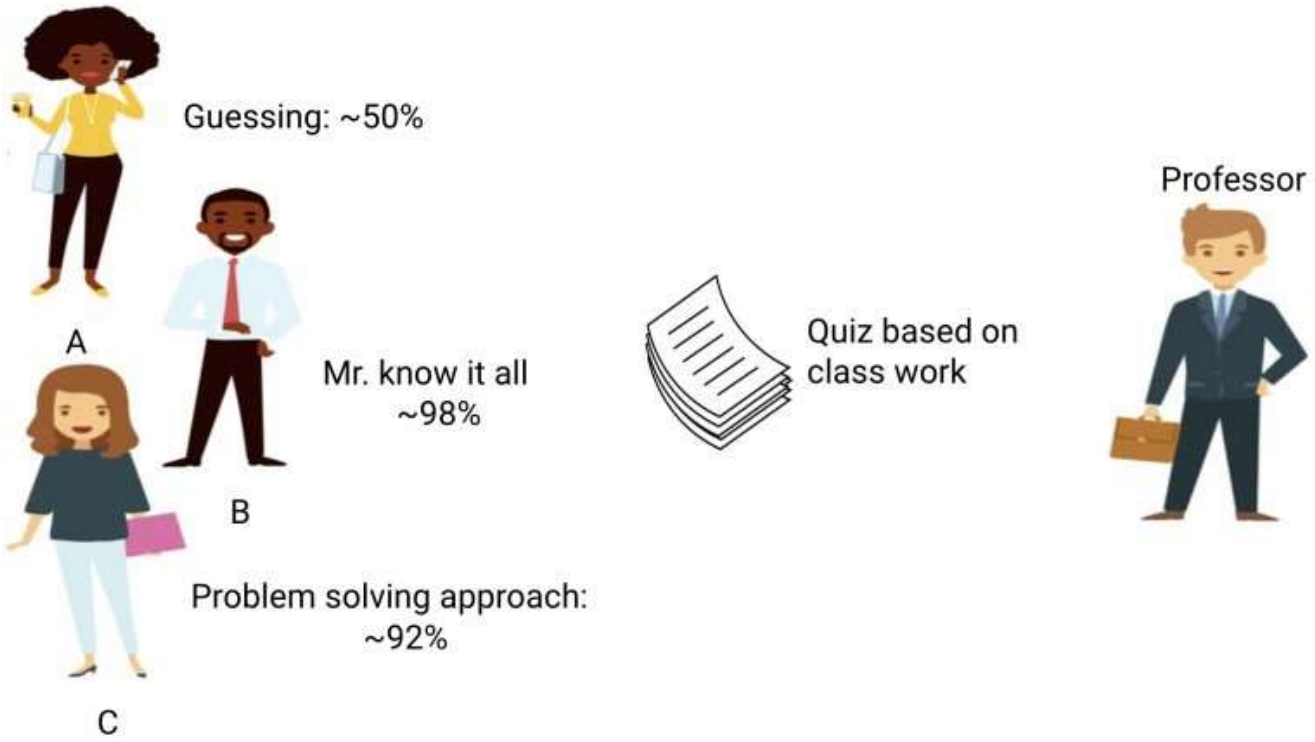
Example

- We all know from experience what happens in a classroom. The professor first delivers lectures and teaches the students about the problems and how to solve them. At the end of the day, the professor simply takes a quiz based on what he taught in the class.



Example

- So, let's discuss what happens when the teacher takes a classroom test at the end of the day:



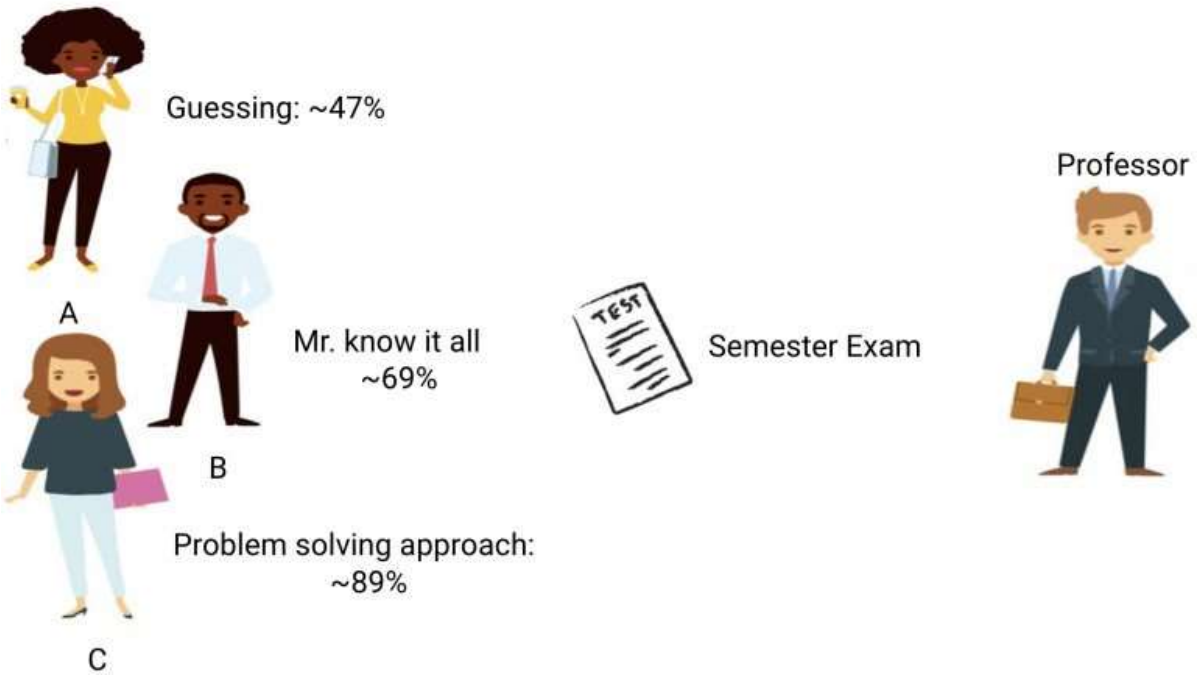
Example

- Student A, who was distracted in his own world, simply guessed the answers and got approximately 50% marks in the test
- On the other hand, the student who memorized each and every question taught in the classroom was able to answer almost every question by memory and therefore obtained 98% marks in the class test
- For student C, she actually solved all the questions using the problem-solving approach she learned in the classroom and scored 92%

tusharkute.com

Example

- Now here's the twist. Let's also look at what happens during the monthly test, when students have to face new unknown questions which are not taught in the class by the teacher.



Example

- In the case of student A, things did not change much and he still randomly answers questions correctly ~50% of the time.
- In the case of Student B, his score dropped significantly. Can you guess why? This is because he always memorized the problems that were taught in the class but this monthly test contained questions which he has never seen before. Therefore, his performance went down significantly
- In the case of Student C, the score remained more or less the same. This is because she focused on learning the problem-solving approach and therefore was able to apply the concepts she learned to solve the unknown questions.

Example



A

Not interested in learning

Class test ~50%
Test ~47%



B

Memorizing the lessons

Class test ~98%
Test ~69%

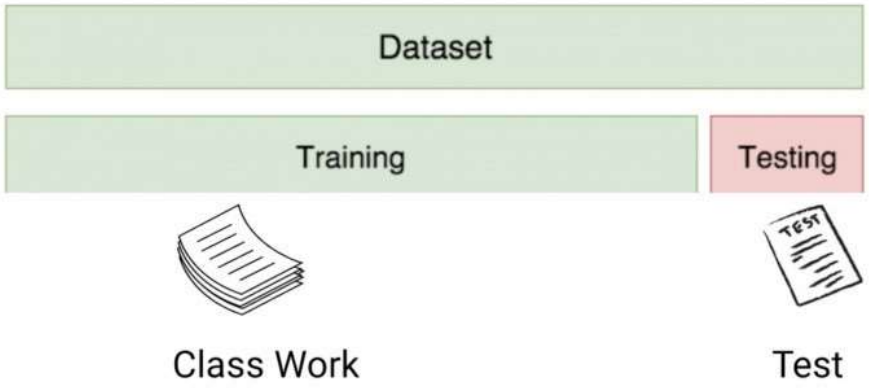


C

Conceptual Learning

Class test ~92%
Test ~89%

Example



It gave a perfect score over the training set but struggled with the test set. Comparing that to the student examples we just discussed, the classifier establishes an analogy with student B who tried to memorize each and every question in the training set.

Conclusion



A

Not interested in learning

Class test ~50%
Test ~47%

Under-fit/ biased learning



B

Memorizing the lessons

Class test ~98%
Test ~69%

Over-fit/ Memorizing



C

Conceptual Learning

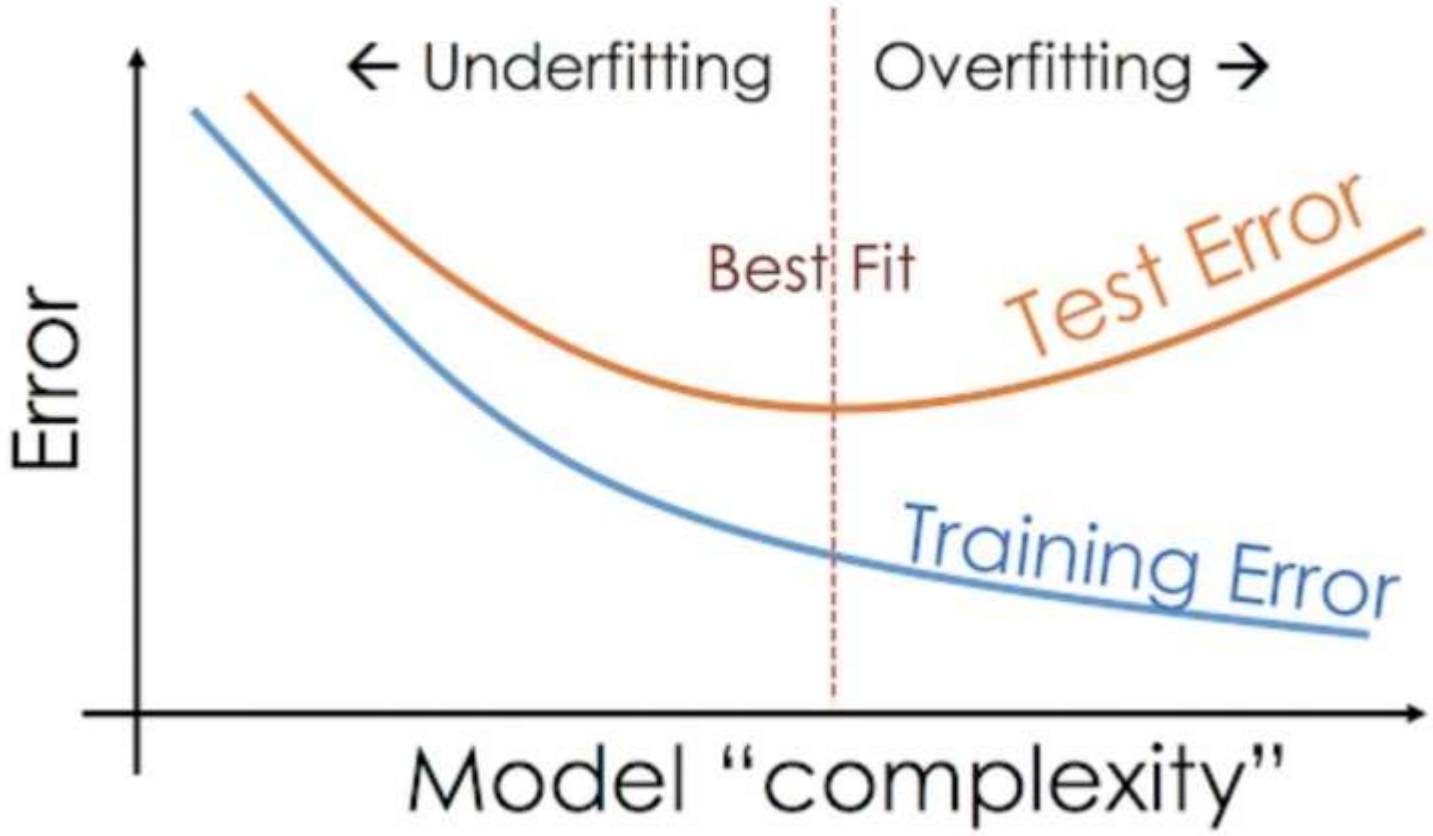
Class test ~92%
Test ~89%

Best-fit

Overfitting

- This situation where any given model is performing too well on the training data but the performance drops significantly over the test set is called an overfitting model.
- For example, non-parametric models like decision trees, KNN, and other tree-based algorithms are very prone to overfitting. These models can learn very complex relations which can result in overfitting.

Concept


← Underfitting
Overfitting →
Best Fit
Test Error
Training Error
Error
Model "complexity"

Underfitting

- On the other hand, if the model is performing poorly over the test and the train set, then we call that an underfitting model.
- An example of this situation would be building a linear regression model over non-linear data.

Overfitting

- Overfitting refers to a model that models the training data too well.
- Overfitting happens when a model learns the detail and noise in the training data to the extent that it negatively impacts the performance of the model on new data.
- This means that the noise or random fluctuations in the training data is picked up and learned as concepts by the model.
- The problem is that these concepts do not apply to new data and negatively impact the models ability to generalize.

Underfitting

- Underfitting refers to a model that can neither model the training data nor generalize to new data.
- An underfit machine learning model is not a suitable model and will be obvious as it will have poor performance on the training data.
- Underfitting is often not discussed as it is easy to detect given a good performance metric. The remedy is to move on and try alternate machine learning algorithms.
- Nevertheless, it does provide a good contrast to the problem of overfitting.

A good fit

- Ideally, you want to select a model at the sweet spot between underfitting and overfitting.
- This is the goal, but is very difficult to do in practice.
- To understand this goal, we can look at the performance of a machine learning algorithm over time as it is learning a training data.
- We can plot both the skill on the training data and the skill on a test dataset we have held back from the training process.

Limiting overfitting

- There are two important techniques that you can use when evaluating machine learning algorithms to limit overfitting:
  - Use a resampling technique to estimate model accuracy.
  - Hold back a validation dataset.

Bias-Variance

Bias and variance are two key concepts that explain the errors a machine learning model can make during prediction.

- **Bias:** Error caused by overly simple assumptions in the model, which may lead to underfitting.
- **Variance:** Error caused by the model being too sensitive to training data, which may lead to overfitting.
- **Goal:** Balance bias and variance so the model captures patterns while still generalizing well to new data.

Bias Variance Tradeoff

The bias variance tradeoff describes the balance between a model being too simple and too complex.

A simple model may miss important patterns (high bias), while a very complex model may learn noise from training data (high variance).

The aim is to balance both so the model performs well on new data.

- Simple models usually have high bias and low variance, which may cause underfitting.
- Complex models usually have low bias but high variance, which may cause overfitting.
- Balanced model achieves an optimal point where both bias and variance are reasonably low.

- Goal of machine learning is to minimize the total prediction error on unseen data.

The total prediction error can be expressed as:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Here:

- **Bias²:** Error caused by incorrect assumptions in the model.
- **Variance:** Error caused by sensitivity to training data.
- **Irreducible Error:** Random noise in the data that cannot be eliminated.

Underfitting mainly occurs due to high bias:

- High bias means model makes strong assumptions
- Ignores patterns
- Learns an overly simple representation
- Variance is low because the model gives similar outputs even if the data changes

Underfitting = High Bias + Low Variance

Overfitting is mainly caused by high variance:

- High variance means model reacts too strongly to training data
- Learns noise as patterns
- Low bias because the model is extremely flexible

Overfitting = Low Bias + High Variance

- **Underfitting (High Bias):** A model that is too simple (like a straight line for curved data) misses key patterns and performs poorly on both training and testing data.
- **Overfitting (High Variance):** A model that is too complex (like a high-degree polynomial) learns noise, fits training data too closely, and performs poorly on new data.
- **Ideal Fit (Balanced):** A moderately complex model captures the main trend without following noise, giving good performance on both training and testing data.



tusharkute.com

Techniques to Reduce Underfitting

- Use a more complex model
- Add new features and perform feature engineering
- Reduce regularization
- Train for more epochs
- Scale features properly

Techniques to Reduce Overfitting

- Collect more training data
- Reduce model complexity
- Use regularization (L1/L2)
- Apply dropout (for neural networks)
- Use early stopping
- Clean noisy data

Importance of Bias Variance Tradeoff

- Helps models perform well on unseen data.
- Helps choose appropriate algorithms and complexity.
- Helps achieve better model performance.

Error Metrics in Regression Problems

Regression is a supervised learning technique used to model and analyze the relationship between input variables (features) and a continuous output variable (target). The primary objective of a regression model is to make accurate numerical predictions.

- They quantify the prediction error of regression models
- Different metrics emphasize different error characteristics
- Some metrics penalize large errors more than small ones
- Metric selection depends on problem requirements and data nature
- They help in model selection and optimization

Types of Regression Metrics

1. Mean Absolute Error (MAE)

Mean Absolute Error measures the average absolute difference between actual and predicted values. It treats all errors equally, regardless of their direction and provides results in the same unit as the target variable, making it easy to interpret.

Useful when all errors should be trea

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

n = total number of observations

y_i = actual value

$\hat{y}_i$ = predicted value

tusharkute.com

2. Mean Squared Error (MSE)

Mean Squared Error calculates the average of squared differences between actual and predicted values. By squaring errors, it penalizes larger mistakes more strongly, making it sensitive to outliers

Used as loss function during model training

.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where:

- n = number of observations
- y_i = actual value
- $\hat{y}_i$ = predicted value

tusharkute.com

3. Root Mean Squared Error (RMSE)

Root Mean Squared Error is the square root of MSE. It maintains the strong penalty for large errors while converting the result back to the original unit of the target variable, improving interpretability.

more importance to large errors

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Where:

- n = number of observations
- y_i = actual value
- $\hat{y}_i$ = predicted value

4. R-squared (R^2 Score)

R-squared represents the proportion of variance in the target variable that is explained by the regression model. It provides insight into how well the model captures underlying data patterns.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Where:

- $\bar{y}$ = mean of actual values
- Numerator = residual sum of squares
- Denominator = total sum of squares

tusharkute.com

5. Mean Absolute Percentage Error (MAPE)

Mean Absolute Percentage Error expresses prediction error as a percentage of the actual values.

Easy to explain to non technical users

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Where,

- n: total number of observations
- y_i: actual value
- $\hat{y}_i$: predicted value

tusharkute.com

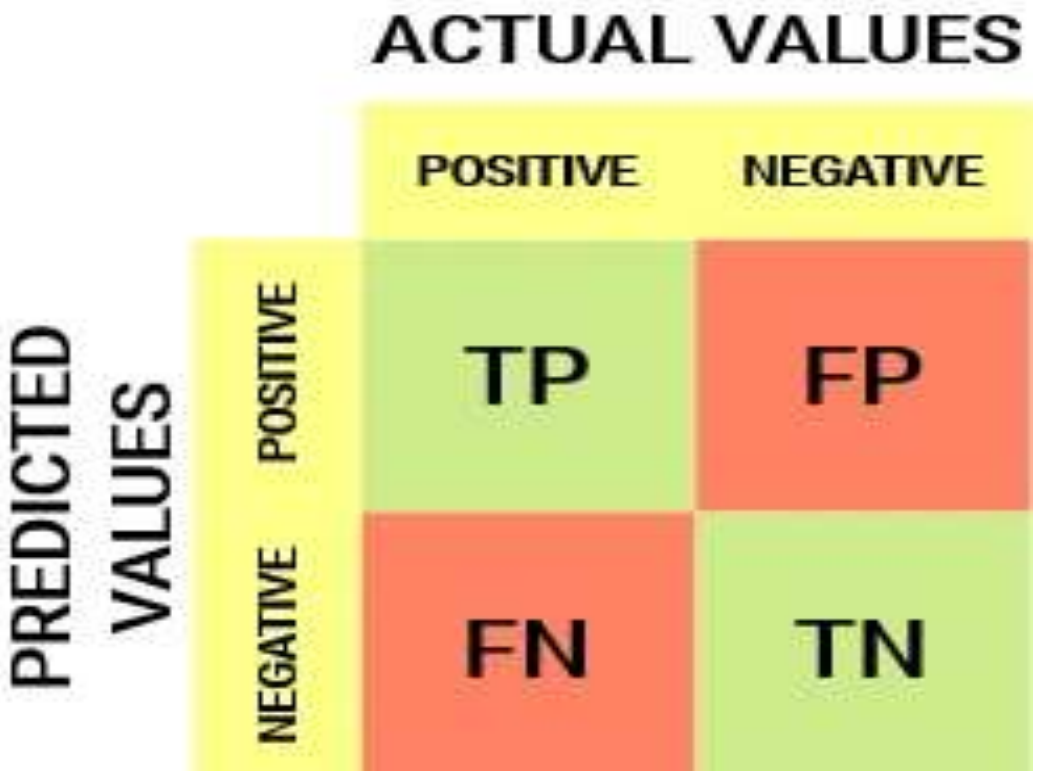
Metric	When to Use
MAE	You want the **average size of the prediction error** and want an easy-to-understand value
MSE	**Large errors are particularly important** and should be penalized strongly
RMSE	Large errors matter, but you want the result in the **same unit as the target**
R^2	You want to know **how well the model explains the variation** in the target
MAPE	You want to express prediction error as a **percentage**

tusharkute.com

Error Measurement in Classification Problems

Confusion matrix is a simple table used to measure how well a classification model is performing.

It compares the predictions made by the model with the actual results and shows where the model was right or wrong.



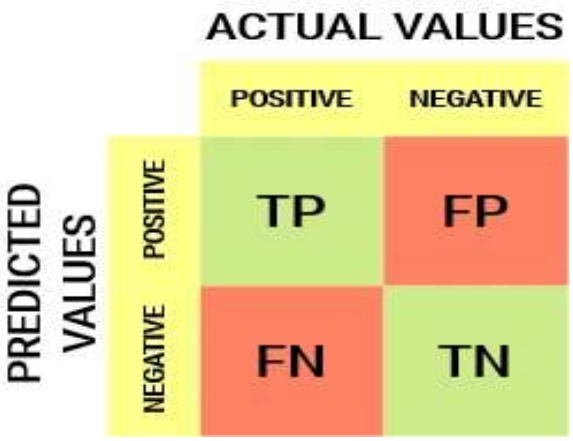
Metrics based on Confusion Matrix Data

1. Accuracy

Accuracy shows how many predictions the model got right out of all the predictions.

It gives idea of overall performance but it can be misleading when one class is more dominant over the other.

For example a model that predicts the majority class correctly most of the time might have high accuracy but still fail to capture important details about other classes.



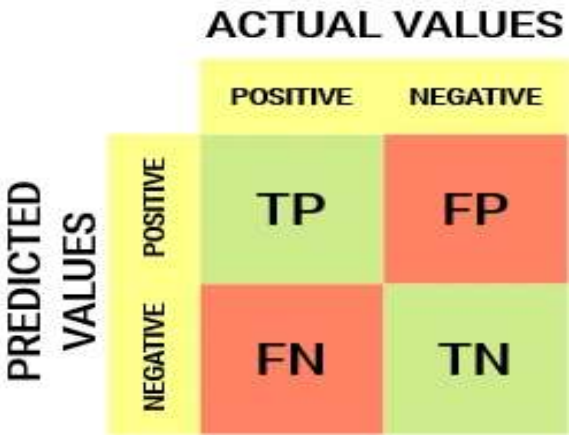
$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

2. Precision

Precision focus on the quality of the model's positive predictions.

It tells us how many of the "positive" predictions were actually correct.

It is important in situations where false positives need to be minimized such as detecting spam emails or fraud. The formula of precision is:



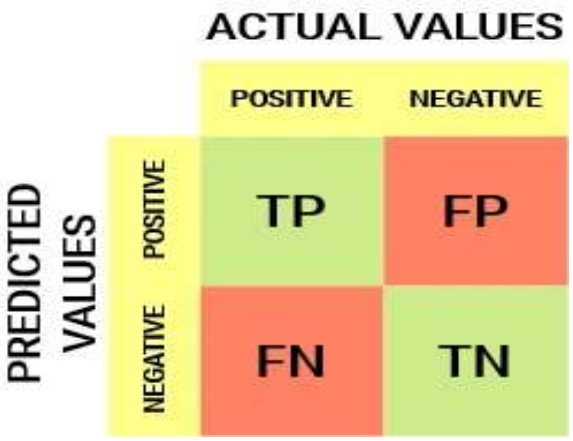
$$\text{Precision} = \frac{TP}{TP+FP}$$

3. Recall

Recall measures how good the model is at predicting positives.

It shows the proportion of true positives detected out of all the actual positive instances.

High recall is essential when missing positive cases has significant consequences like in medical tests.



$$\text{Recall} = \frac{TP}{TP+FN}$$

4. F1-Score

F1-score combines precision and recall into a single metric to balance their trade-off.

It provides a better sense of a model's overall performance particularly for imbalanced datasets.

It is helpful when both false positives and false negatives are important though it assumes precision and recall are equally important but in some situations one might matter more than the other.

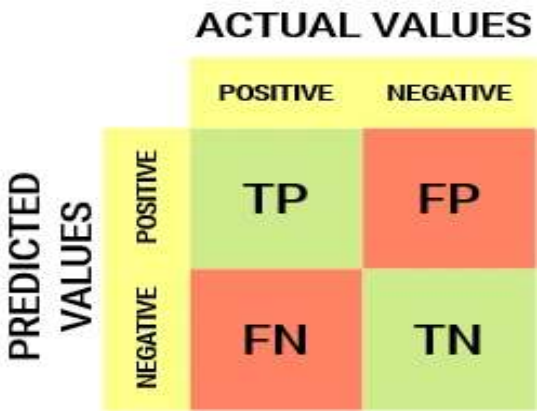
$$\text{F1-Score} = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

5. Specificity

Specificity is another important metric particularly in binary classification.

It measures the ability of a model to correctly identify negative instances.

Specificity is also known as the True Negative Rate



$$\text{Specificity} = \frac{TN}{TN+FP}$$

6. Type 1 and Type 2 error

Type 1 error: It occurs when the model incorrectly predicts a positive instance but the actual instance is negative. This is also known as a **false positive**. Type 1 Errors affect the **precision** of a model which measures the accuracy of positive predictions.

Type 2 error: This occurs when the model fails to predict a positive instance even though it is actually positive. This is also known as a **false negative**. Type 2 Errors impact the **recall** of a model which measures how well the model identifies all actual positive cases.

$$\text{Type 1 Error} = \frac{FP}{FP+TN}$$

$$\text{Type 2 Error} = \frac{FN}{TP+FN}$$

Example: A diagnostic test is used to detect a particular disease in patients.

- **Type 1 Error (False Positive):** This occurs when the test predicts a patient has the disease (positive result) but the patient is actually healthy (negative case).
- **Type 2 Error (False Negative):** This occurs when the test predicts the patient is healthy (negative result) but the patient actually has the disease (positive case).

Example: Confusion Matrix for Dog Image Recognition with Numbers

Index	1	2	3	4	5	6	7	8	9	10
Actual	Dog	Dog	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Not Dog	Not Dog
Predicted	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Dog	Dog	Not Dog	Not Dog
Result	TP	FN	TP	TN	TP	FP	TP	TP	TN	TN

- Actual Dog Counts = 6
- Actual Not Dog Counts = 4
- True Positive Counts = 5
- False Positive Counts = 1
- True Negative Counts = 3
- False Negative Counts = 1

		Predicted	
		Dog	Not Dog
Actual	Dog	True Positive (TP =5)	False Negative (FN =1)
	Not Dog	False Positive (FP=1)	True Negative (TN=3)

Confusion Matrix

Machine Learning

Predicted & Actual values

Actual \ Predicted	No	Yes	
No	65 TN	5 FP	70
Yes	10 FN	90 TP	100
	75	95	170

$$\text{Accuracy} = \frac{TP+TN}{\text{Total}} = \frac{90+65}{170} = 0.91$$

$$\text{Error rate} = \frac{FP+FN}{\text{Total}} = \frac{5+10}{170} = 0.09$$

$$\text{Precision} = \frac{TP}{\text{Predicted Yes}} = \frac{90}{95} = 0.95$$

$$\text{Recall} = \frac{TP}{\text{Actual Yes}} = \frac{90}{100} = 0.9$$